

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI:

**XÂY DỰNG HỆ THỐNG QUẢN LÝ
VẬN HÀNH TÒA NHÀ – TOWNHUB**

Giảng viên hướng dẫn : ThS. Hoàng Nam Thắng

Nhóm sinh viên thực hiện (03 sinh viên):

1. Nguyễn Xuân Thuởng – MSSV: 0220766
2. Lương Thanh Tài
3. Nguyễn Xuân Khải

Lớp : 66KSCS

Ngành : Công nghệ thông tin

HÀ NỘI – 2026

Nhận xét của giảng viên hướng dẫn

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI KHOA CÔNG NGHỆ THÔNG TIN	CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM Độc lập - Tự do - Hạnh phúc
--	---

Phiếu nhận xét của giảng viên hướng dẫn

(Ban hành kèm theo Quyết định số /QĐ-ĐHXDHN ngày tháng năm 2026 của Hiệu trưởng Trường Đại học Xây dựng Hà Nội)

Họ và tên SV	Nguyễn Xuân Thường
0220766	Mã SV
Tên đề tài	Xây dựng hệ thống quản lý vận hành tòa nhà – TownHub
2026	Đợt
GV hướng dẫn	ThS. Hoàng Nam Thắng
Khoa Công nghệ thông tin	Đơn vị

I. Nội dung nhận xét

II. Kết luận

☐ Đồng ý cho bảo vệ ☐ Không đồng ý cho bảo vệ

Điểm đánh giá của GVHD (nếu có): /10

....., ngày tháng năm 2026

Giảng viên hướng dẫn

(Ký, ghi rõ họ tên)

Nhận xét của giảng viên phản biện

Mẫu DATN-05

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI KHOA CÔNG NGHỆ THÔNG TIN	CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM Độc lập - Tự do - Hạnh phúc
--	---

Phiếu phản biện đồ án tốt nghiệp

(Ban hành kèm theo Quyết định số /QĐ-ĐHXDHN ngày tháng năm 2026 của Hiệu trưởng Trường Đại học Xây dựng Hà Nội)

Họ và tên SV	Nguyễn Xuân Thường
0220766	Mã SV
Tên đề tài	Xây dựng hệ thống quản lý vận hành tòa nhà – TownHub
2026	Đợt
Người phản biện	Đơn vị
Ngày phản biện	/...../2026

- I. Nhận xét chung
- II. Đánh giá theo CLO

CLO	Nội dung/tiêu chí đánh giá	0	1	2	3	4	Nhận xét/minh chứng CLOa
	☐	☐	☐	☐	☐		
☐	☐	☐	☐	☐		CLOb	☐
☐	☐	☐	☐		CLOc		
☐	☐	☐				☐	☐

III. Kiến nghị của người phản biện

☐ Đồng ý cho bảo vệ ☐ Đề nghị chỉnh sửa trước bảo vệ ☐ Chưa đủ điều kiện bảo vệ

Các yêu cầu chỉnh sửa chính:

....., ngày tháng năm 2026

Người phản biện

(Ký, ghi rõ họ tên)

Tóm tắt

Tên đề tài: Xây dựng hệ thống quản lý vận hành tòa nhà TownHub.

Sinh viên thực hiện: Nguyễn Xuân Thường **Mã SV:** 0220766 **Lớp:** 66KSCS

Nhóm thực hiện: Nguyễn Xuân Thường – Lương Thanh Tài – Nguyễn Xuân Khải
(đồ án nhóm 03 sinh viên)

Quản lý vận hành tòa nhà, chung cư hiện nay còn nhiều bất cập khi thực hiện thủ công và rời rạc: khó theo dõi tài sản – thiết bị, bỏ sót lịch bảo trì, xử lý sự cố chậm, số liệu không khớp sổ kế toán; thông tin cư dân – căn hộ khó tra cứu, thu phí dễ sai sót, an ninh ra/vào thiếu công cụ giám sát và các dịch vụ tiện ích chưa được kết nối minh bạch. Đề tài xây dựng hệ thống quản lý vận hành tòa nhà TownHub nhằm số hóa và tập trung toàn bộ các nghiệp vụ trên một nền tảng thống nhất.

Đề tài được thực hiện theo nhóm 03 sinh viên. Hệ thống phát triển theo kiến trúc module hóa gồm phần Base dùng chung do cả nhóm xây dựng (xác thực – phân quyền, người dùng, thông báo, cấu hình, báo cáo) và ba phân hệ nghiệp vụ riêng: phân hệ Quản lý tài sản gồm Tài sản & bảo trì, Kho – Mua sắm – Nhà cung cấp và Sự cố & SLA (Nguyễn Xuân Thường - tác giả báo cáo); phân hệ Quản lý dân cư kèm an ninh AI nhận diện khuôn mặt (Lương Thanh Tài); và phân hệ Quản lý dịch vụ (Nguyễn Xuân Khải). Công nghệ sử dụng: backend .NET 8 (EF Core, PostgreSQL), frontend Next.js 16 + React 19, xác thực JWT/OAuth/MFA, Redis, Cloudinary, OCR hóa đơn bằng VietOCR và nhận diện khuôn mặt YuNet + SFace.

Kết quả, nhóm đã hoàn thiện hệ thống chạy được với đầy đủ giao diện và API, các phân hệ độc lập nhưng liên thông dữ liệu với nhau: quản lý trọn vẹn vòng đời tài sản (khấu hao, mã QR, điều chuyển, bảo trì, thanh lý, tích hợp kế toán), quy trình mua sắm PR→PO→hóa đơn kèm OCR và quản lý sự cố theo cam kết SLA; quản lý căn hộ – cư dân – phí và cảnh báo người lạ bằng nhận diện khuôn mặt; quản lý dịch vụ tiện ích với quy trình yêu cầu – cung ứng nhiều bước duyệt. Hệ thống được kiểm thử và triển khai bằng Docker/Fly.io.

Từ khóa: *quản lý vận hành tòa nhà, quản lý tài sản, bảo trì phòng ngừa, SLA, OCR*

hóa đơn, quản lý dân cư, nhận diện khuôn mặt, quản lý dịch vụ, .NET 8, Next.js.

Nhiệm vụ đồ án tốt nghiệp

Mẫu ĐATN-01

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI KHOA CÔNG NGHỆ THÔNG TIN	CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM Độc lập - Tự do - Hạnh phúc
--	---

Nhiệm vụ đồ án tốt nghiệp

(Ban hành kèm theo Quyết định số /QĐ-ĐHXDHN ngày tháng năm 2026 của Hiệu trưởng Trường Đại học Xây dựng Hà Nội)

Họ và tên sinh viên	Nguyễn Xuân Thường
Mã SV	
0220766	66KSCS
Lớp	Khóa
66	Công nghệ thông tin
Ngành / Chuyên ngành	Hệ đào tạo
Chính quy	Xây dựng hệ thống quản lý vận hành tòa nhà – TownHub
Tên đề tài ĐATN	Khóa
Công nghệ thông tin	Công nghệ phần mềm
Nhóm chuyên môn	GV hướng dẫn
ThS. Hoàng Nam Thắng	
Email / ĐT	GV đồng hướng dẫn (nếu có)
(không có)	(không có)
Doanh nghiệp/đơn vị phối hợp (nếu có)	Địa điểm thực hiện
Trường Đại học Xây dựng Hà Nội	Học kỳ đồ án tốt nghiệp, năm 2026
Thời gian thực hiện	Đợt ĐATN
2026	

1. Mục tiêu và yêu cầu của ĐATN

- Xây dựng hệ thống quản lý vận hành tòa nhà TownHub theo kiến trúc module hóa trên nền .NET 8, Next.js 16 và PostgreSQL.

- Phần do sinh viên đảm nhiệm: phân hệ Quản lý tài sản, gồm các nghiệp vụ Tài sản & bảo trì; Kho – Mua sắm – Nhà cung cấp (kèm OCR hóa đơn tự động); Quản lý sự cố & SLA.

- Ứng dụng AI/OCR (VietOCR, PaddleOCR) để số hóa hóa đơn; tự động hóa khâu hao, mã QR tài sản và quy trình mua sắm PR → PO.

2. Phạm vi, nội dung công việc và sản phẩm cần nộp

- Phân tích yêu cầu, thiết kế cơ sở dữ liệu (schema asset), thiết kế API và giao diện cho các phân hệ phụ trách.

- Cài đặt chức năng; tích hợp, huấn luyện và đánh giá mô hình OCR; kiểm thử hệ thống.

- Sản phẩm cần nộp: mã nguồn, cơ sở dữ liệu, hệ thống chạy được (demo), quyền thuyết minh ĐATN và biên bản kiểm thử.

3. Dữ liệu đầu vào, giả thiết, tiêu chuẩn/quy chuẩn, phần mềm/công cụ sử dụng

- Công cụ: .NET 8, Next.js 16, PostgreSQL, Redis, Cloudinary; AI/OCR: VietOCR, PaddleOCR, Gemini; draw.io, Git.

- Dữ liệu: dữ liệu vận hành tòa nhà mô phỏng; bộ dữ liệu hóa đơn tiếng Việt tổng hợp phục vụ huấn luyện OCR.

4. Kế hoạch/mốc tiến độ dự kiến

STT	Kế hoạch	Thời gian (Mốc)	Ghi chú
Khảo sát, phân tích yêu cầu; thiết kế kiến trúc & cơ sở dữ liệu	Tuần 1 – 2	2	Cài đặt nghiệp vụ Quản lý tài sản & bảo trì
Tuần 3 – 4	Mốc M1	Cài đặt Kho – Mua sắm – NCC; tích hợp & huấn luyện OCR hóa đơn	Tuần 5 – 6
4	Cài đặt Quản lý sự cố & SLA; tích hợp toàn hệ thống	Tuần 7	Mốc M2
Kiểm thử, hoàn thiện, viết thuyết minh và bảo vệ	Tuần 8 – 9		

Sinh viên <i>(Ký, ghi rõ họ tên)</i>	GV hướng dẫn <i>(Ký, ghi rõ họ tên)</i>	Chủ tịch Hội đồng <i>(Ký, ghi rõ họ tên)</i>
--	---	--

Lời nói đầu

Chuyển đổi số trong quản lý vận hành tòa nhà, chung cư đang là nhu cầu tất yếu khi quy mô và mức độ phức tạp của các khu đô thị ngày càng tăng. Xuất phát từ thực tiễn đó, nhóm đã lựa chọn và thực hiện đề tài “Xây dựng hệ thống quản lý vận hành tòa nhà TownHub” làm đồ án tốt nghiệp, với mong muốn vận dụng kiến thức đã học để giải quyết một bài toán thực tế trọn vẹn.

Trong quá trình thực hiện, nhóm đã học hỏi và áp dụng nhiều công nghệ hiện đại, rèn luyện kỹ năng phân tích – thiết kế, lập trình, kiểm thử và làm việc nhóm. Hệ thống được phân chia thành các module độc lập nhưng liên thông: cả nhóm cùng phối hợp xây dựng phần lõi dùng chung (Base, Auth), trên nền tảng đó cá nhân em đảm nhiệm trọn vẹn phân hệ Quản lý tài sản (gồm các nghiệp vụ Tài sản & bảo trì, Kho – Mua sắm – Nhà cung cấp và Sự cố & SLA); một thành viên phát triển phân hệ Quản lý dân cư và một thành viên phát triển phân hệ Quản lý dịch vụ. Đồ án là kết quả của sự nỗ lực và phối hợp của cả nhóm cùng sự hướng dẫn tận tình của thầy cô.

Em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn đã định hướng, góp ý và đồng hành trong suốt quá trình thực hiện; cảm ơn quý thầy cô Khoa Công nghệ thông tin – Trường Đại học Xây dựng Hà Nội đã trang bị kiến thức nền tảng; và cảm ơn các thành viên trong nhóm đã phối hợp, hỗ trợ lẫn nhau. Do thời gian và kiến thức còn hạn chế, đồ án khó tránh khỏi thiếu sót, em rất mong nhận được sự góp ý của quý thầy cô.

Em xin chân thành cảm ơn!

Lời cam đoan

Em xin cam đoan đồ án tốt nghiệp “Xây dựng hệ thống quản lý vận hành tòa nhà Town-Hub” là công trình do nhóm tự nghiên cứu và thực hiện dưới sự hướng dẫn của giảng viên hướng dẫn. Các nội dung, số liệu, mã nguồn và kết quả trình bày trong đồ án là trung thực; phần do cá nhân em đảm nhiệm (phân hệ Quản lý tài sản gồm các nghiệp vụ Tài sản & bảo trì, Kho – Mua sắm – Nhà cung cấp, Sự cố & SLA) do chính em thực hiện.

Mọi tài liệu tham khảo đều được trích dẫn đầy đủ theo quy định. Em xin chịu hoàn toàn trách nhiệm nếu có vi phạm về liêm chính học thuật.

Sinh viên thực hiện

(Ký, ghi rõ họ tên)

Mục lục

Nhận xét của giảng viên hướng dẫn	i
Nhận xét của giảng viên phản biện	ii
Tóm tắt	v
Nhiệm vụ đồ án tốt nghiệp	vii
Lời nói đầu	x
Lời cam đoan	xi
Danh mục hình vẽ	xvi
Danh mục ký hiệu, chữ viết tắt	xviii
Mở đầu	1
1. Lý do chọn đề tài	1
2. Mục tiêu của đề tài	1
3. Đối tượng và phạm vi	2
4. Phương pháp thực hiện	2
5. Phân công công việc trong nhóm	2
6. Bố cục báo cáo	7
Chương 1. Tổng quan và khảo sát bài toán	8
1.1. Tổng quan về quản lý vận hành tòa nhà	8
1.1.1. Các bên liên quan và vai trò	8
1.1.2. Các nghiệp vụ chính	9
1.2. Khảo sát hiện trạng và vấn đề đặt ra	9

1.3. Khảo sát các giải pháp hiện có	10
1.4. Đề xuất giải pháp và kiến trúc module hoá	11
1.5. Mục tiêu, phạm vi và yêu cầu hệ thống	12
1.5.1. Yêu cầu chức năng	12
1.5.2. Yêu cầu phi chức năng	12
1.6. Kết luận chương	13
Chương 2. Cơ sở lý thuyết và công nghệ sử dụng	14
2.1. Kiến trúc phần mềm áp dụng	14
2.2. Công nghệ Backend (.NET 8 Web API)	14
2.3. Công nghệ Frontend (Next.js 16 + React 19)	15
2.4. Cơ sở dữ liệu PostgreSQL	15
2.5. Bảo mật và phân quyền (JWT, OAuth, MFA/TOTP, RBAC)	16
2.6. Hạ tầng cache, lưu trữ và email	16
2.7. Công nghệ AI/OCR (VietOCR) và xử lý nền	16
2.8. Công cụ quản lý dự án và triển khai	17
2.9. Kết luận chương	17
Chương 3. Phân tích và thiết kế module chung (Base)	18
3.1. Kiến trúc tổng thể hệ thống TownHub	18
3.2. Biểu đồ Use Case tổng quát	19
3.3. Thiết kế cơ sở dữ liệu lõi (schema auth)	20
3.4. Đặc tả các phân hệ Base	22
3.4.1. Auth & Security - Xác thực và phân quyền	22
3.4.2. User Management - Người dùng và nhật ký hoạt động	23
3.4.3. Notification Engine - Thông báo	23
3.4.4. Configuration - Cấu hình và danh mục dùng chung	24
3.4.5. Reporting - Dashboard và KPI	24
3.5. Thiết kế API dùng chung và điểm mở rộng cho phân hệ riêng	25
3.6. Kết luận chương	25
Chương 4. Phân tích và thiết kế phân hệ Quản lý tài sản (phần riêng)	26
4.1. Nghiệp vụ Tài sản & bảo trì	26
4.1.1. Đặt vấn đề và phạm vi	26
4.1.2. Mối liên hệ và kế thừa từ module Base	27
4.1.3. Phân tích yêu cầu chức năng	27

4.1.4. Yêu cầu phi chức năng	29
4.1.5. Thiết kế phân hệ	29
4.1.6. Thiết kế giao diện	36
4.1.7. Thành phần thông minh: sinh mã & khấu hao tự động	38
4.2. Nghiệp vụ Kho, Mua sắm và Nhà cung cấp	38
4.2.1. Đặt vấn đề và phạm vi	38
4.2.2. Mối liên hệ với Base và các phân hệ khác	39
4.2.3. Phân tích yêu cầu chức năng	39
4.2.4. Yêu cầu phi chức năng	44
4.2.5. Thiết kế phân hệ	44
4.2.6. Thiết kế giao diện	47
4.2.7. Chi tiết kỹ thuật OCR hóa đơn: mô hình, huấn luyện và đánh giá	49
4.3. Nghiệp vụ Quản lý sự cố và SLA	63
4.3.1. Đặt vấn đề và phạm vi	63
4.3.2. Mối liên hệ với Base và các phân hệ khác	64
4.3.3. Phân tích yêu cầu chức năng	64
4.3.4. Yêu cầu phi chức năng	66
4.3.5. Thiết kế phân hệ	66
4.3.6. Thiết kế giao diện	68
4.4. Kết luận chương	70
Chương 5. Triển khai, kiểm thử và đánh giá	71
5.1. Môi trường và công nghệ triển khai	71
5.2. Kết quả triển khai module Base	72
5.3. Kết quả triển khai phân hệ Quản lý tài sản (phần riêng)	73
5.4. Kiểm thử hệ thống	74
5.5. Đánh giá hệ thống	75
5.6. Đánh giá đạo đức, an toàn dữ liệu và bối cảnh xã hội	75
5.7. Kết luận chương	76
Kết luận	77
1. Kết quả đạt được	77
2. Hạn chế	77
3. Hướng phát triển	77
4. Đóng góp của cá nhân	78

Tài liệu tham khảo	79
Phụ lục	80

Danh mục hình vẽ

Hình 0.1. Kế hoạch – tiến độ nhóm (biểu đồ Gantt)	4
Hình 0.2. Bảng công việc Kanban của nhóm	4
Hình 0.3. Hợp nhóm trực tuyến (đang trình bày tiến độ)	5
Hình 0.4. Trao đổi công việc trên nhóm chat	5
Hình 0.5. Biên bản họp nhóm (bản mẫu)	6
Hình 3.1. Kiến trúc tổng thể hệ thống TownHub	19
Hình 3.2. Biểu đồ Use Case tổng quát module Base	20
Hình 3.3. Sơ đồ thực thể – liên kết (ERD) schema auth	21
Hình 3.4. Biểu đồ tuần tự: Đăng nhập có MFA và cấp JWT	22
Hình 3.5. Giao diện màn hình đăng nhập	23
Hình 3.6. Giao diện Bảng điều khiển (Dashboard) tổng quan	24
Hình 4.1. Biểu đồ Use Case – Quản lý tài sản & bảo trì	30
Hình 4.2. Biểu đồ tuần tự: Ghi tăng tài sản	31
Hình 4.3. Lưu đồ nghiệp vụ kế toán tài sản: ghi tăng – khấu hao – thanh lý . .	32
Hình 4.4. Lưu đồ nghiệp vụ: Bảo trì định kỳ (PM/Work Order)	33
Hình 4.5. Sơ đồ thực thể – liên kết (ERD) schema asset	34
Hình 4.6. Giao diện Danh sách tài sản	37
Hình 4.7. Giao diện Chi tiết tài sản (thông số, khấu hao, lịch sử)	37
Hình 4.8. Giao diện Lệnh công việc bảo trì (Work Order)	38
Hình 4.9. Lưu đồ nghiệp vụ: Kiểm kê kho định kỳ	40
Hình 4.10. Lưu đồ nghiệp vụ: Quy trình mua sắm PR → PO và OCR hóa đơn	41
Biểu đồ tuần tự use case Trích xuất hóa đơn tự động	44
Hình 4.11. Biểu đồ Use Case – Kho, Mua sắm và Nhà cung cấp	45
Hình 4.12. Sơ đồ thực thể – liên kết (ERD) phân hệ Kho, Mua sắm và Nhà cung cấp	46
Hình 4.13. Giao diện Tồn kho vật tư (cảnh báo tồn thấp)	48
Hình 4.14. Giao diện Upload OCR hóa đơn	48

Hình 4.15. Giao diện Danh sách nhà cung cấp	49
Hình 4.16. Kiến trúc pipeline trích xuất hóa đơn (dịch vụ OCR bốn engine) . .	50
Hình 4.17. Luồng AI hai tầng (EasyOCR + VietOCR) so với PaddleOCR hợp nhất	53
Hình 4.18. Đường cong huấn luyện PaddleOCR (rec): loss và accuracy theo bước	55
Hình 4.19. Tác động của fine-tune (Sequence và Character accuracy)	57
Hình 4.20. So sánh ba engine theo Seq-acc / Char-acc / NED (tập val 300 mẫu)	57
Hình 4.21. Độ trễ suy luận (ms/ảnh) và thông lượng (ảnh/giây) trên CPU . . .	58
Hình 4.22. Tính hỗ trợ giữa các engine (mẫu nào engine nào đọc đúng) . . .	58
Hình 4.23. Sơ đồ khối thuật toán bóc tách hóa đơn kháng nghiêng (deskew). .	60
Hình 4.24. Luồng xử lý AI end-to-end của dịch vụ OCR: các lớp xử lý và vị trí bước khử nghiêng.	62
Hình 4.25. Lưu đồ nghiệp vụ: Xử lý sự cố và giám sát SLA	65
Hình 4.26. Biểu đồ Use Case – Quản lý sự cố và SLA	66
Hình 4.27. Sơ đồ trạng thái vòng đời ticket sự cố	67
Hình 4.28. Sơ đồ thực thể – liên kết (ERD) phân hệ Quản lý sự cố & SLA . .	67
Hình 4.29. Giao diện Danh sách phiếu sự cố (CM)	69
Hình 4.30. Giao diện Bảng theo dõi SLA	69
Hình 5.1. Giao diện Quản trị Tài khoản người dùng	72
Hình 5.2. Giao diện Vai trò & phân quyền (RBAC)	73
Hình 5.3. Nhật ký lỗi & kiểm thử trên Excel (sheet Lỗi Base)	75

Danh mục ký hiệu, chữ viết tắt

STT	Từ viết tắt	Diễn giải
API / REST	Application Programming Interface / Representational State Transfer	JWT / OAuth
JSON Web Token / Open Authorization	MFA / TOTP	Multi-Factor Authentication / Time-based One-Time Password
RBAC	Role-Based Access Control (phân quyền theo vai trò)	EF Core / ORM
Entity Framework Core / Object-Relational Mapping	PM / CM	Preventive Maintenance / Complaint (Incident) Management
WO	Work Order (lệnh/phiếu công việc bảo trì)	SLA
Service Level Agreement (cam kết mức dịch vụ)	PR / PO	Purchase Request / Purchase Order
NCC	Nhà cung cấp	OCR
Optical Character Recognition (VietOCR)	MTTR / MTBF	Mean Time To Repair / Mean Time Between Failures
KPI	Key Performance Indicator	UML / ERD
Unified Modeling Language / Entity Relationship Diagram		

Mở đầu

1. Lý do chọn đề tài

Cùng với tốc độ đô thị hóa, số lượng tòa nhà, chung cư và khu phức hợp ngày càng tăng, kéo theo nhu cầu quản lý vận hành ngày một phức tạp. Một tòa nhà điển hình vừa sở hữu hàng trăm tài sản và thiết bị kỹ thuật (thang máy, máy bơm, hệ thống PCCC, máy phát điện, thiết bị điện – nước...) cần được bảo trì và xử lý sự cố liên tục, vừa phục vụ hàng nghìn cư dân với các nhu cầu về chỗ ở, phí dịch vụ, an ninh ra/vào và dịch vụ tiện ích hằng ngày.

Thực tế nhiều đơn vị vẫn quản lý thủ công bằng sổ sách hoặc bảng tính rời rạc, dẫn tới khó tra cứu, thất lạc thông tin, tính khấu hao sai lệch, bỏ sót lịch bảo trì, xử lý sự cố chậm trễ; thông tin cư dân – căn hộ và việc thu phí dễ sai sót; an ninh ra/vào thiếu công cụ giám sát chủ động; việc kết nối cư dân với các dịch vụ tiện ích thiếu minh bạch. Từ nhu cầu đó, nhóm lựa chọn đề tài xây dựng hệ thống quản lý vận hành tòa nhà TownHub nhằm số hóa và tập trung các nghiệp vụ trên một nền tảng thống nhất, giúp minh bạch, tự động hóa và nâng cao hiệu quả vận hành.

2. Mục tiêu của đề tài

Đề tài hướng tới các mục tiêu:

- Mục tiêu chung: xây dựng hệ thống TownHub theo kiến trúc module - phần Base (lõi dùng chung: xác thực – phân quyền, người dùng, thông báo, cấu hình, báo cáo) do cả nhóm cùng phát triển và các phân hệ nghiệp vụ riêng của từng thành viên - chạy được, có giao diện và API hoàn chỉnh.
- Phần riêng của Nguyễn Xuân Thương (tác giả báo cáo): phân hệ Quản lý tài sản gồm các nghiệp vụ Tài sản & bảo trì (khấu hao, mã QR, điều chuyển, bảo trì), Kho – Mua sắm – Nhà cung cấp (kèm OCR hóa đơn) và Quản lý sự cố & SLA.

- Phần riêng của Lương Thanh Tài: phân hệ Quản lý dân cư (tòa nhà – tầng – căn hộ, cư dân, phí) và an ninh AI nhận diện khuôn mặt.
- Phần riêng của Nguyễn Xuân Khải: phân hệ Quản lý dịch vụ (dịch vụ tiện ích, nhà cung cấp dịch vụ, quy trình yêu cầu dịch vụ).
- Áp dụng công nghệ hiện đại (.NET 8, Next.js 16, PostgreSQL) và các thành phần thông minh (OCR hóa đơn, khẩu hao tự động, nhận diện khuôn mặt) để tăng tính thực tiễn của sản phẩm.

3. Đối tượng và phạm vi

Đối tượng sử dụng gồm: quản trị viên, ban quản lý vận hành, kế toán, kỹ thuật viên, thủ kho, nhân viên mua sắm, nhân viên an ninh, nhà cung cấp dịch vụ và cư dân. Về phạm vi, hệ thống bao phủ các nghiệp vụ cốt lõi của quản lý vận hành tòa nhà, chia thành phần Base dùng chung và ba phân hệ nghiệp vụ: (1) phân hệ Quản lý tài sản gồm Tài sản & bảo trì, Kho – Mua sắm – Nhà cung cấp, và Sự cố & SLA (Nguyễn Xuân Thương); (2) phân hệ Quản lý dân cư (Lương Thanh Tài); (3) phân hệ Quản lý dịch vụ (Nguyễn Xuân Khải). Báo cáo trình bày bức tranh tổng thể của hệ thống, trong đó đi sâu vào phần Base và phân hệ Quản lý tài sản do tác giả thực hiện.

4. Phương pháp thực hiện

Đề tài được thực hiện theo quy trình kỹ thuật chuyên nghiệp:

- Khảo sát thực tế nghiệp vụ vận hành tòa nhà và tham khảo các giải pháp hiện có.
- Phân tích và thiết kế hướng đối tượng bằng UML (use case, lớp, tuần tự, trạng thái) và mô hình dữ liệu ERD.
- Phát triển theo mô hình Agile/Scrum với kiến trúc phân tầng, tách module rõ ràng; quản lý mã nguồn bằng Git (nhánh riêng từng thành viên, review chéo khi hợp nhất).
- Kiểm thử qua Swagger và kiểm thử chức năng theo ca kiểm thử.

5. Phân công công việc trong nhóm

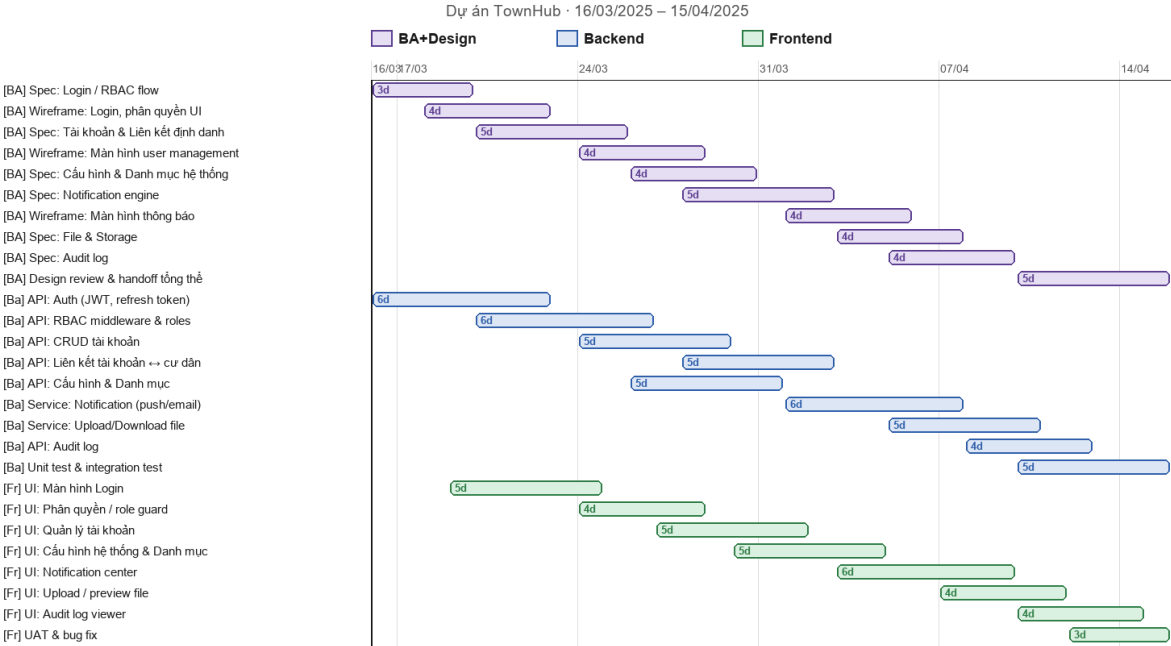
Hệ thống gồm phần Base dùng chung (cả nhóm cùng xây dựng) và các phân hệ riêng theo từng thành viên (bảng RACI):

Phân hệ / Chức năng	Loại	Phụ trách chính
Base (chung)	Cả nhóm	Auth & Security - đăng nhập, JWT, Google OAuth, MFA, RBAC
Cả nhóm	User Management - tài khoản, vai trò, quyền, phiên, Audit Log	Base (chung)
Notification Engine - thông báo, template, lịch sử	Base (chung)	Cả nhóm
Base (chung)	Reporting - dashboard, KPI, báo cáo chi phí	Configuration - cấu hình hệ thống, danh mục dùng chung
Cả nhóm	Riêng	Base (chung)
Quản lý tài sản & vận hành kỹ thuật (Tài sản, Bảo trì)	Nguyễn Xuân Thường	Nguyễn Xuân Thường
Riêng	Quản lý sự cố & SLA	Kho – Mua sắm – Nhà cung cấp (kèm OCR hóa đơn)
Nguyễn Xuân Thường	Riêng	Riêng
Quản lý dân cư (căn hộ, cư dân, phí)		Lương Thanh Tài
Riêng	Nguyễn Xuân Khải	Quản lý dịch vụ

Theo Điều 7 Quy định ĐATN, sản phẩm nhóm thể hiện rõ phần chung và phần riêng, mỗi thành viên chịu trách nhiệm phần cá nhân của mình. Báo cáo trình bày phần Base (chung) và phân hệ Quản lý tài sản do Nguyễn Xuân Thường đảm nhiệm (gồm các nghiệp vụ Tài sản & bảo trì, Kho – Mua sắm – Nhà cung cấp, Sự cố & SLA). Hai phân hệ Quản lý dân cư (Lương Thanh Tài) và Quản lý dịch vụ (Nguyễn Xuân Khải) do các thành viên khác đảm nhiệm, không nằm trong phạm vi báo cáo này. Đóng góp cá nhân của tác giả được nêu rõ ở phần Kết luận.

Nhóm áp dụng quy trình làm việc chuyên nghiệp: lập kế hoạch và theo dõi tiến độ bằng biểu đồ Gantt (Hình 0.1); quản trị công việc trực quan trên bảng Kanban (Hình 0.2); họp nhóm định kỳ hằng tuần (trực tiếp và trực tuyến, Hình 0.3) và trao đổi công việc hằng ngày qua nhóm chat (Hình 0.4); mỗi buổi họp đều lập biên bản (Hình 0.5). Toàn bộ minh chứng được tổng hợp trong Hồ sơ minh chứng kèm theo.

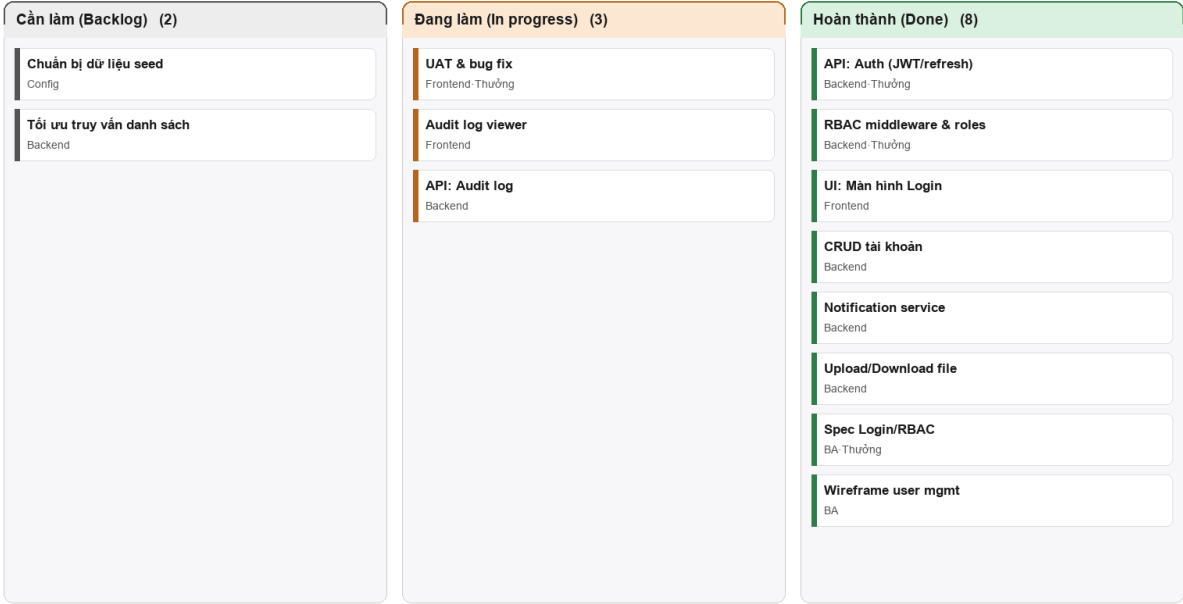
Hình PL-1. Biểu đồ tiến độ (Gantt) giai đoạn xây dựng module Base



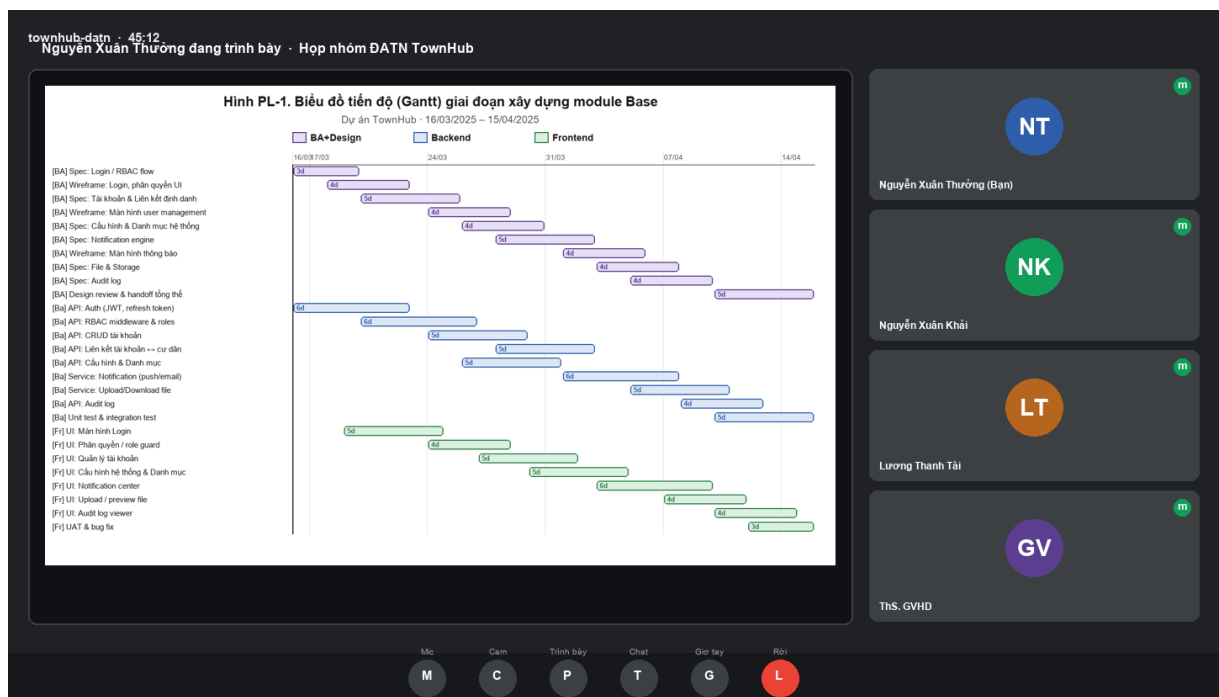
Hình 0.1. Kế hoạch – tiến độ nhóm (biểu đồ Gantt)

Hình PL-2. Bảng công việc (Kanban) trên công cụ quản trị nhóm

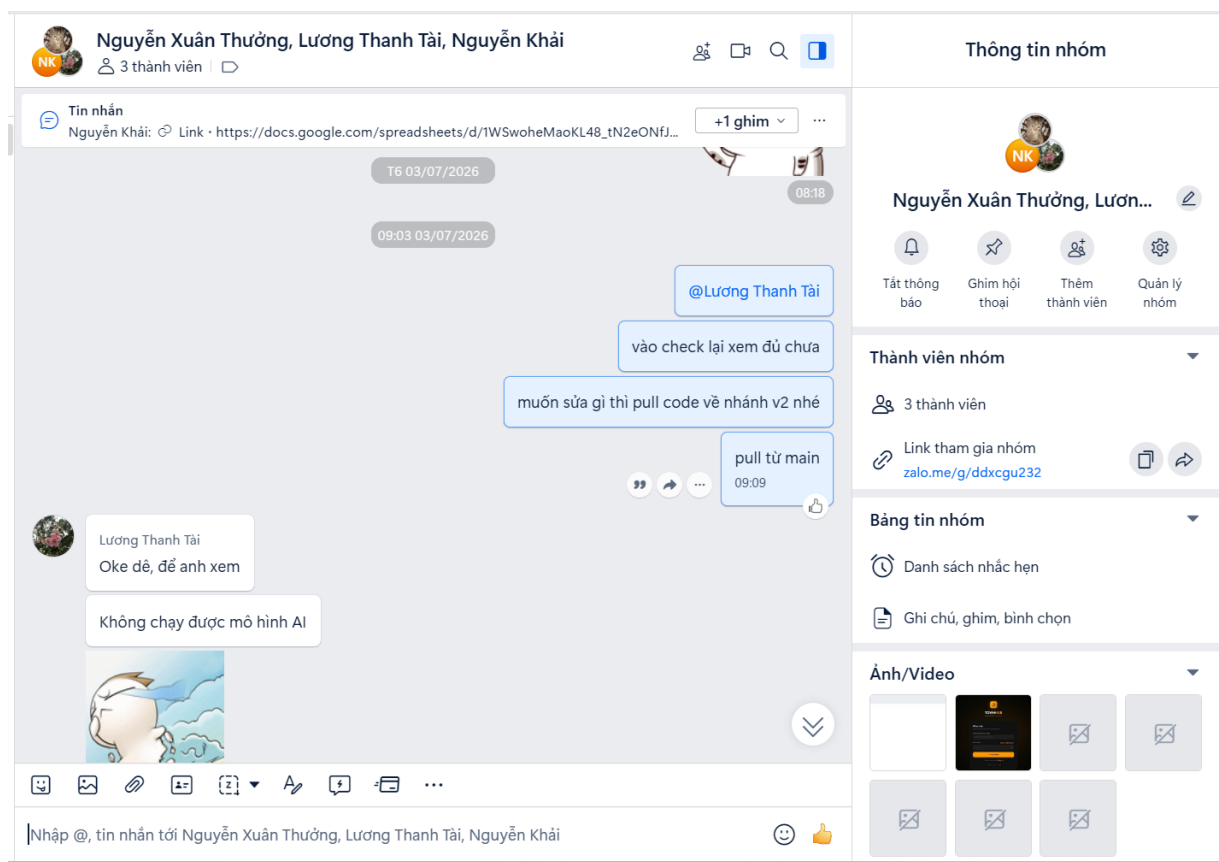
Ảnh chụp cuối giai đoạn Base — phần lớn công việc đã Hoàn thành



Hình 0.2. Bảng công việc Kanban của nhóm



Hình 0.3. Họp nhóm trực tuyến (đang trình bày tiến độ)



Hình 0.4. Trao đổi công việc trên nhóm chat

BIÊN BẢN LÀM VIỆC (HỌP NHÓM ĐATN)

Số: 06-25/BB-ĐATN

I. Thời gian, địa điểm

Hôm nay, hồi 21 giờ 00, ngày 12/04/2025, nhóm ĐATN tổ chức họp trực tuyến (Google Meet) để review tiến độ tuần và chuẩn bị nghiệm thu mốc M1.

II. Thành phần tham dự

- 1/ Nguyễn Xuân Thường — Nhóm trưởng (MSSV 0220766)
- 2/ Lương Thanh Tài — Thành viên (phân hệ Quản lý dân cư)
- 3/ Nguyễn Xuân Khải — Thành viên (phân hệ Quản lý dịch vụ)
- 4/ ThS. — Giảng viên hướng dẫn

III. Nội dung & kết quả làm việc

1. Nội dung đã thực hiện:
 - Hoàn thành Audit log (API + viewer) và kiểm thử tích hợp module Base.
 - Chuẩn hóa tài liệu, rà soát danh mục lỗi trên file kiểm thử chung.
 - Thống nhất bố cục báo cáo và phân công viết thuyết minh.
2. Vấn đề / khó khăn:
 - Dashboard tải chậm với dữ liệu lớn — lên kế hoạch tối ưu truy vấn.
3. Phân công cho tuần tiếp theo:
 - Thường: chốt Base, chuẩn bị nghiệm thu M1; bắt đầu phân hệ Tài sản.
 - Tài: hoàn thiện liên kết cư dân ↔ căn hộ; Khải: hoàn thiện Config/Audit.

Kết luận: Các thành viên nhất trí nội dung trên và thực hiện theo phân công. Biên bản kết thúc hồi 21 giờ 45 cùng ngày.

Thư ký cuộc họp
(đã ký)

Lương Thanh Tài

Nhóm trưởng
(đã ký)

Nguyễn Xuân Thường

Hình 0.5. Biên bản họp nhóm (bản mẫu)

6. Bố cục báo cáo

Báo cáo gồm năm chương: Chương 1 tổng quan và khảo sát bài toán; Chương 2 cơ sở lý thuyết và công nghệ; Chương 3 phân tích – thiết kế module chung (Base); Chương 4 phân hệ Quản lý tài sản do tác giả đảm nhiệm (gồm Tài sản & bảo trì, Kho – Mua sắm – Nhà cung cấp, Sự cố & SLA); Chương 5 triển khai, kiểm thử và đánh giá; cuối cùng là Kết luận, Tài liệu tham khảo và Phụ lục.

Chương 1. Tổng quan và khảo sát bài toán

Chương này khảo sát bài toán quản lý vận hành tòa nhà một cách có hệ thống: tổng quan nghiệp vụ và các bên liên quan, hiện trạng và vấn đề đặt ra, so sánh các giải pháp hiện có, từ đó đề xuất giải pháp cùng kiến trúc và xác định mục tiêu, phạm vi, yêu cầu của hệ thống.

1.1. Tổng quan về quản lý vận hành tòa nhà

Quản lý vận hành tòa nhà (building operations) là tập hợp các hoạt động nhằm duy trì tòa nhà hoạt động an toàn, hiệu quả và tiết kiệm chi phí trong suốt vòng đời. Các hoạt động này trải rộng trên ba nhóm nghiệp vụ gắn kết chặt chẽ: quản lý tài sản (thiết bị, bảo trì, vật tư – kho, mua sắm, nhà cung cấp); quản lý dân cư (hồ sơ cư dân, căn hộ, kiểm soát ra vào, thông báo); và quản lý dịch vụ (kết nối cư dân với nhà cung cấp dịch vụ, tiếp nhận và điều phối yêu cầu dịch vụ). Ba nhóm nghiệp vụ này dùng chung dữ liệu nền (người dùng, căn hộ, tòa nhà) nên rất phù hợp để số hóa trên một nền tảng tập trung.

1.1.1. Các bên liên quan và vai trò

Hệ thống phục vụ nhiều nhóm người dùng với vai trò khác nhau, trải đều trên cả ba phân hệ:

- **Chung – nền tảng:** quản trị viên (cấu hình, phân quyền); ban quản lý/quản lý vận hành (giám sát, ra quyết định, phê duyệt).
- **Phân hệ tài sản:** kế toán (tài sản, khấu hao, chi phí); kỹ thuật viên (bảo trì, xử lý sự cố thiết bị); thủ kho (quản lý vật tư); nhân viên mua sắm (đề xuất mua, đơn hàng).

- **Phân hệ dân cư:** nhân viên quản lý cư dân (lập hồ sơ cư dân, căn hộ, đăng ký nhận diện khuôn mặt ra vào); bảo vệ (giám sát ra vào).
- **Phân hệ dịch vụ:** cư dân (gửi yêu cầu dịch vụ, nhận thông báo); nhà cung cấp dịch vụ (đăng ký hồ sơ, niêm yết dịch vụ, tiếp nhận và xử lý yêu cầu); cư dân là chủ hộ kinh doanh (đăng ký trở thành nhà cung cấp trong tòa nhà).

Việc phân định vai trò trải đều trên cả ba phân hệ là cơ sở để thiết kế phân quyền theo vai trò (RBAC) ở phần Base.

1.1.2. Các nghiệp vụ chính

Các nghiệp vụ chính được chia đều theo ba phân hệ:

- **Tài sản:** quản lý vòng đời tài sản (ghi tăng, điều chuyển, khấu hao, thanh lý); bảo trì phòng ngừa và khắc phục (lịch bảo trì, lệnh công việc); quản lý kho vật tư (nhập/xuất/kiểm kê); mua sắm (đề xuất mua, đơn mua, hóa đơn) và quản lý nhà cung cấp vật tư.
- **Dân cư:** quản lý hồ sơ cư dân và căn hộ/đơn vị trong tòa nhà (chủ hộ, thành viên, ngày chuyển đến); kiểm soát ra vào bằng nhận diện khuôn mặt (đăng ký khuôn mặt, ghi nhận sự kiện ra vào); gửi thông báo tới cư dân.
- **Dịch vụ:** đăng ký và phê duyệt hồ sơ nhà cung cấp dịch vụ; niêm yết danh mục dịch vụ (điện, nước, vệ sinh, sửa chữa, lắp đặt camera...); tiếp nhận yêu cầu dịch vụ từ cư dân, phê duyệt của ban quản lý khi cần, phân công nhà cung cấp, theo dõi tiến độ và nghiệm thu.
- **Nền tảng dùng chung:** xác thực, phân quyền, quản lý người dùng, thông báo, cấu hình và báo cáo tổng hợp.

1.2. Khảo sát hiện trạng và vấn đề đặt ra

Qua khảo sát thực tế, nhóm nhận thấy phần lớn hoạt động quản lý vận hành hiện được thực hiện thủ công bằng giấy tờ hoặc bảng tính rời rạc, thiếu liên kết giữa các nghiệp vụ. Vấn đề trải đều trên cả ba phân hệ:

- **Về tài sản:** khó tra cứu nhanh thông tin một tài sản; không theo dõi được giá trị còn lại và khấu hao; dễ bỏ sót lịch bảo trì; tồn kho vật tư không chính xác; quy trình mua sắm và chứng từ khó kiểm soát; số liệu quản lý vật lý không khớp với sổ kế toán.

- **Về dân cư:** hồ sơ cư dân và căn hộ lưu rải rác, khó cập nhật khi có biến động (chuyển đến/đi, đổi chủ hộ); kiểm soát ra vào chủ yếu thủ công bằng thẻ hoặc ghi sổ, khó truy vết; thông báo tới cư dân phân tán qua nhiều kênh, không đồng nhất.
- **Về dịch vụ:** cư dân khó tìm được nhà cung cấp dịch vụ uy tín trong tòa nhà; yêu cầu dịch vụ trao đổi qua điện thoại/tin nhắn nên thiếu công cụ theo dõi trạng thái, thời gian xử lý và trách nhiệm; ban quản lý khó kiểm soát và đánh giá chất lượng nhà cung cấp.

Những hạn chế này đặt ra yêu cầu xây dựng một hệ thống tập trung, tự động hóa và liên thông dữ liệu cho cả ba phân hệ.

1.3. Khảo sát các giải pháp hiện có

Nhóm đã cùng nhau khảo sát hai nhóm giải pháp phổ biến là quản lý thủ công bằng bảng tính và các phần mềm quản lý bảo trì/tài sản (CMMS/EAM) thương mại, so sánh với hệ thống TownHub đề xuất theo các tiêu chí trải đều ba phân hệ tài sản, dân cư và dịch vụ:

Tiêu chí	Quản lý thủ công (Excel)	CMMS thương mại	TownHub (đề xuất)
Thủ công, dễ sai	Có	Có, tự động <u>Tài sản – Bảo trì phòng ngừa (PM/ WO)</u>	<u>Tài sản – Vòng đời & khấu hao</u> Không
Có	Có <u>Tài sản – Kho vật tư & mua sắm (PR/ PO)</u>	Rời rạc	Một phần
Có, liên thông <u>Tài sản – OCR hóa đơn tự động</u> Thủ công	Không Tùy sản phẩm	Hiếm Có (bút toán Nợ/ Có) <u>Dân cư – Hồ sơ cư dân & căn hộ</u>	Có (VietOCR) <u>Tài sản – Tích hợp kế toán & chi phí</u> Rời rạc
Ít hỗ trợ	Có, tập trung <u>Dân cư – Kiểm soát ra vào</u>	Thủ công (thẻ/số)	Tùy tích hợp
Có (nhận diện khuôn mặt) <u>Dịch vụ – Kết nối cư dân – NCC dịch vụ</u> Không	Không Một phần	Hiếm Có, có quy trình duyệt	Có (niêm yết dịch vụ) <u>Dịch vụ – Yêu cầu & phê duyệt, theo dõi</u>

Các phần mềm thương mại thường mạnh về tài sản – bảo trì nhưng chi phí cao, khó tùy biến theo đặc thù chung cư Việt Nam, ít tích hợp OCR hóa đơn tiếng Việt và gần như không đáp ứng nghiệp vụ dân cư và kết nối dịch vụ nội khu. Quản lý thủ công thì rẻ nhưng rời rạc và dễ sai sót trên cả ba phân hệ. Đây là khoảng trống mà TownHub hướng tới.

1.4. Đề xuất giải pháp và kiến trúc module hoá

Trên cơ sở khảo sát, nhóm chúng em đề xuất xây dựng hệ thống TownHub theo kiến trúc module hóa – phân chia module độc lập nhưng liên thông. Về mặt nghiệp vụ, hệ thống gồm ba phân hệ ngang hàng – Tài sản, Dân cư và Dịch vụ – cùng kế thừa lớp nền tảng Base (lõi dùng chung: xác thực – phân quyền, người dùng, thông báo, cấu hình, báo cáo,

cùng dữ liệu nền dân cư và dịch vụ). Về triển khai, backend được chia thành ba module Auth, Base và Asset tương ứng ba schema cơ sở dữ liệu auth, base và asset (ng nghiệp vụ dân cư và dịch vụ nằm trong Base); các module giao tiếp qua định danh cross-service dạng Guid, không phụ thuộc chặt vào nhau.

Cách tiếp cận này giúp: tránh trùng lặp chức năng nền tảng; cho phép các thành viên phát triển song song phân hệ riêng của mình (tài sản, dân cư, dịch vụ) mà vẫn kế thừa dữ liệu và dịch vụ dùng chung; dễ bảo trì, kiểm thử và triển khai từng phần; đồng thời thuận lợi cho việc mở rộng thêm phân hệ mới trong tương lai. Đây cũng là cách nhóm phân định rõ khối lượng công việc: phần chung do cả nhóm chịu trách nhiệm, mỗi phân hệ nghiệp vụ thể hiện đóng góp cá nhân của từng thành viên.

1.5. Mục tiêu, phạm vi và yêu cầu hệ thống

1.5.1. Yêu cầu chức năng

- **Xác thực & phân quyền (chung):** đăng nhập đa phương thức, MFA, phân quyền theo vai trò (RBAC); quản lý người dùng, thông báo, cấu hình, danh mục dùng chung và báo cáo tổng hợp.
- **Phân hệ Tài sản:** danh mục, ghi tăng, mã QR, điều chuyển, khấu hao, thanh lý; lịch bảo trì định kỳ, lệnh công việc, checklist, nghiệm thu; tồn kho, PR/PO/hóa đơn, hợp đồng, đánh giá NCC, OCR hóa đơn.
- **Phân hệ Dân cư:** quản lý hồ sơ cư dân, căn hộ/đơn vị và tòa nhà; đăng ký nhận diện khuôn mặt và ghi nhận sự kiện kiểm soát ra vào; gửi và theo dõi thông báo tới cư dân.
- **Phân hệ Dịch vụ:** đăng ký và phê duyệt nhà cung cấp dịch vụ; niêm yết danh mục dịch vụ; cư dân gửi yêu cầu dịch vụ; quy trình phê duyệt của ban quản lý, phân công nhà cung cấp, theo dõi trạng thái và nghiệm thu.

1.5.2. Yêu cầu phi chức năng

- **Bảo mật:** xác thực JWT, phân quyền tối thiểu theo vai trò, bảo vệ dữ liệu nhạy cảm (thông tin cư dân, dữ liệu sinh trắc học khuôn mặt).
- **Hiệu năng:** phản hồi nhanh, hỗ trợ phân trang – lọc trên các danh sách lớn (tài sản, cư dân, yêu cầu dịch vụ).
- **Khả năng mở rộng và bảo trì:** kiến trúc module, tách schema, chuẩn hóa API.

- Tính khả dụng và trải nghiệm: giao diện thân thiện, responsive, hỗ trợ đa thiết bị.

1.6. Kết luận chương

Chương 1 đã khảo sát nghiệp vụ, phân tích hiện trạng và các giải pháp hiện có trên cả ba phân hệ – tài sản, dân cư và dịch vụ – từ đó đề xuất giải pháp TownHub theo kiến trúc module hóa cùng mục tiêu, phạm vi và yêu cầu cụ thể. Đây là căn cứ để lựa chọn công nghệ ở Chương 2 và thiết kế hệ thống ở các chương tiếp theo.

Chương 2. Cơ sở lý thuyết và công nghệ sử dụng

Chương này trình bày kiến trúc phần mềm áp dụng và các công nghệ được lựa chọn để xây dựng hệ thống TownHub, bao gồm nền tảng backend, frontend, cơ sở dữ liệu, cơ chế bảo mật – phân quyền, hạ tầng phụ trợ, thành phần AI/OCR và công cụ triển khai. Các lựa chọn công nghệ được cả nhóm thảo luận và thống nhất ngay từ giai đoạn thiết kế để bảo đảm các module tích hợp đồng nhất, đồng thời bám sát yêu cầu đã nêu ở Chương 1 và hiện trạng mã nguồn thực tế của dự án.

2.1. Kiến trúc phần mềm áp dụng

Hệ thống áp dụng mô hình client–server, giao tiếp qua REST API định dạng JSON và xác thực bằng JWT. Tầng backend tổ chức theo kiến trúc phân tầng (layered architecture): Controllers tiếp nhận yêu cầu và kiểm tra quyền → ApplicationService xử lý logic nghiệp vụ → Domain chứa thực thể và Infrastructure đảm nhiệm truy cập dữ liệu qua EF Core. Toàn bộ backend được chia thành ba module độc lập Auth, Base và Asset, mỗi module có đầy đủ bốn tầng và tương ứng một schema cơ sở dữ liệu, giúp phân tách trách nhiệm rõ ràng và cho phép phát triển song song.

2.2. Công nghệ Backend (.NET 8 Web API)

Backend được xây dựng bằng .NET 8 với Entity Framework Core làm ORM truy cập PostgreSQL. Các thành phần chính được tổng hợp trong Bảng 2.1.

Hạng mục	Công nghệ
.NET 8.0 (nullable + implicit usings)	Framework
ORM	Entity Framework Core 8.0.11
PostgreSQL (Npgsql.	CSDL
EntityFrameworkCore.PostgreSQL)	Swashbuckle / Swagger 6.6.2
API Docs	Xác thực
JWT Bearer, Google OAuth 2.0, Identity.	Otp.NET (TOTP)
Core	Cache / Session
MFA	
StackExchange.Redis	MailKit
Email	Lưu trữ file
CloudinaryDotNet	

2.3. Công nghệ Frontend (Next.js 16 + React 19)

Giao diện được phát triển bằng Next.js 16 (App Router) và React 19 với TypeScript, sử dụng Tailwind CSS và bộ component Radix UI để bảo đảm tính nhất quán và khả năng tùy biến. Các thư viện hỗ trợ được liệt kê trong Bảng 2.2.

Hạng mục	Công nghệ
Next.js 16.2.1 (App Router)	Framework
UI	React 19, TypeScript 5 (strict)
Tailwind CSS v4 (CSS variables, dark mode)	Styling
Component	Radix UI (29+ primitives)
react-hook-form, input-otp	Form / MFA
Biểu đồ	recharts
qrcode (sinh), jsqr (quét)	QR Code

Toàn bộ lời gọi API được tập trung ở tầng client (lib/api.ts) và tự động gắn token xác thực; trạng thái người dùng, quyền và phiên được quản lý qua AuthContext. Menu và chức năng hiển thị theo quyền (RBAC phía frontend) đồng bộ với backend.

2.4. Cơ sở dữ liệu PostgreSQL

Hệ thống sử dụng PostgreSQL - hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, mạnh về toàn vẹn dữ liệu, kiểu dữ liệu phong phú (JSON, ENUM) và giao dịch (transaction).

Dữ liệu được tổ chức thành ba schema độc lập auth, base và asset tương ứng ba module backend, giúp cô lập dữ liệu giữa các module và thuận tiện phân quyền, sao lưu. EF Core quản lý ánh xạ thực thể và migration để đồng bộ lược đồ.

2.5. Bảo mật và phân quyền (JWT, OAuth, MFA/TOTP, RBAC)

Về xác thực, hệ thống hỗ trợ ba phương thức: email/mật khẩu, Google OAuth 2.0 và luồng riêng cho di động. Sau khi đăng nhập, hệ thống cấp AccessToken (hiệu lực khoảng 30 phút) và RefreshToken (khoảng 7 ngày), áp dụng cơ chế token rotation và trường tokenVersion để vô hiệu hóa token khi cần. Với web, token lưu trong cookie HttpOnly kèm CSRF token; phiên được quản lý theo thiết bị, cho phép đăng xuất trên một hoặc tất cả thiết bị. Bảo mật được tăng cường bằng xác thực đa yếu tố (MFA) dùng mã OTP theo thời gian (TOTP).

Về phân quyền, hệ thống áp dụng mô hình RBAC (Role-Based Access Control): quyền định nghĩa chi tiết theo cặp resource + action, gán cho vai trò và vai trò gán cho người dùng; mỗi API được bảo vệ bằng policy tương ứng. Chi tiết thiết kế được trình bày ở Chương 3.

2.6. Hạ tầng cache, lưu trữ và email

Redis được dùng làm bộ nhớ đệm và lưu trạng thái phiên/MFA, giúp giảm tải truy vấn và tăng tốc xác thực. Cloudinary lưu trữ ảnh và tệp đính kèm (ảnh tài sản, hóa đơn, minh chứng sự cố). MailKit đảm nhiệm gửi email phục vụ xác thực tài khoản, gửi OTP và đặt lại mật khẩu.

2.7. Công nghệ AI/OCR (VietOCR) và xử lý nền

Để tự động hóa việc nhập liệu hóa đơn, hệ thống tích hợp OCR (nhận dạng ký tự) bằng VietOCR. Do xử lý ảnh tốn thời gian, chức năng này được thiết kế bất đồng bộ theo mô hình hàng đợi – tiến trình nền: yêu cầu OCR được đẩy vào OcrJobQueue, tiến trình nền OcrProcessingWorker lấy ra và gọi IInvoiceOcrEngine (bản cài đặt VietOcrEngine, kèm MockOcrEngine phục vụ kiểm thử) để trích xuất dữ liệu và tạo bản ghi hóa đơn. Cách làm này giúp giao diện không bị chặn và dễ mở rộng khối lượng xử lý.

Chi tiết mô hình AI (VietOCR, PaddleOCR), cơ sở lý thuyết, quá trình huấn luyện trên dữ liệu tự sinh và kết quả đánh giá – so sánh các engine được trình bày trong Chương 4, mục 4.2.7 (phân hệ Kho – Mua sắm).

2.8. Công cụ quản lý dự án và triển khai

Mã nguồn được quản lý bằng Git. Ứng dụng được đóng gói bằng Docker (multi-stage build) và triển khai trên nền tảng Fly.io; tài liệu API được sinh tự động bằng Swagger phục vụ phát triển và kiểm thử. Nhóm sử dụng công cụ quản lý công việc để phối hợp theo quy trình Agile/Scrum.

2.9. Kết luận chương

Chương 2 đã trình bày kiến trúc phần mềm và các công nghệ nền tảng của TownHub: backend .NET 8, frontend Next.js 16, cơ sở dữ liệu PostgreSQL, cơ chế bảo mật – phân quyền, hạ tầng phụ trợ và thành phần OCR. Trên nền tảng này, các chương tiếp theo trình bày phân tích – thiết kế chi tiết của module Base và các phân hệ nghiệp vụ.

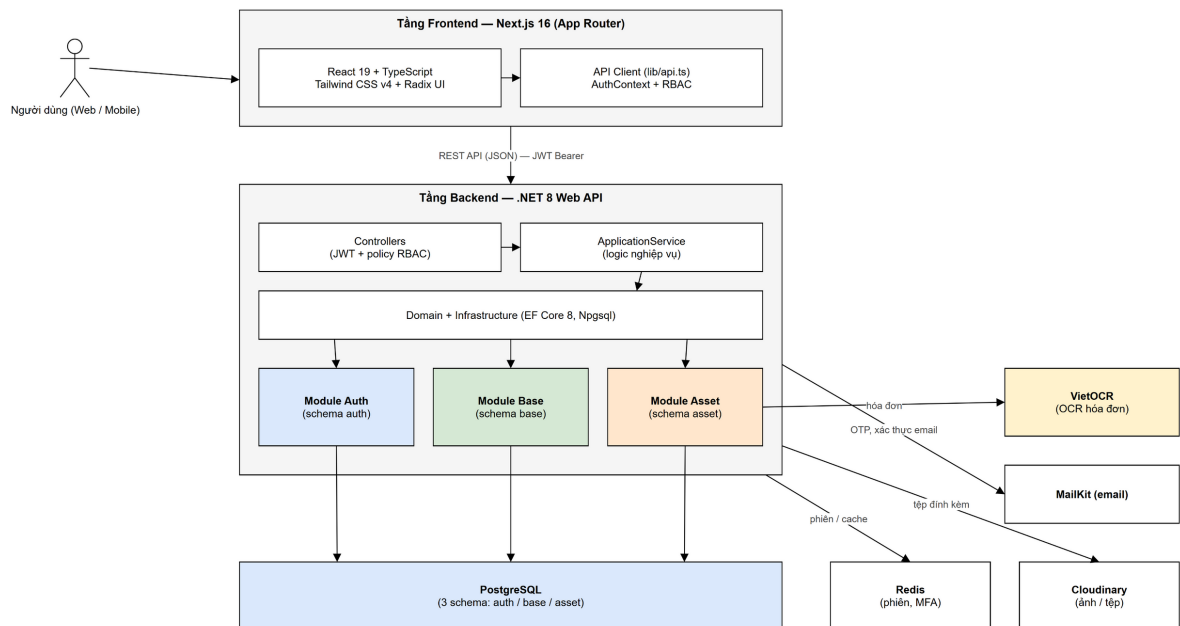
Chương 3. Phân tích và thiết kế module chung (Base)

Module chung (Base) do cả nhóm cùng phối hợp xây dựng, là xương sống của hệ thống TownHub, cung cấp các dịch vụ dùng chung mà mọi phân hệ nghiệp vụ đều kế thừa: xác thực và phân quyền, quản lý người dùng, thông báo, cấu hình – danh mục dùng chung và báo cáo tổng hợp. Việc tách phần Base giúp tránh trùng lặp, chuẩn hóa định danh và bảo mật cho toàn hệ thống, đồng thời cho phép các phân hệ riêng (Quản lý tài sản, Quản lý dân cư, Quản lý dịch vụ...) mở rộng độc lập. Chương này trình bày kiến trúc tổng thể, biểu đồ use case, thiết kế cơ sở dữ liệu lõi và đặc tả từng phân hệ Base.

3.1. Kiến trúc tổng thể hệ thống TownHub

Hệ thống được xây dựng theo mô hình client–server nhiều tầng, tách biệt rõ ràng giữa tầng trình bày (frontend) và tầng dịch vụ (backend), giao tiếp qua REST API định dạng JSON và bảo mật bằng JWT Bearer. Kiến trúc tổng thể được minh họa ở Hình 3.1.

Hình 3.1 — Kiến trúc tổng thể hệ thống TownHub



Hình 3.1. Kiến trúc tổng thể hệ thống TownHub

Tầng Frontend được phát triển bằng Next.js 16 (App Router) và React 19, sử dụng TypeScript, Tailwind CSS v4 và bộ component Radix UI; toàn bộ lời gọi API tập trung ở tầng client (*lib/api.ts*) và tự động gắn token xác thực vào mỗi yêu cầu.

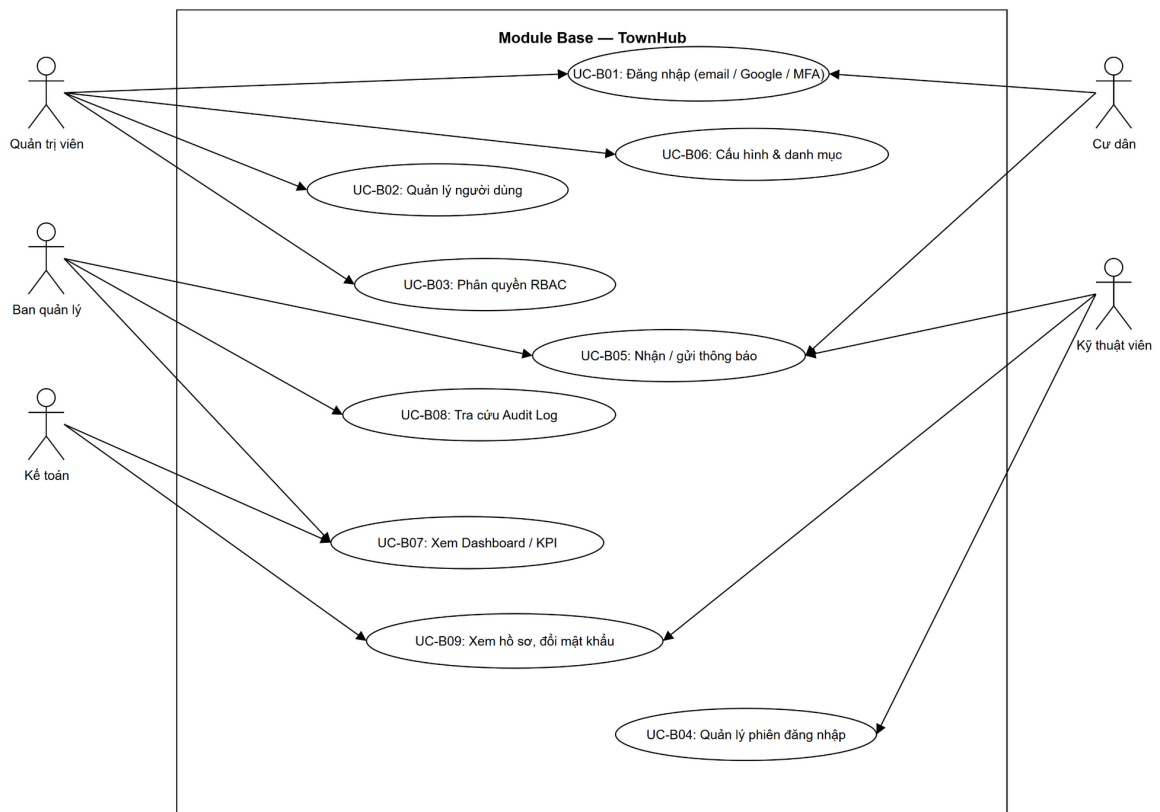
Tầng Backend là ứng dụng .NET 8 Web API tổ chức theo kiến trúc phân tầng: *Controllers* tiếp nhận yêu cầu và kiểm tra quyền → *ApplicationService* xử lý logic nghiệp vụ → *Domain* (thực thể) và *Infrastructure* (EF Core 8, Npgsql) truy cập dữ liệu. Backend được chia thành ba module độc lập là *Auth*, *Base* và *Asset*, tương ứng ba nhóm nghiệp vụ và ba schema cơ sở dữ liệu.

Tầng dữ liệu và dịch vụ ngoài gồm: PostgreSQL (ba schema *auth/asset/base*) làm CSDL chính; Redis lưu phiên và trạng thái MFA; Cloudinary lưu ảnh/tệp; VietOCR và MailKit phục vụ nhận dạng hóa đơn và gửi email. Mọi API trả về theo cấu trúc thống nhất *ResponseDto<T>* gồm *ErrorCode*, *ErrorMessage* và *Data*, giúp frontend xử lý phản hồi nhất quán.

3.2. Biểu đồ Use Case tổng quát

Các phân hệ Base phục vụ nhiều nhóm người dùng với vai trò khác nhau. Biểu đồ use case tổng quát của module Base được thể hiện ở Hình 3.2, mô tả các chức năng dùng chung và mối quan hệ với từng tác nhân.

Hình 3.2 — Biểu đồ Use Case tổng quát module Base



Hình 3.2. Biểu đồ Use Case tổng quát module Base

Các tác nhân chính của hệ thống:

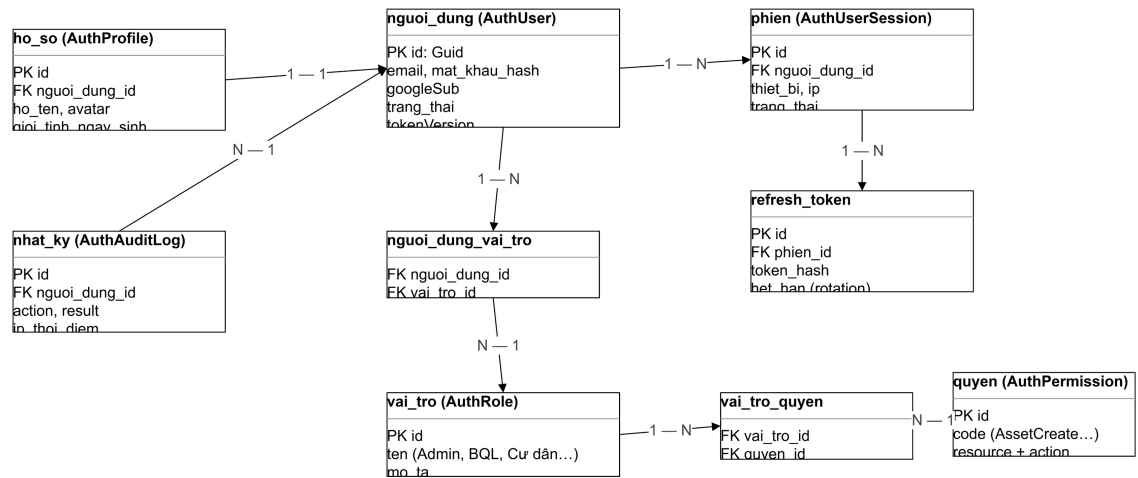
Tác nhân	Mô tả vai trò
Cấu hình hệ thống, quản lý tài khoản, phân quyền; toàn quyền hệ thống. Ban quản lý	Quản trị viên
Nghiệp vụ tài chính (tài sản, chi phí), xem báo cáo. Cư dân	Theo dõi vận hành, xem báo cáo, nhận và xử lý thông báo. Kế toán
Đăng nhập, nhận việc/thông báo, cập nhật xử lý.	Đăng nhập, nhận thông báo, gửi yêu cầu/sự cố. Kỹ thuật viên

3.3. Thiết kế cơ sở dữ liệu lõi (schema auth)

Cơ sở dữ liệu lõi của Base tập trung ở schema auth, quản lý người dùng, phân quyền theo vai trò (RBAC), phiên đăng nhập và nhật ký hoạt động. Sơ đồ thực thể – liên kết (ERD)

được trình bày ở Hình 3.3.

Hình 3.3 — ERD schema auth



Hình 3.3. Sơ đồ thực thể – liên kết (ERD) schema auth

Các thực thể chính và quan hệ:

Thực thể	Ý nghĩa	Quan hệ chính
Tài khoản: email, mật khẩu, googleSub, trạng thái, tokenVersion	1-1 ho_so; 1-N phiên, nhật ký	nguoi_dung (AuthUser)
Thuộc một người dùng vai_tro (AuthRole)	ho_so (AuthProfile)	Hồ sơ cá nhân: họ tên, avatar, giới tính...
Quyền chi tiết theo code (AssetCreate...)	Vai trò (Admin, BQL, Cư dân, Kỹ thuật...)	N-N người dùng; N-N quyền
Bảng trung gian N-N phien (AuthUserSession)	N-N vai trò nguoi_dung_vai_tro / vai_tro_quyen	quyen (AuthPermission)
Refresh token, hỗ trợ rotation	Phiên theo thiết bị (đa thiết bị)	Bảng nối gán vai trò ↔ người dùng, quyền ↔ vai trò
Thuộc một người dùng	Thuộc một phiên nhap_ky (AuthAuditLog)	1-N refresh_token refresh_token
		Nhật ký hành động: action, result, IP...

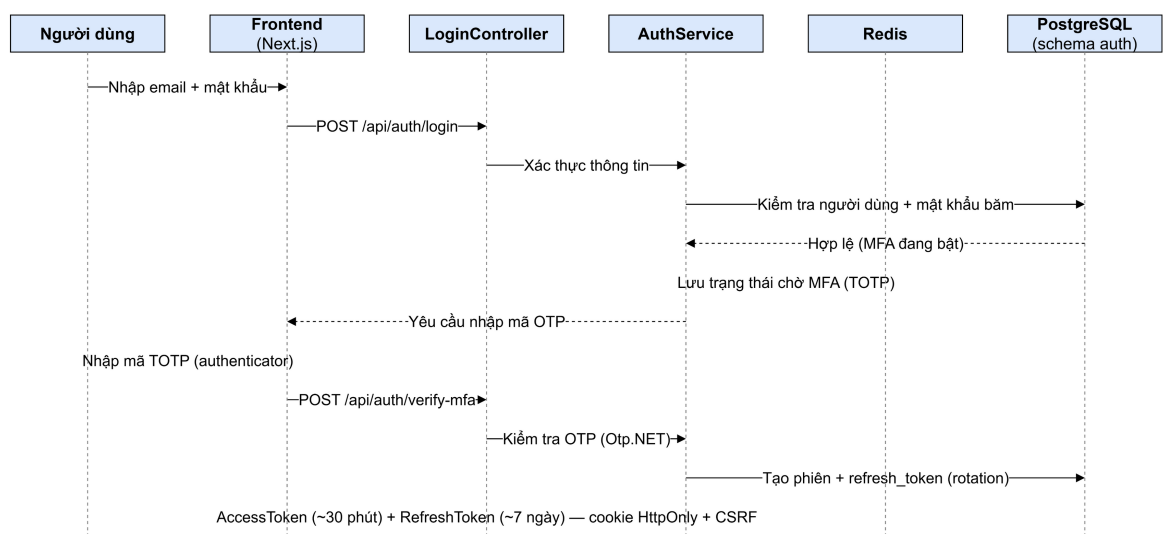
3.4. Đặc tả các phân hệ Base

3.4.1. Auth & Security - Xác thực và phân quyền

Phân hệ xác thực hỗ trợ ba phương thức đăng nhập: email/mật khẩu, đăng nhập Google (OAuth 2.0) và luồng riêng cho ứng dụng di động (trả token trong nội dung phản hồi thay vì cookie). Sau khi xác thực thành công, hệ thống cấp AccessToken (hiệu lực khoảng 30 phút) và RefreshToken (khoảng 7 ngày); áp dụng token rotation và trường tokenVersion để vô hiệu hóa token khi cần. Với giao diện web, token được lưu trong cookie HttpOnly kèm CSRF token; hệ thống quản lý phiên theo từng thiết bị, cho phép đăng xuất trên một hoặc tất cả thiết bị.

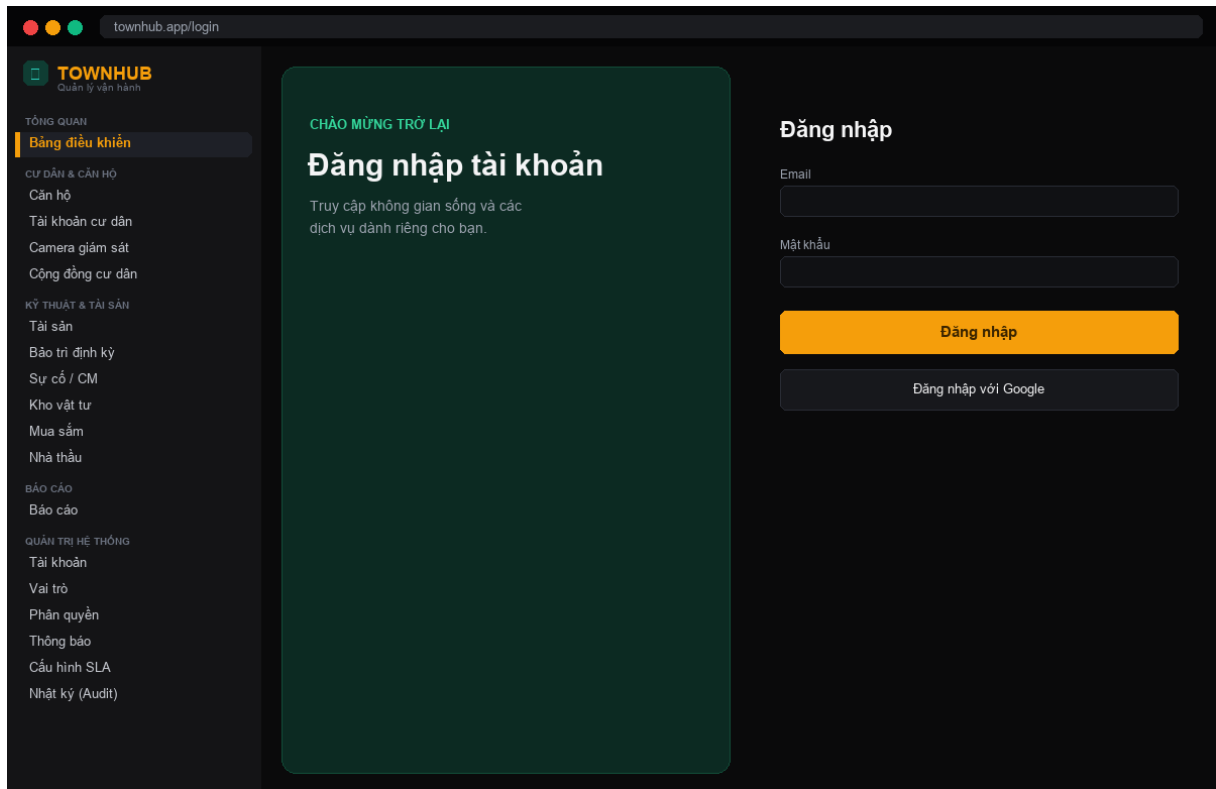
Để tăng cường bảo mật, hệ thống hỗ trợ xác thực đa yếu tố (MFA) bằng mã OTP theo thời gian (TOTP, thư viện Otp.NET) qua ứng dụng authenticator, cùng chức năng xác thực email và đặt lại mật khẩu qua email (MailKit). Luồng đăng nhập có MFA và cấp JWT được minh họa ở Hình 3.4.

Hình 3.4 — Biểu đồ tuần tự: Đăng nhập có MFA và cấp JWT



Hình 3.4. Biểu đồ tuần tự: Đăng nhập có MFA và cấp JWT

Giao diện màn hình đăng nhập thực tế của hệ thống được thể hiện ở Hình 3.5.



Hình 3.5. Giao diện màn hình đăng nhập

Về phân quyền, hệ thống áp dụng RBAC: quyền được định nghĩa chi tiết theo cặp resource + action (ví dụ AssetCreate, AssetUpdate, AccountMfaSetup...) và gán cho vai trò; vai trò gán cho người dùng. Các controller chính: LoginController, RegisterController, AccountController (xác thực); RoleController, PermissionController, RolePermissionController, UserRoleController (phân quyền); UserSessionController (quản lý phiên).

3.4.2. User Management - Người dùng và nhật ký hoạt động

Phân hệ quản lý người dùng cung cấp các thao tác CRUD tài khoản (UserController) kèm trạng thái hoạt động, vai trò và thông tin hồ sơ. Base cung cấp định danh trung tâm để các phân hệ khác liên kết (ví dụ phân hệ Quản lý dân cư gắn tài khoản với cư dân/căn hộ). Toàn bộ hành động quan trọng trong hệ thống được ghi lại qua AuditLogController (đăng nhập, thay đổi quyền, thao tác dữ liệu...) phục vụ truy vết và an toàn thông tin.

3.4.3. Notification Engine - Thông báo

Phân hệ thông báo cho phép gửi thông báo đa kênh (push, email, SMS) qua NotificationController, quản lý mẫu thông báo tái sử dụng qua NotificationTemplateController và tra

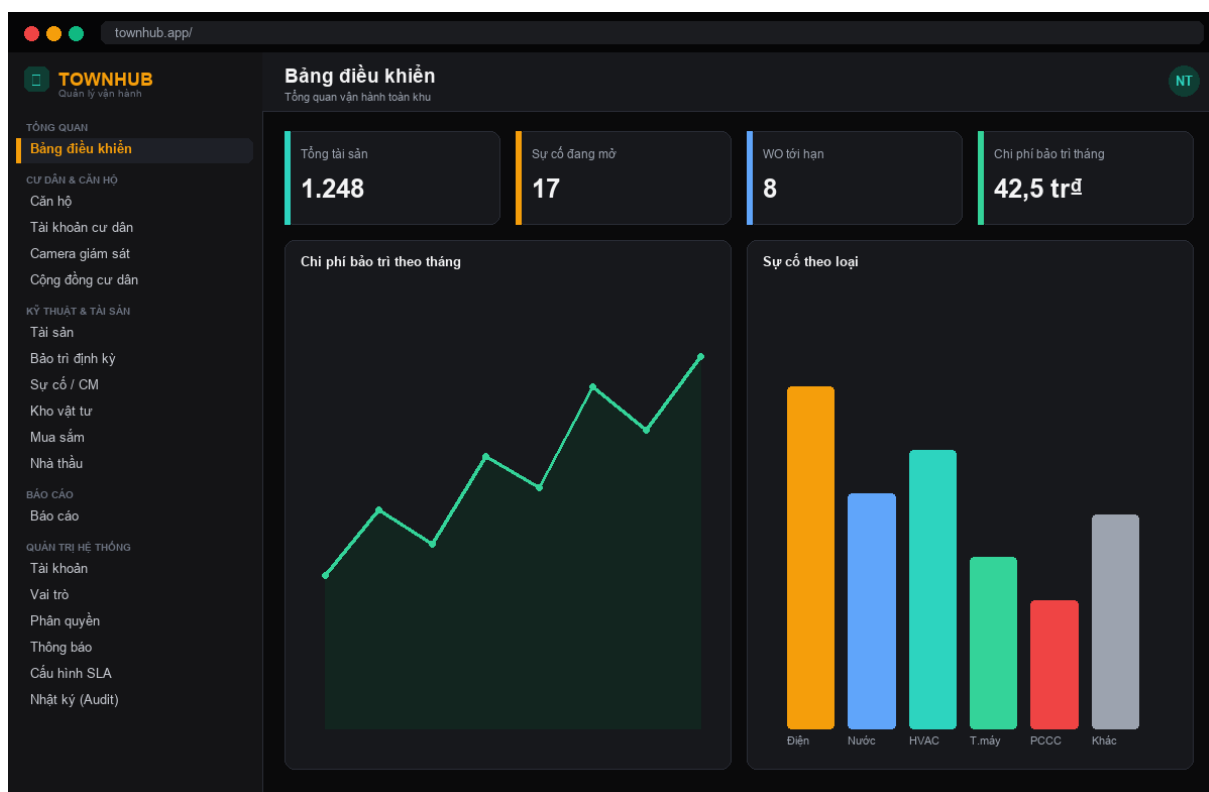
cứu lịch sử thông báo đã gửi. Đây là dịch vụ dùng chung cho các phân hệ nghiệp vụ, ví dụ cảnh báo tồn kho thấp, nhắc hạn bảo trì hoặc cảnh báo sắp/đã quá hạn SLA.

3.4.4. Configuration - Cấu hình và danh mục dùng chung

Phân hệ cấu hình quản lý các tham số chung của hệ thống (tên khu, logo, định dạng và độ dài sinh mã, phương pháp khấu hao mặc định...) và các danh mục dùng chung như loại tài sản, loại phí, loại dịch vụ. Việc tập trung danh mục giúp chuẩn hóa dữ liệu đầu vào và cho phép các phân hệ tự động áp dụng giá trị mặc định.

3.4.5. Reporting - Dashboard và KPI

Phân hệ báo cáo tổng hợp số liệu toàn hệ thống thành dashboard trực quan và các báo cáo KPI, báo cáo chi phí (route /reports, /reports/kpi, /reports/cost; KpiSnapshotController, CostTrackingController). Số liệu được thu thập từ các phân hệ nghiệp vụ để phục vụ ra quyết định vận hành. Giao diện Bảng điều khiển được thể hiện ở Hình 3.6.



Hình 3.6. Giao diện Bảng điều khiển (Dashboard) tổng quan

3.5. Thiết kế API dùng chung và điểm mở rộng cho phân hệ riêng

Base định ra các quy ước kỹ thuật để các phân hệ riêng tích hợp đồng nhất:

- Mọi API trả về cấu trúc `ResponseDto<T> { ErrorCode, ErrorMessage, Data }` để chuẩn hóa xử lý phản hồi và lỗi.
- Mọi endpoint yêu cầu xác thực JWT và kiểm tra quyền bằng policy RBAC gắn ở controller.
- Các phân hệ tham chiếu định danh cross-service bằng Guid (ví dụ `buildingId`, `floorId` của Base; `createdBy`, `assignedTo` của Auth) và KHÔNG dùng navigation property xuyên module, giữ cho ba module độc lập, dễ bảo trì và triển khai.
- Thông báo, audit log, cấu hình và danh mục được cung cấp dưới dạng dịch vụ dùng lại cho mọi phân hệ.

3.6. Kết luận chương

Chương 3 đã trình bày kiến trúc tổng thể, biểu đồ use case, thiết kế cơ sở dữ liệu lỗi (schema auth) và đặc tả năm phân hệ Base (Auth & Security, User Management, Notification Engine, Configuration, Reporting) cùng các quy ước tích hợp. Đây là nền tảng để các phân hệ nghiệp vụ - trong đó có phân hệ Quản lý tài sản do tác giả đảm nhiệm - được xây dựng ở các chương tiếp theo.

Chương 4. Phân tích và thiết kế phân hệ Quản lý tài sản (phần riêng)

Sinh viên thực hiện: Nguyễn Xuân Thường – MSSV: 0220766 – Lớp: 66KSCS

Chương này trình bày phân tích và thiết kế phân hệ Quản lý tài sản do tác giả (Nguyễn Xuân Thường) đảm nhiệm, gồm ba nghiệp vụ liên thông: (4.1) Tài sản & bảo trì, (4.2) Kho – Mua sắm – Nhà cung cấp, (4.3) Sự cố & SLA. Cả ba đều thuộc module Asset ở backend và kế thừa các dịch vụ nền tảng của module Base (Chương 3). Hai phân hệ Quản lý dân cư (Lương Thanh Tài) và Quản lý dịch vụ (Nguyễn Xuân Khải) do các thành viên khác trong nhóm đảm nhiệm và không thuộc phạm vi trình bày của báo cáo này.

4.1. Nghiệp vụ Tài sản & bảo trì

4.1.1. Đặt vấn đề và phạm vi

Một tòa nhà sở hữu số lượng lớn tài sản cố định và thiết bị kỹ thuật. Quản lý thủ công dẫn tới khó tra cứu, không theo dõi được vòng đời và giá trị còn lại, tính khấu hao sai lệch, bỏ sót lịch bảo trì, số liệu quản lý vật lý không khớp sổ kế toán.

Nghiệp vụ Tài sản & bảo trì là lõi nghiệp vụ của TownHub, quản lý toàn bộ vòng đời tài sản: ghi tăng (mua sắm) → cấp phát – điều chuyển → khấu hao định kỳ → bảo trì → thanh lý; mỗi biến động đều sinh chứng từ kế toán. Phần vận hành kỹ thuật (bảo trì phòng ngừa PM/Work Order) bảo đảm thiết bị được bảo dưỡng đúng hạn.

- Quản lý danh mục tài sản, vị trí lắp đặt và mã QR cho từng tài sản.
- Ghi tăng tài sản (đơn lẻ hoặc theo lô) và tự sinh mã theo cấu hình.

- Cấp phát, thu hồi, luân chuyển tài sản giữa phòng ban và người dùng.
- Tính và ghi nhận khấu hao (đường thẳng, số dư giảm dần, tổng số năm).
- Bảo trì phòng ngừa theo lịch (Work Order, checklist, check-in kỹ thuật viên).
- Thanh lý tài sản, tính lãi/lỗ; tự động sinh chứng từ và bút toán Nợ/Có.

4.1.2. Môi liên hệ và kế thừa từ module Base

Theo nguyên tắc phân chia module độc lập nhưng liên thông mà nhóm thống nhất, phân hệ không xây dựng lại các dịch vụ nền tảng mà kế thừa trực tiếp thành quả chung từ module Base:

- **Auth/RBAC:** mọi API yêu cầu JWT; policy AssetView, AssetCreate, AssetUpdate, AssetDelete gắn ở controller.
- **Định danh không gian (Base):** tài sản/vị trí tham chiếu buildingId, floorId dạng Guid cross-service.
- **Configuration:** định dạng sinh mã, phương pháp khấu hao mặc định, danh mục loại tài sản/tài khoản kế toán.
- **Notification & Reporting:** cảnh báo hạn bảo trì/quá hạn; số liệu giá trị & chi phí đẩy lên Dashboard/KPI.

4.1.3. Phân tích yêu cầu chức năng

Các tác nhân chính:

Tác nhân	Vai trò
Cấu hình, danh mục, phân quyền. Kế toán	Quản trị viên Ghi tăng, khấu hao, thanh lý, đối chiếu chứng từ. Quản lý / BQL
Duyệt điều chuyển, theo dõi giá trị & tình trạng. Kỹ thuật viên	Cấp phát, kiểm kê, quét QR, thực hiện Work Order bảo trì.

a) Danh mục và vị trí tài sản

Khai báo danh mục (mã, tên, tiền tố sinh mã, thời gian khấu hao mặc định, tài khoản hạch toán) và vị trí lắp đặt theo tòa nhà/tầng; dùng lại khi ghi tăng để chuẩn hóa dữ liệu.

b) Ghi tăng và vòng đời tài sản

Nhập danh mục, tên, số seri, nhà sản xuất, thông số kỹ thuật (JSON), thông tin tài chính (nguyên giá, ngày mua, thời gian & phương pháp khấu hao, giá trị thu hồi). Trạng thái: nhân rồi → đang sử dụng → đang bảo trì → đã thanh lý.

c) Mã QR tài sản

Mỗi tài sản có mã QR sinh tự động; nhân viên quét QR bằng camera để mở nhanh trang chi tiết - hỗ trợ kiểm kê hiện trường.

d) Cấp phát, điều chuyển, thu hồi

Chọn loại điều chuyển (cấp phát/thu hồi/luân chuyển), phòng ban/người nhận và nguồn; hệ thống tạo bản ghi điều chuyển, cập nhật phòng ban/người dùng và trạng thái; lịch sử hiển thị ở trang chi tiết.

e) Khấu hao định kỳ

Đến kỳ (đầu tháng), hệ thống tính khấu hao theo phương pháp của từng tài sản, cộng dồn khấu hao lũy kế, trừ giá trị còn lại, ghi lịch sử khấu hao và sinh chứng từ. Ba phương pháp:

- **Đường thẳng:** khấu hao tháng = nguyên giá / thời gian khấu hao (số tháng).
- **Số dư giảm dần:** khấu hao nhanh giai đoạn đầu theo tỉ lệ trên giá trị còn lại.
- **Tổng số năm:** phân bổ theo tỉ trọng số năm sử dụng còn lại.

f) Bảo trì phòng ngừa (PM / Work Order)

Luồng bảo trì: Lịch bảo trì định kỳ (chu kỳ, lead time) → tự sinh Work Order khi tới hạn → phân công kỹ thuật viên → check-in hiện trường → thực hiện checklist + đính kèm ảnh → nộp → nghiệm thu (duyet/từ chối). Bảo trì có chi phí: sửa chữa hạch toán chi phí trong kỳ, nâng cấp cộng vào nguyên giá.

g) Thanh lý tài sản

Tính giá trị còn lại từ nguyên giá và khấu hao lũy kế; nhập giá trị thu hồi; hệ thống tính lãi/lỗ, cập nhật trạng thái "đã thanh lý" và sinh chứng từ thanh lý.

h) Sổ kế toán và báo cáo tài chính tài sản

Toàn bộ biến động tài sản (ghi tăng, khấu hao, thanh lý) đều sinh chứng từ kèm định khoản Nợ/Có; trên nền dữ liệu chứng từ đó, phân hệ cung cấp hệ thống sổ sách và báo cáo kế toán theo hình thức Nhật ký chung. Các sổ và báo cáo được dẫn xuất trực tiếp từ chứng từ nên luôn khớp sổ và tuân thủ hệ thống tài khoản theo Thông tư 200:

- **Nhật ký chung:** ghi nhận toàn bộ bút toán phát sinh theo trình tự thời gian, cho phép lọc theo khoảng thời gian và theo tài khoản.
- **Sổ cái:** theo dõi phát sinh Nợ/Có và số dư (đầu kỳ, trong kỳ, cuối kỳ) của từng tài khoản kế toán, kèm số dư lũy kế và tài khoản đối ứng của mỗi bút toán.
- **Bảng cân đối số phát sinh:** tổng hợp số dư đầu kỳ, phát sinh trong kỳ và số dư cuối kỳ của mọi tài khoản, đối chiếu cân đối tổng phát sinh Nợ bằng tổng phát sinh Có.
- **Sổ tài sản cố định:** liệt kê nguyên giá, hao mòn lũy kế và giá trị còn lại của từng tài sản (sổ chi tiết TSCĐ) kèm số liệu tổng hợp toàn danh mục.
- **Báo cáo tăng/giảm tài sản:** tổng hợp tài sản ghi tăng (mua mới) và ghi giảm (thanh lý) theo kỳ, kèm lãi/lỗ khi thanh lý.

4.1.4. Yêu cầu phi chức năng

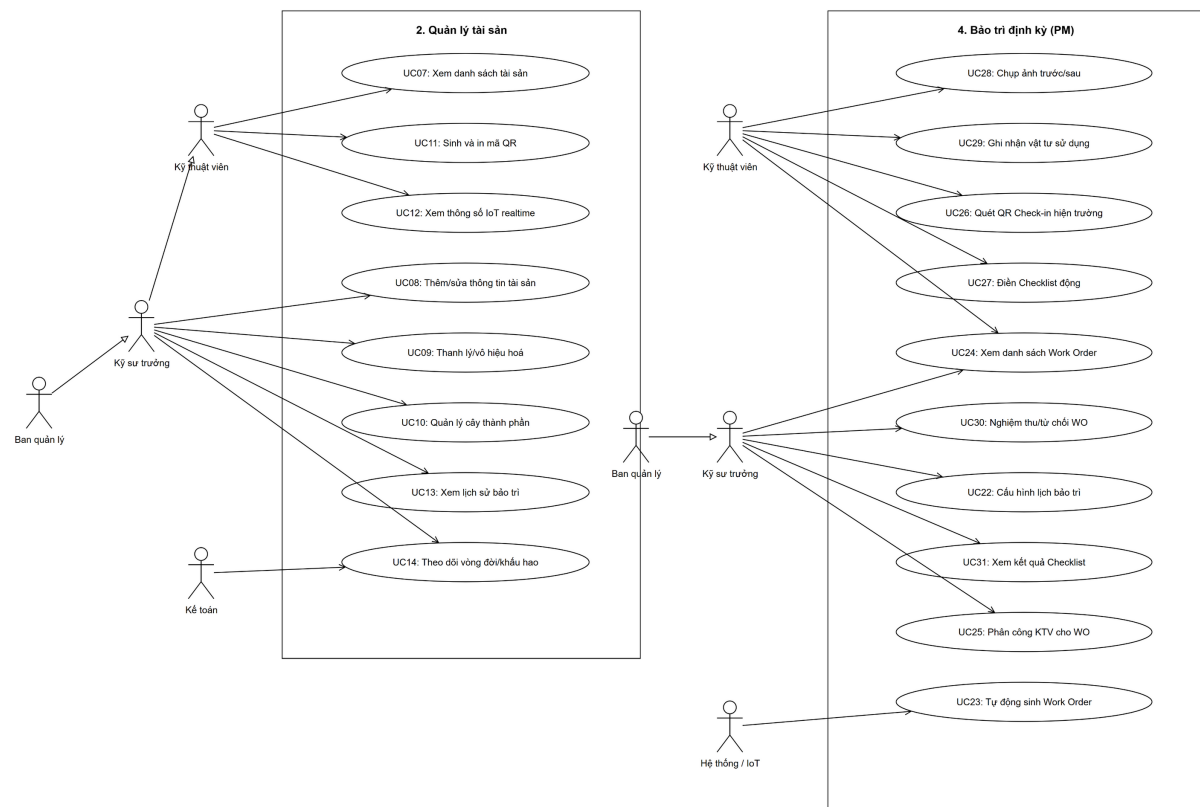
- Bảo mật theo RBAC; dữ liệu tài chính chỉ hiển thị cho vai trò phù hợp.
- Toàn vẹn: mọi biến động sinh chứng từ; giao dịch nhiều bảng nhất quán.
- Hiệu năng: danh sách phân trang, lọc theo tòa nhà/danh mục/trạng thái.

4.1.5. Thiết kế phân hệ

a) Danh sách Use Case chính

Các use case chính của phân hệ được thể hiện trên biểu đồ use case ở Hình 4.1 và liệt kê trong bảng dưới đây.

Hình 4.1 — Biểu đồ Use Case: Quản lý tài sản & bảo trì



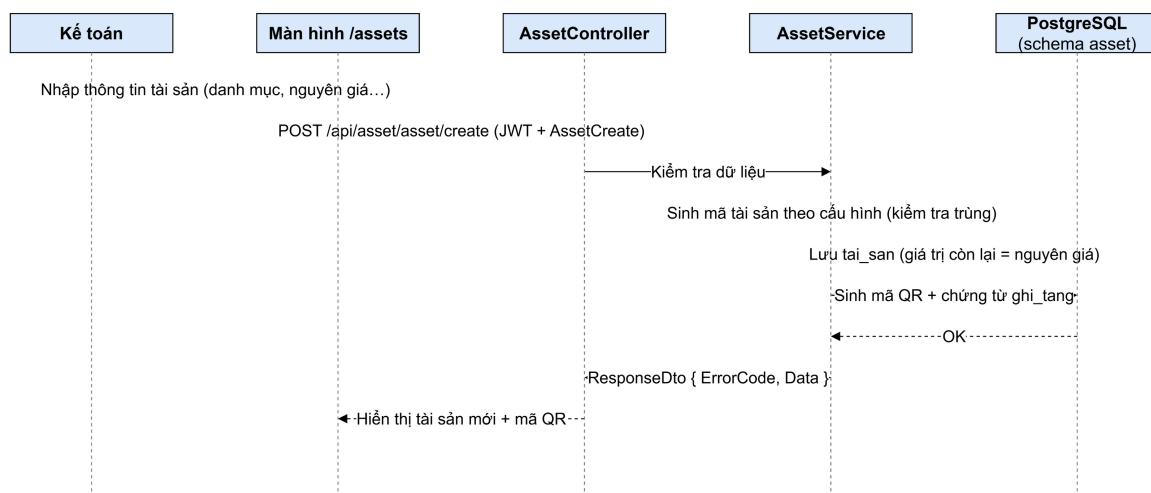
Hình 4.1. Biểu đồ Use Case – Quản lý tài sản & bảo trì

Mã UC	Use case	Tác nhân
Quản lý danh mục & vị trí tài sản	Admin/Kế toán	UC-AS-01
Kế toán	UC-AS-02	Ghi tăng tài sản (sinh mã + QR)
UC-AS-03	Quét QR tra cứu tài sản	Kỹ thuật
Điều chuyển / cấp phát / thu hồi	Quản lý/Kỹ thuật	UC-AS-04
Kế toán/Hệ thống	UC-AS-05	Tính & ghi nhận khấu hao
UC-AS-06	Lập lịch & thực hiện Work Order bảo trì	Kỹ thuật
Thanh lý tài sản (tính lãi/lỗ)	Kế toán	UC-AS-07
Kế toán/Quản lý	UC-AS-08	Xem sổ kế toán & báo cáo tài chính

b) Biểu đồ tuần tự và lưu đồ luồng tiêu biểu

Luồng ghi tăng tài sản được minh họa ở Hình 4.2: người dùng nhập thông tin ở màn hình /assets → AssetController nhận POST create → AssetService kiểm tra dữ liệu và sinh mã (kiểm tra trùng), khởi tạo giá trị còn lại bằng nguyên giá → lưu tài sản → sinh mã QR và chứng từ ghi_tang → trả ResponseDto về giao diện.

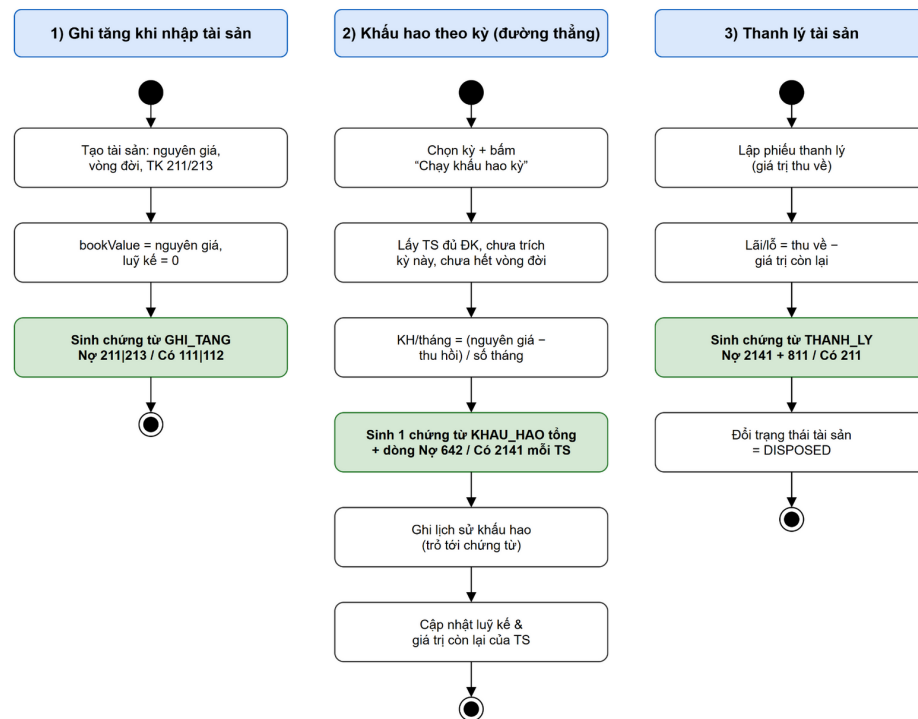
Hình 4.2 — Biểu đồ tuần tự: Ghi tăng tài sản



Hình 4.2. Biểu đồ tuần tự: Ghi tăng tài sản

Luồng khấu hao định kỳ được minh họa ở Hình 4.3: người quản lý chọn kỳ và bấm “Chạy khấu hao kỳ”; hệ thống lấy các tài sản đủ điều kiện chưa trích trong kỳ, tính khấu hao đường thẳng, sinh một chứng từ khấu hao tổng kèm định khoản Nợ 642 / Có 2141 cho từng tài sản, ghi lịch sử khấu hao (trở tới chứng từ) và cập nhật khấu hao lũy kế, giá trị còn lại trên tài sản. Lưu đồ cũng thể hiện hai nghiệp vụ kế toán còn lại: ghi tăng khi nhập tài sản (sinh chứng từ Nợ 211|213 / Có 111|112) và thanh lý (sinh chứng từ Nợ 2141+811 / Có 211, đổi trạng thái tài sản sang DISPOSED) lũy kế và giá trị còn lại, ghi lịch_su_khau_hao và sinh chứng từ (Nợ 627 / Có 214), lập cho đến khi xử lý hết.

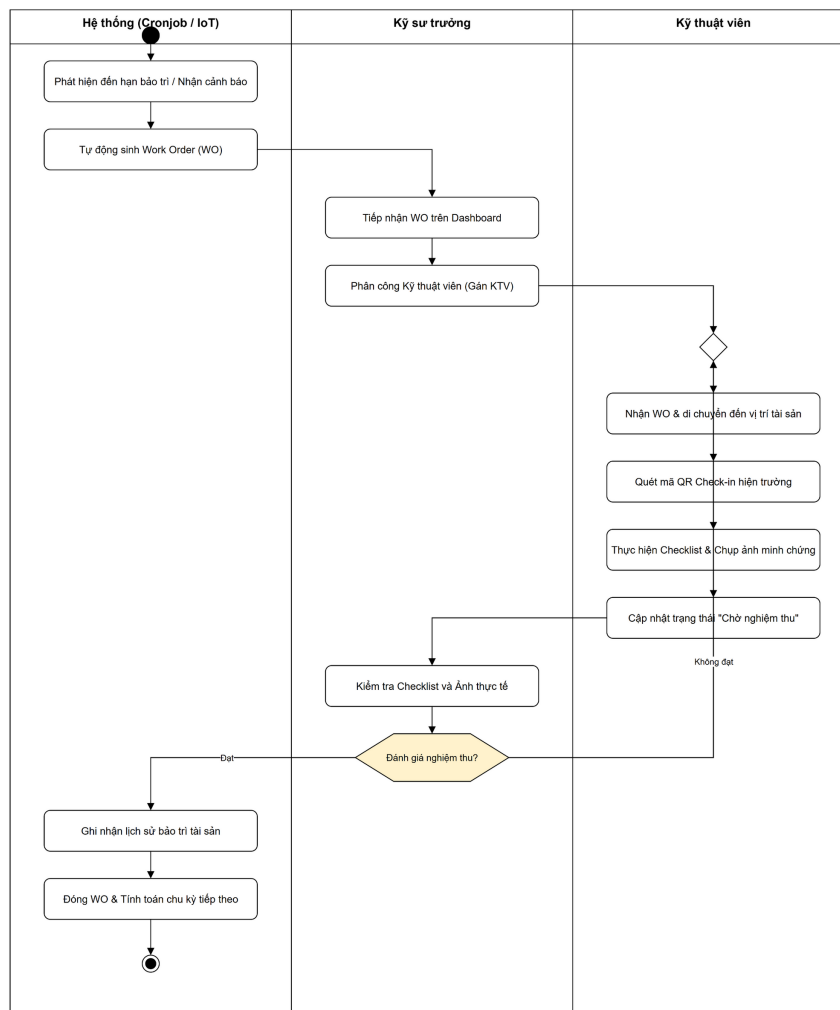
Hình 4.3 — Lưu đồ nghiệp vụ kế toán tài sản (ghi tăng / khấu hao / thanh lý)



Hình 4.3. Lưu đồ nghiệp vụ kế toán tài sản: ghi tăng – khấu hao – thanh lý

Với bảo trì phòng ngừa, quy trình khép kín từ tự động sinh Work Order khi tới hạn, phân công kỹ thuật viên, check-in bằng mã QR tại hiện trường, thực hiện checklist có ảnh minh chứng đến nghiệm thu được thể hiện ở Hình 4.4.

Hình 4.4 — Lưu đồ nghiệp vụ: Bảo trì định kỳ (PM/Work Order)

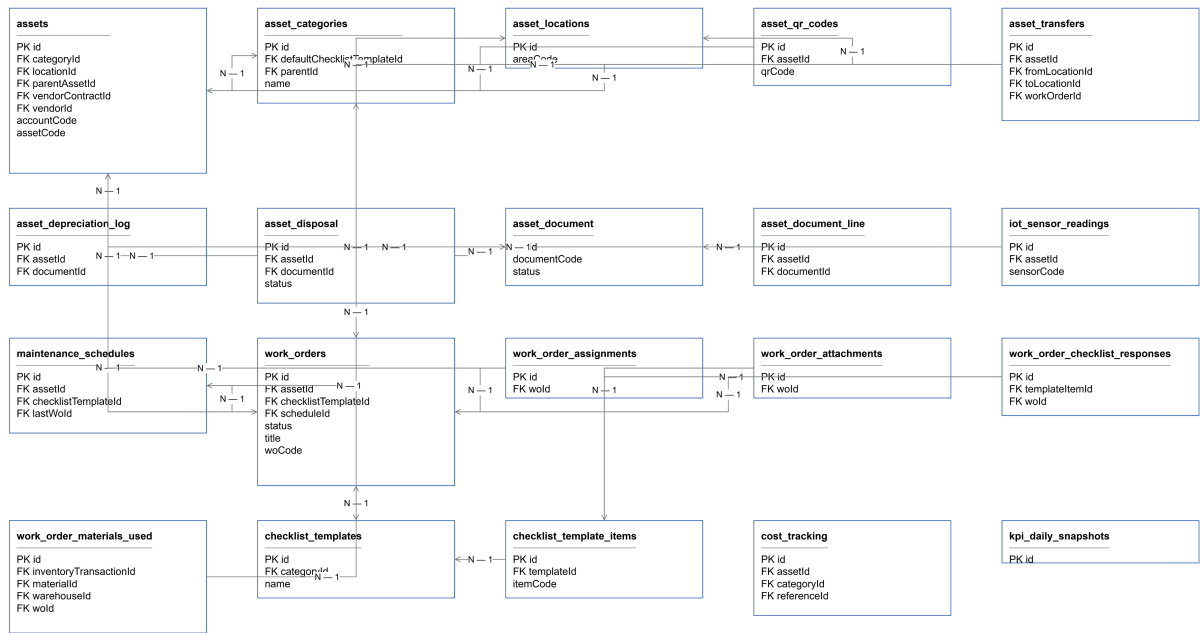


Hình 4.4. Lưu đồ nghiệp vụ: Bảo trì định kỳ (PM/Work Order)

c) Thiết kế cơ sở dữ liệu (schema asset)

Sơ đồ thực thể – liên kết của schema asset (Hình 4.5) lấy bảng `tai_san` làm trung tâm, liên kết tới danh mục, lô, điều chuyển, bảo trì, thanh lý, lịch sử khấu hao và chứng từ kế toán.

ERD schema asset — phân hệ Quản lý tài sản & bảo trì



Hình 4.5. Sơ đồ thực thể – liên kết (ERD) schema asset

Bảng	Ý nghĩa	Trường tiêu biểu
Tài sản (trung tâm)	ma_tai_san, ten_tai_san, danh_muc_id, trang_thai, nguyen_gia, gia_tri_con_lai, khau_hao_luy_ke, phuong_phap_khau_hao danh_muc_tai_san	tai_san Danh mục tài sản
ma_danh_muc, tien_to, thoi_gian_khau_hao, ma_tai_khoan dieu_chuyen_tai_san	Điều chuyển	loai_dieu_chuyen, tu/ den_phong_ban, tu/ den_nguoi_dung bao_tri_tai_san
Bảo trì	loai_bao_tri, chi_phi, loai_chi_phi, trang_thai thanh_ly_tai_san	Thanh lý
gia_tri_con_lai, gia_tri_thanh_ly, lai_lo, ly_do lich_su_khau_hao	Lịch sử khấu hao	ky_khau_hao, so_tien, luy_ke_sau_khau_hao, con_lai_sau_khau_hao MaintenanceSchedule / WorkOrder
Lịch bảo trì & lệnh công việc	chu kỳ, lead time; trạng thái WO, phân công, checklist, đính kèm chung_tu / chi_tiet_chung_tu	Chứng từ & bút toán
loai_chung_tu; tai_khoan_no, tai_khoan_co, so_tien		

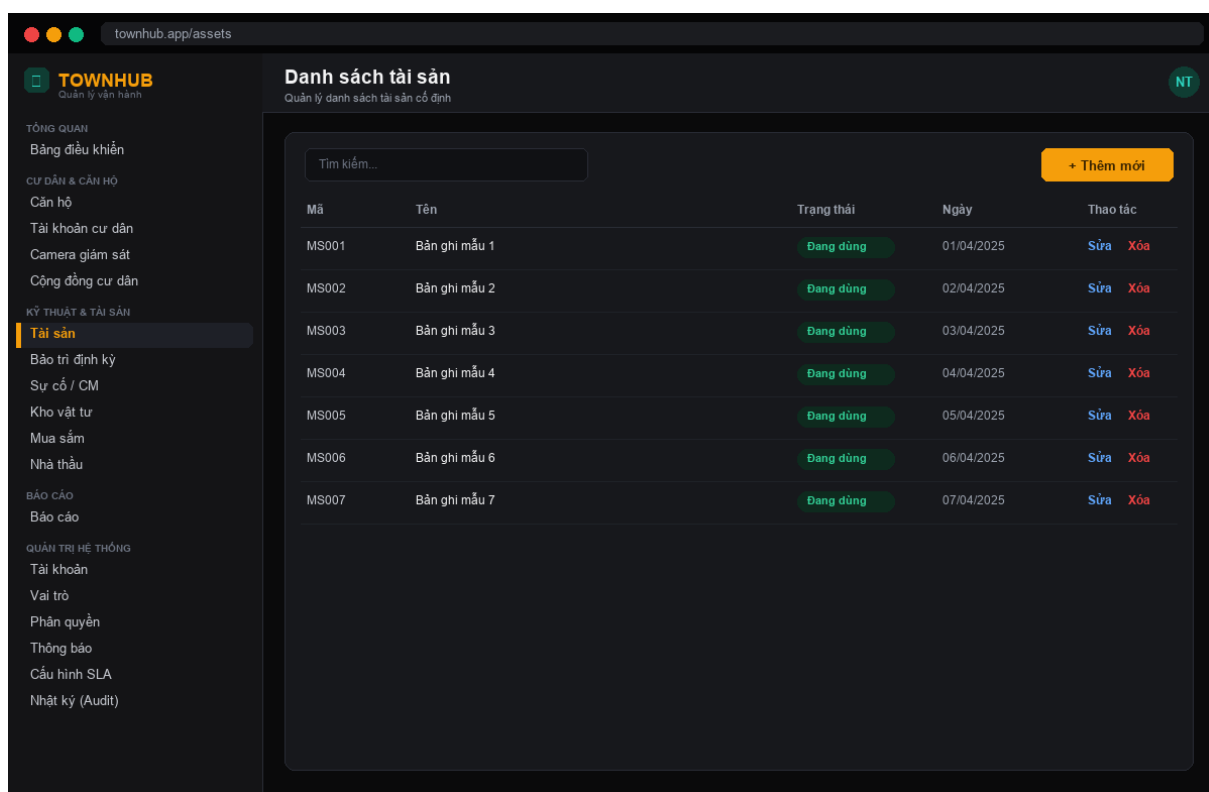
d) Thiết kế API và phân quyền

Phương thức & Endpoint	Chức năng	Quyền
Ghi tăng tài sản	AssetCreate	POST /api/asset/asset/create
AssetView /api/asset/asset-category/*	GET /api/asset/asset/get-all CRUD danh mục	Danh sách (lọc) Asset* /api/asset/asset-qrcode/*
Sinh & tra cứu QR	AssetView/Create /api/asset/asset-transfer/*	Điều chuyển tài sản
AssetUpdate /api/asset/asset-depreciation/*	Tra cứu khấu hao	AssetView /api/asset/asset-report/*
Nhật ký chung, sổ cái, báo cáo	AssetView /api/asset/maintenance-schedule/*	Lịch bảo trì (get-overdue)
AssetView/Update /api/asset/work-order/*	Work Order (assign, checklist, attachment)	AssetView/Update

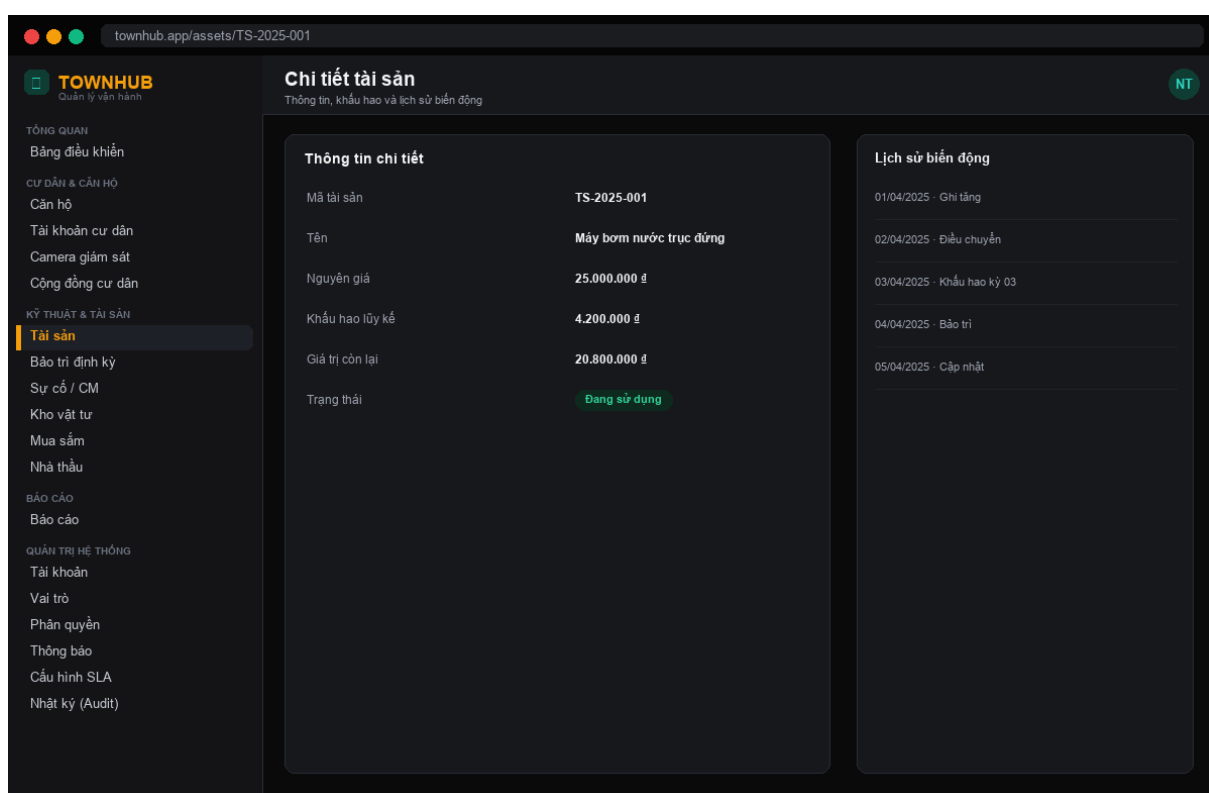
4.1.6. Thiết kế giao diện

Route	Chức năng
Danh sách & chi tiết tài sản /assets/scan	/assets, /assets/[id] Quét QR tra cứu (jsqr) /assets/categories, /assets/depreciation
Danh mục & theo dõi khấu hao /assets/journal, /assets/ledger, /assets/reports	Nhật ký chung, sổ cái & báo cáo kế toán /pm/work-orders, /pm/schedules
Work Order & lịch bảo trì (check-in, checklist, review)	

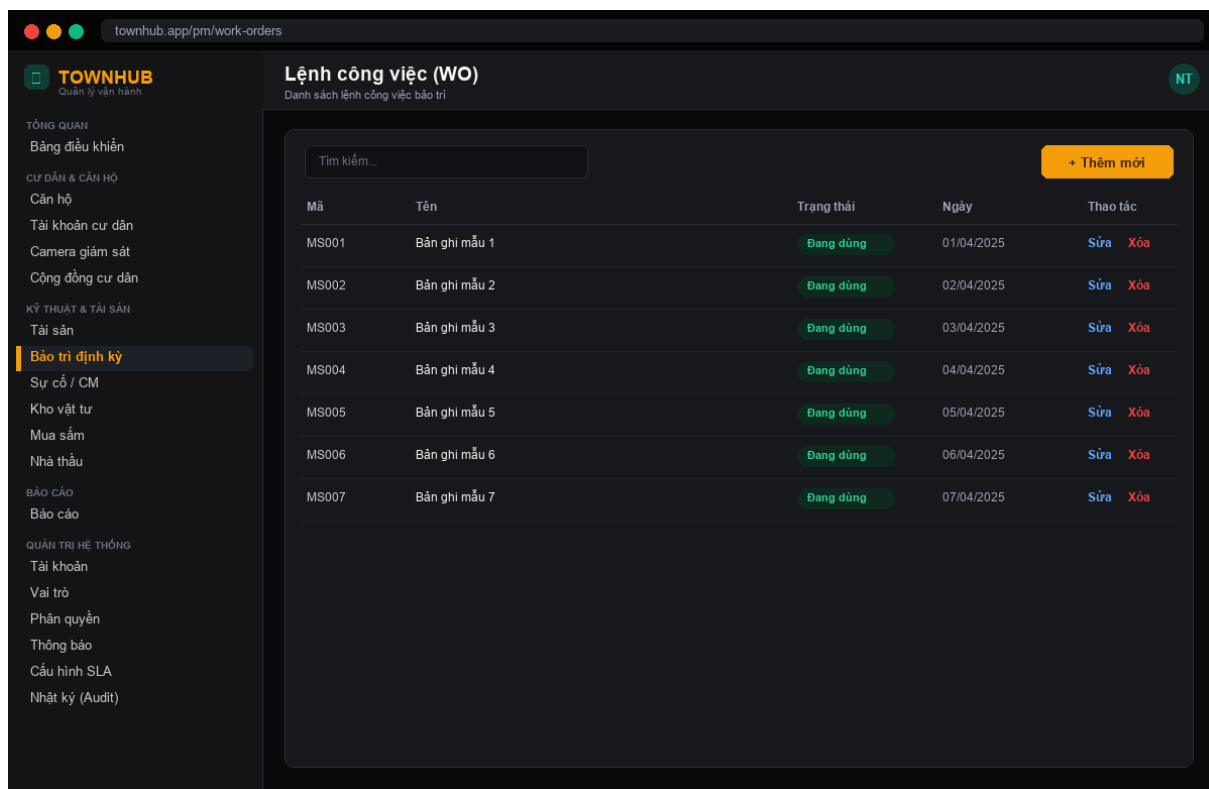
Một số màn hình tiêu biểu của phân hệ:



Hình 4.6. Giao diện Danh sách tài sản



Hình 4.7. Giao diện Chi tiết tài sản (thông số, khấu hao, lịch sử)



Hình 4.8. Giao diện Lệnh công việc bảo trì (Work Order)

4.1.7. Thành phần thông minh: sinh mã & khấu hao tự động

Đây là hai thành phần tự động hóa do tác giả tự thiết kế và cài đặt. Sinh mã tài sản tự động theo định dạng cấu hình (ví dụ {DANH_MUC}-{SO_THU_TU}) với tiền tố lấy từ danh mục; kiểm tra trùng trước khi lưu. Khấu hao chạy tự động đầu tháng khi bật cờ tự động khấu hao, giảm thao tác thủ công của kế toán và bảo đảm khớp sổ kế toán.

4.2. Nghiệp vụ Kho, Mua sắm và Nhà cung cấp

4.2.1. Đặt vấn đề và phạm vi

Hoạt động bảo trì và xử lý sự cố cần vật tư; vật tư cần được quản lý tồn kho, bổ sung qua quy trình mua sắm và làm việc với nhà cung cấp. Phân hệ này quản lý ba mảng liên thông: Kho vật tư (tồn kho, xuất/nhập, kiểm kê), Mua sắm (đề xuất mua PR → đơn mua PO → hóa đơn) và Nhà cung cấp (hồ sơ, hợp đồng, đánh giá), kèm OCR hóa đơn để số hóa chứng từ.

- Quản lý kho, danh mục vật tư và mức tồn kho; cảnh báo tồn thấp.
- Xuất/nhập/điều chỉnh tồn qua phiếu giao dịch kho; kiểm kê.

- Quy trình mua sắm nhiều cấp: PR (đề xuất) → duyệt → PO (đơn mua) → Invoice (hóa đơn).
- Quản lý nhà cung cấp, hợp đồng (cảnh báo sắp hết hạn) và đánh giá hiệu suất.
- OCR hóa đơn tự động (VietOCR) để trích xuất dữ liệu hóa đơn.

4.2.2. Môi liên hệ với Base và các phân hệ khác

Phân hệ do cá nhân em phát triển nhưng không hoạt động cô lập - dữ liệu và nghiệp vụ liên thông chặt chẽ với các phân hệ lân cận, đồng thời kế thừa các dịch vụ dùng chung do cả nhóm xây dựng:

- **Với Bảo trì & Sự cố:** Work Order/ticket tiêu hao vật tư từ kho; khi thiếu vật tư có thể sinh đề xuất mua (PR).
- **Với Tài sản:** nhà cung cấp gắn với hợp đồng và tài sản (vendorId, vendorContractId); hóa đơn mua phục vụ ghi tăng tài sản.
- **Với Base:** phân quyền RBAC, thông báo cảnh báo tồn thấp/hợp đồng sắp hết hạn, tổng hợp chi phí lên Reporting.

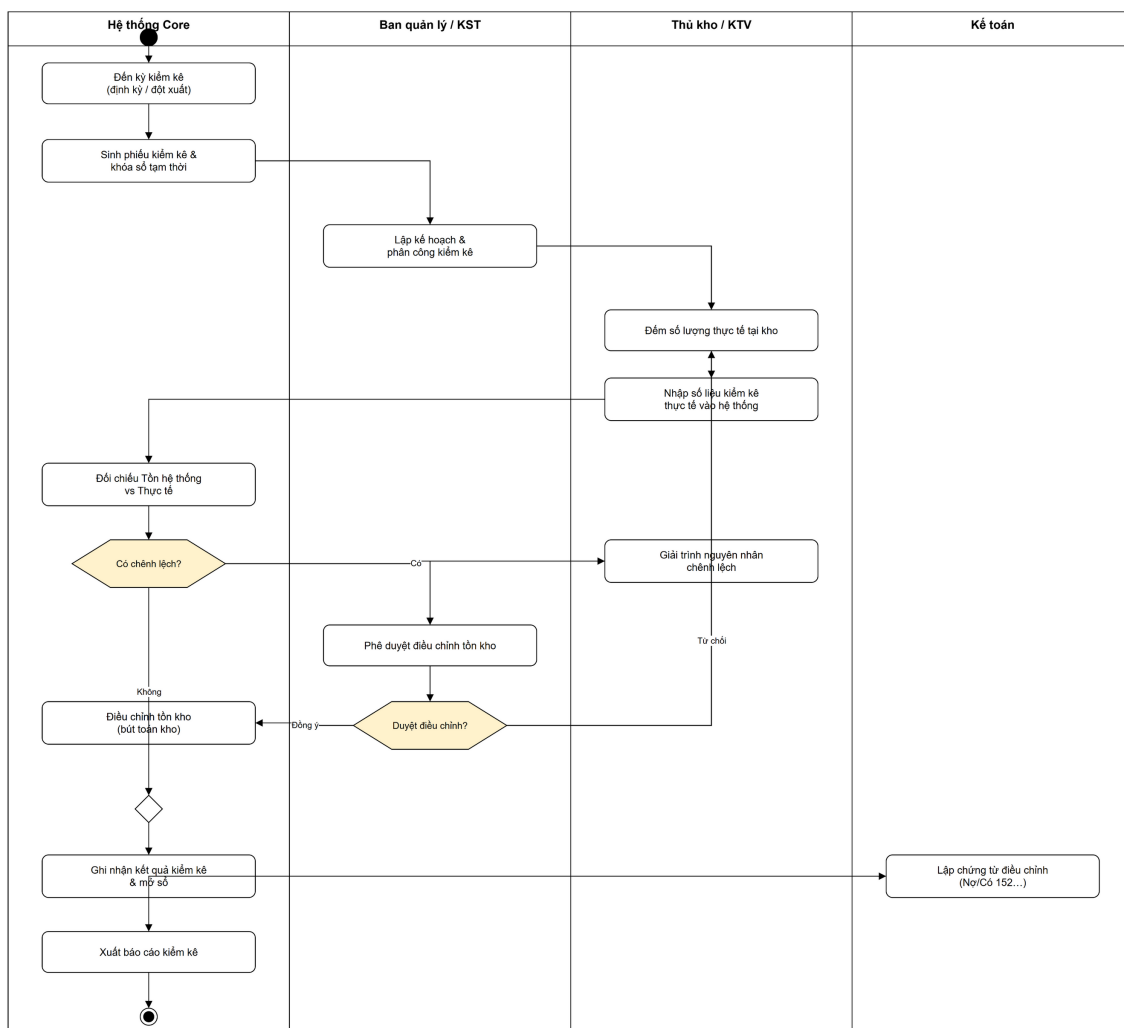
4.2.3. Phân tích yêu cầu chức năng

a) Quản lý kho và vật tư

Khai báo kho (Warehouse), danh mục vật tư (Material) và mức tồn (InventoryLevel). Mọi biến động tồn ghi qua InventoryTransaction (nhập/xuất/điều chỉnh). Hệ thống cảnh báo các vật tư dưới ngưỡng tồn tối thiểu (get-low-stock).

Quy trình kiểm kê kho định kỳ - chốt số liệu tồn hệ thống, đếm thực tế tại kho, đối chiếu chênh lệch và phê duyệt phiếu điều chỉnh - được minh họa ở Hình 4.9.

Hình 4.9 — Lưu đồ nghiệp vụ: Kiểm kê kho định kỳ

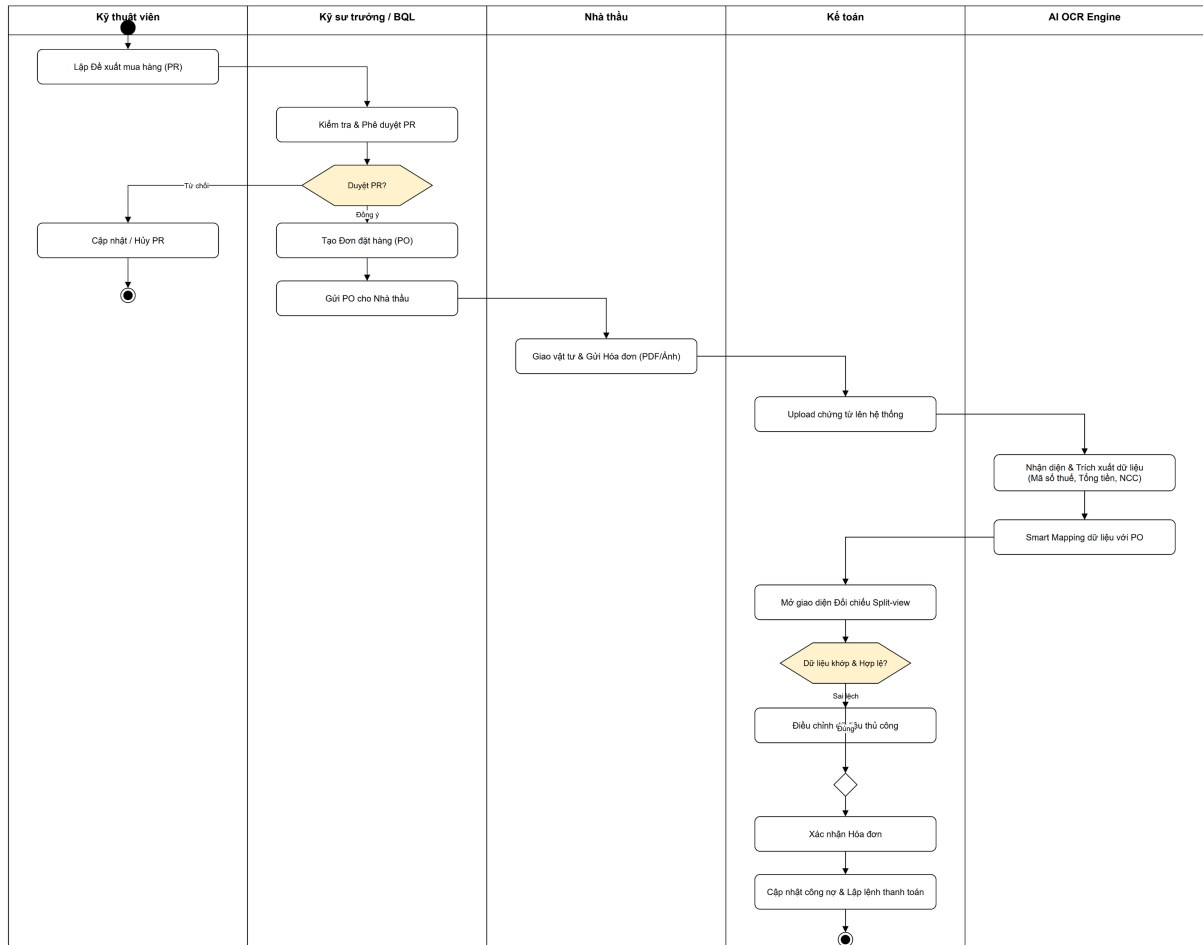


Hình 4.9. Lưu đồ nghiệp vụ: Kiểm kê kho định kỳ

b) Quy trình mua sắm (PR → PO → Invoice)

Người dùng lập Đề xuất mua (PurchaseRequest) gồm nhiều dòng vật tư; quản lý duyệt hoặc từ chối. PR đã duyệt được chuyển thành Đơn mua (PurchaseOrder) gửi nhà cung cấp; khi nhận hàng/hóa đơn tạo Invoice để đối chiếu. Quy trình duyệt nhiều cấp được cấu hình qua PoApprovalWorkflow. Toàn bộ quy trình mua sắm và bước OCR hóa đơn được minh họa ở Hình 4.10.

Hình 4.10 — Lưu đồ nghiệp vụ: Quy trình mua sắm PR → PO và OCR hóa đơn



Hình 4.10. Lưu đồ nghiệp vụ: Quy trình mua sắm PR → PO và OCR hóa đơn

c) Quản lý nhà cung cấp

Quản lý hồ sơ nhà cung cấp (Vendor) với thao tác blacklist/kích hoạt; hợp đồng (Vendor-Contract) có cảnh báo sắp hết hạn (get-expiring) và các dịch vụ theo hợp đồng (Vendor-ContractService); đánh giá hiệu suất định kỳ (VendorEvaluation).

d) OCR hóa đơn tự động

Người dùng tải ảnh hóa đơn; hệ thống đẩy công việc vào hàng đợi `OcrJobQueue`; tiến trình nền `OcrProcessingWorker` gọi `InvoiceOcrEngine` (bản cài đặt `VietOcrEngine`, có `MockOcrEngine` để kiểm thử) trích xuất các trường và tạo bản ghi `Invoice` để người dùng xác nhận. Xử lý nền giúp giao diện không bị chặn và mở rộng được số lượng công việc.

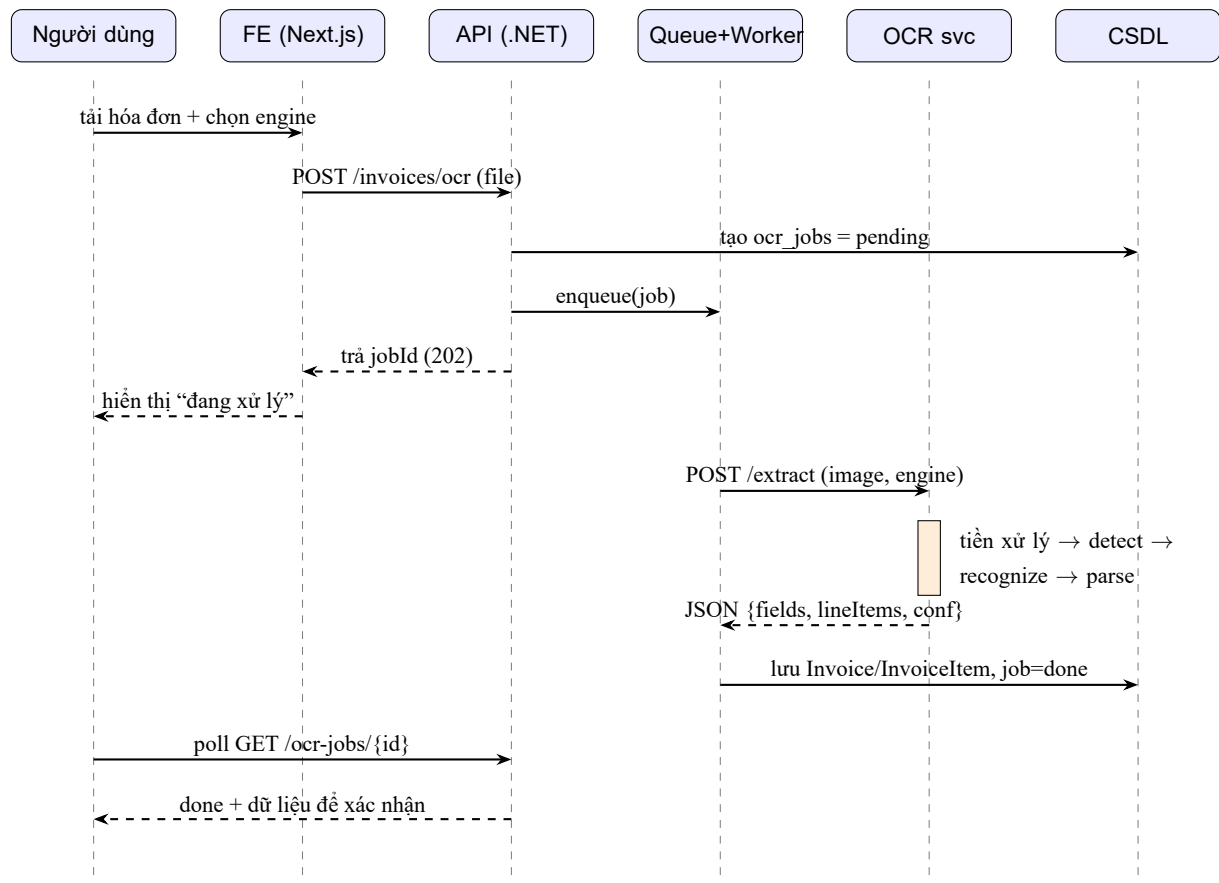
Đặc tả use case. Nghiệp vụ OCR hóa đơn được đặc tả chi tiết ở bảng dưới đây. Đây là use case trung tâm của quy trình mua sắm: nó chuyển thao tác nhập liệu thủ công từng

dòng hàng thành thao tác tải ảnh và xác nhận, đồng thời gắn kết quả trích xuất với Đơn mua (PO) để đối chiếu.

Bảng. Đặc tả use case *Trích xuất hóa đơn tự động (UC-OCR)*

Mã use case	UC-OCR — Trích xuất hóa đơn tự động
Tác nhân chính	Nhân viên kho / Nhân viên mua sắm (<i>Người dùng</i>)
Tác nhân phụ	Dịch vụ OCR (FastAPI), tiến trình nền <i>OcrProcessingWorker</i>
Mục tiêu	Số hóa hóa đơn giấy/PDF thành các trường có cấu trúc (số hóa đơn, ngày, MST, tiền hàng, thuế, tổng) và bảng chi tiết hàng hóa, giảm nhập liệu thủ công và sai sót.
Tiền điều kiện	Người dùng đã đăng nhập, có quyền trên phân hệ Kho–Mua sắm; đã tồn tại PO cần đối chiếu (tùy chọn); dịch vụ OCR đang chạy.
Kích hoạt	Người dùng chọn chức năng <i>Upload hóa đơn</i> và tải lên tệp ảnh/PDF.
Luồng chính	1. Người dùng chọn engine OCR (mặc định paddleocr) và tải tệp hóa đơn lên. 2. Backend .NET tạo bản ghi <i>ocr_jobs</i> (trạng thái <i>pending</i>) và đẩy vào hàng đợi <i>OcrJobQueue</i>, trả về ngay <i>jobId</i> (không chặn giao diện). 3. <i>OcrProcessingWorker</i> lấy job khỏi hàng đợi, gọi <i>IInvoiceOcrEngine</i> → HTTP POST <i>/extract</i> sang dịch vụ OCR. 4. Dịch vụ OCR tiền xử lý ảnh, chạy phát hiện + nhận dạng theo engine đã chọn, hậu xử lý (khử nghiêng, gom hàng, trích trường/bảng) và trả về JSON {fields, lineItems, confidence}. 5. Worker chuẩn hóa kết quả, tạo bản ghi <i>Invoice/InvoiceItem</i> (trạng thái <i>chờ xác nhận</i>) và cập nhật <i>ocr_jobs</i> = <i>done</i>. 6. Người dùng mở màn hình xác nhận, đối chiếu với ảnh gốc và với PO, chỉnh sửa nếu cần rồi lưu chính thức.
Luồng thay thế / ngoại lệ	3a. Tệp là PDF có lớp văn bản: hệ thống đọc thẳng text-layer, bỏ qua OCR ảnh. 4a. Bố cục lạ hoặc chất lượng ảnh kém: người dùng chọn lại engine <i>gemini</i> để trích bằng mô hình thị giác–ngôn ngữ. 4b. Dịch vụ OCR lỗi/timeout: job chuyển <i>failed</i>, ghi log lỗi; người dùng có thể thử lại hoặc nhập tay. 6a. Độ tin cậy (confidence) thấp hơn ngưỡng: hệ thống tô cảnh báo các trường nghi ngờ để người dùng kiểm tra kỹ.
Hậu điều kiện	Có bản ghi <i>Invoice</i> đã xác nhận, liên kết PO; số liệu sẵn sàng cho đối chiếu công nợ và báo cáo.
Yêu cầu phi chức năng	Xử lý bất đồng bộ (không chặn UI); truy vết trạng thái job; thay đổi engine không ảnh hưởng phần bóc tách trường.

Luồng xử lý theo thời gian. Biểu đồ tuần tự dưới đây mô tả luồng của use case, làm rõ tính *bất đồng bộ*: backend trả jobId cho người dùng ngay khi nhận tệp, còn việc gọi dịch vụ OCR và tạo Invoice diễn ra ở tiến trình nền; giao diện hỏi trạng thái theo chu kỳ (polling) cho tới khi job hoàn tất.



Hình. Biểu đồ tuần tự use case Trích xuất hóa đơn tự động (luồng bất đồng bộ hàng đợi-worker).

4.2.4. Yêu cầu phi chức năng

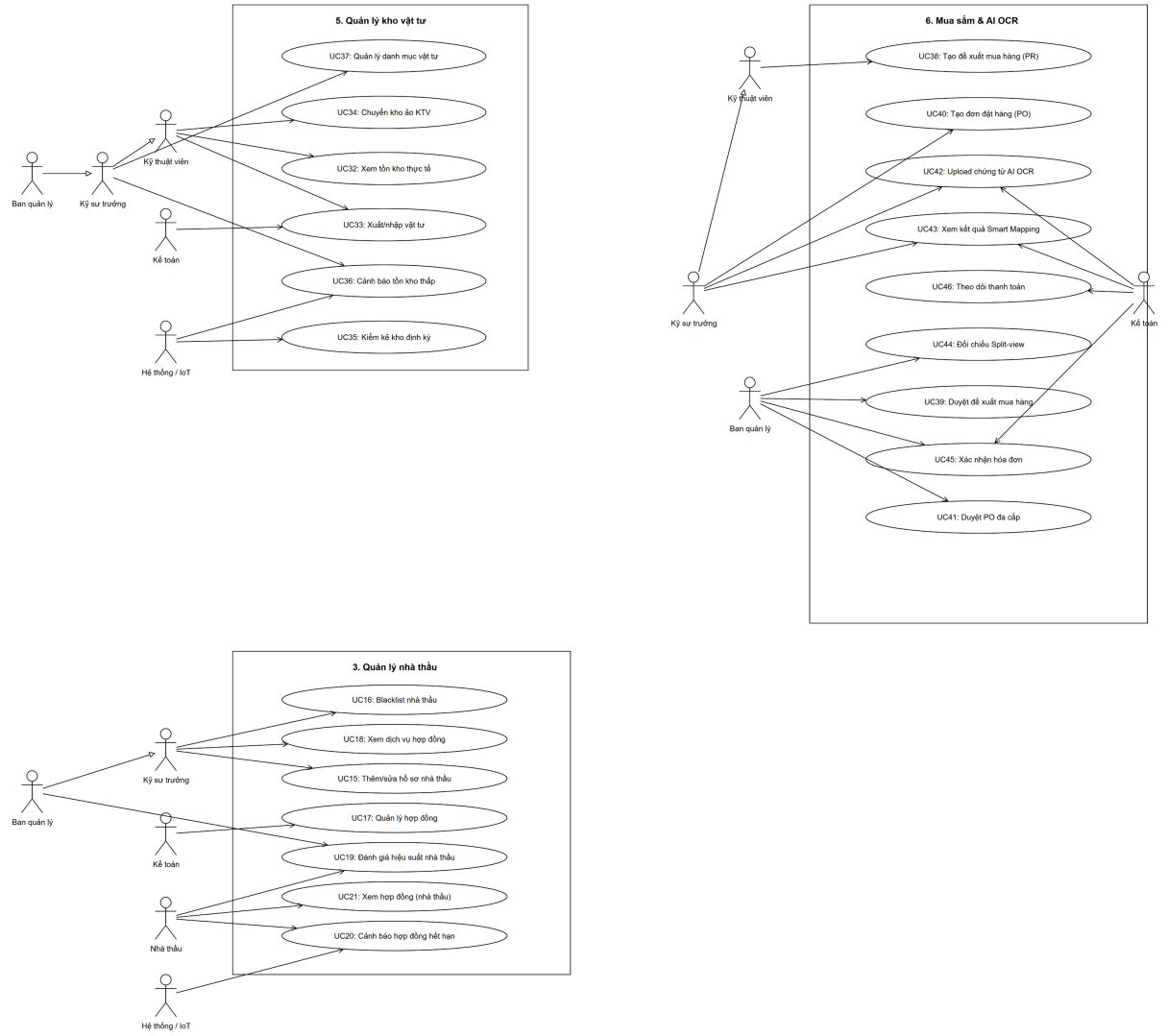
- Nhất quán tồn kho khi nhiều giao dịch đồng thời.
- Truy vết: mọi PR/PO/giao dịch kho đều lưu lịch sử và người thực hiện.

4.2.5. Thiết kế phân hệ

a) Danh sách Use Case chính

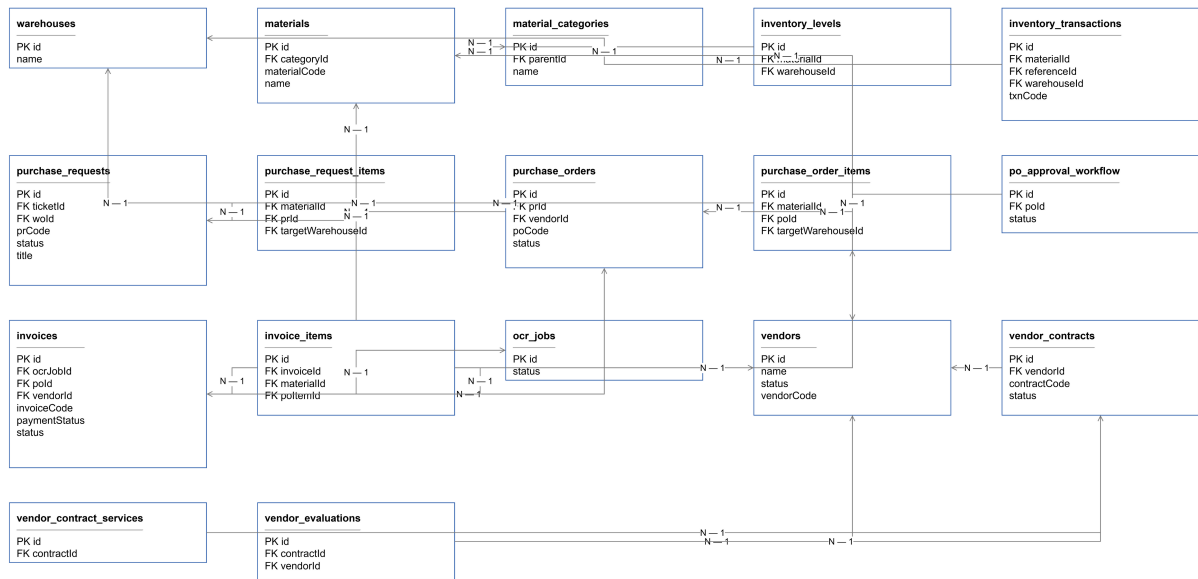
Biểu đồ use case của phân hệ Kho – Mua sắm – Nhà cung cấp được thể hiện ở Hình 4.11.

Hình 4.11 — Biểu đồ Use Case: Kho, Mua sắm và Nhà cung cấp



Hình 4.11. Biểu đồ Use Case – Kho, Mua sắm và Nhà cung cấp

ERD schema asset — phân hệ Kho, Mua sắm & Nhà cung cấp



Hình 4.12. Sơ đồ thực thể – liên kết (ERD) phân hệ Kho, Mua sắm và Nhà cung cấp

Mã UC	Use case	Tác nhân
Quản lý kho & vật tư, mức tồn	Thủ kho UC-INV-02	Nhập/xuất/kiểm kê vật tư UC-INV-01
Thủ kho UC-PR-01	Lập & duyệt/từ chối đề xuất mua (PR)	NV mua sắm/Quản lý UC-PO-01
Tạo đơn mua (PO) & hóa đơn	NV mua sắm UC-VD-01	Quản lý nhà cung cấp & hợp đồng
Quản lý UC-VD-02	Đánh giá hiệu suất nhà cung cấp	Quản lý UC-OCR-01
OCR hóa đơn tự động	NV mua sắm/Hệ thống	

b) Thiết kế cơ sở dữ liệu (thực thể chính)

Thực thể	Ý nghĩa
Kho, danh mục vật tư, mức tồn	Warehouse, Material, InventoryLevel
InventoryTransaction	Phiếu nhập/xuất/điều chỉnh tồn
Đề xuất mua và dòng vật tư	PurchaseRequest / PurchaseRequestItem
PurchaseOrder / PurchaseOrderItem	Đơn mua và dòng hàng
Hóa đơn, dòng hóa đơn, công việc OCR	Invoice / InvoiceItem, OcrJob
Vendor, VendorContract, VendorContractService, VendorEvaluation	Nhà cung cấp, hợp đồng, dịch vụ, đánh giá

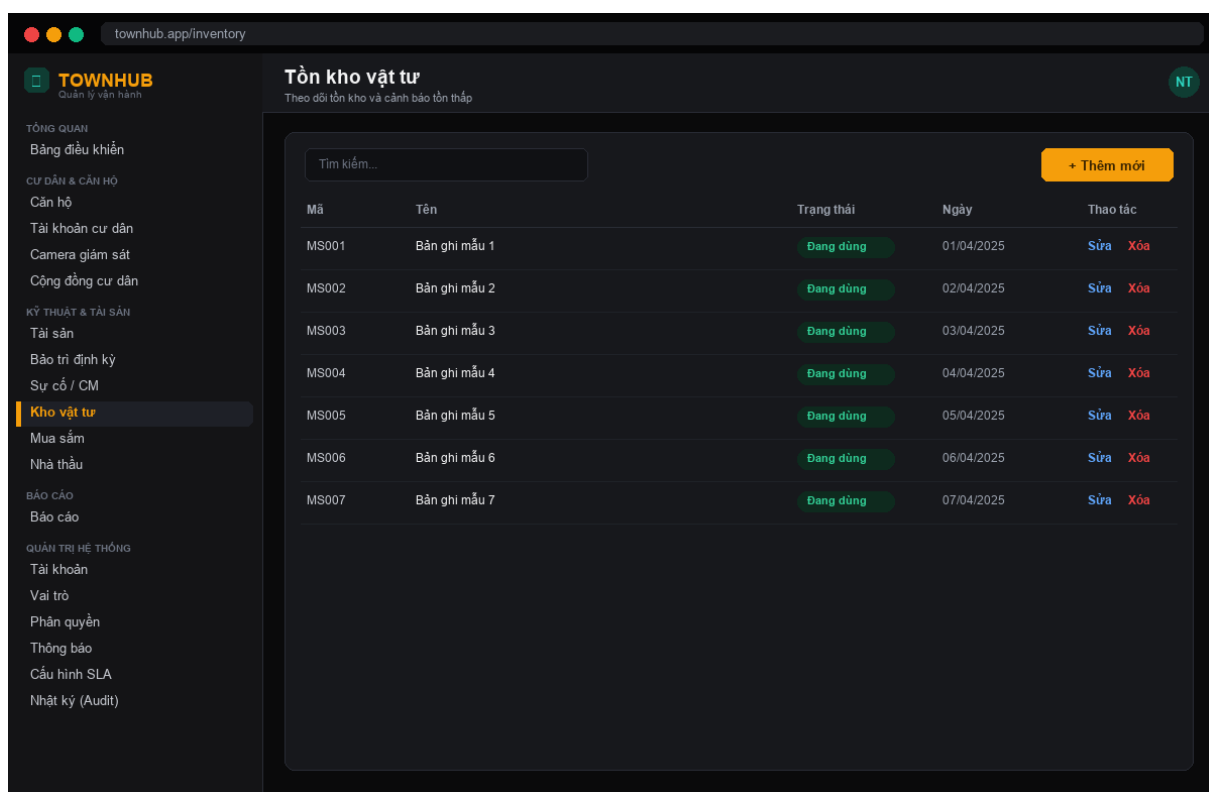
c) Thiết kế API

Nhóm Endpoint	Chức năng
CRUD kho & vật tư; get-low-stock, get-inventory-levels	/api/asset/warehouse/*, /api/asset/material/*
/api/asset/inventory-transaction/*	Tạo & tra cứu giao dịch kho
PR: tạo, duyệt (approve), từ chối (reject), thêm dòng	/api/asset/purchase-request/*
/api/asset/purchase-order/*	PO: tạo, cập nhật, thêm dòng
NCC: CRUD, blacklist, activate	/api/asset/vendor/*
/api/asset/vendor-contract/*	Hợp đồng: CRUD, get-expiring
Đánh giá theo nhà cung cấp/hợp đồng	/api/asset/vendor-evaluation/*

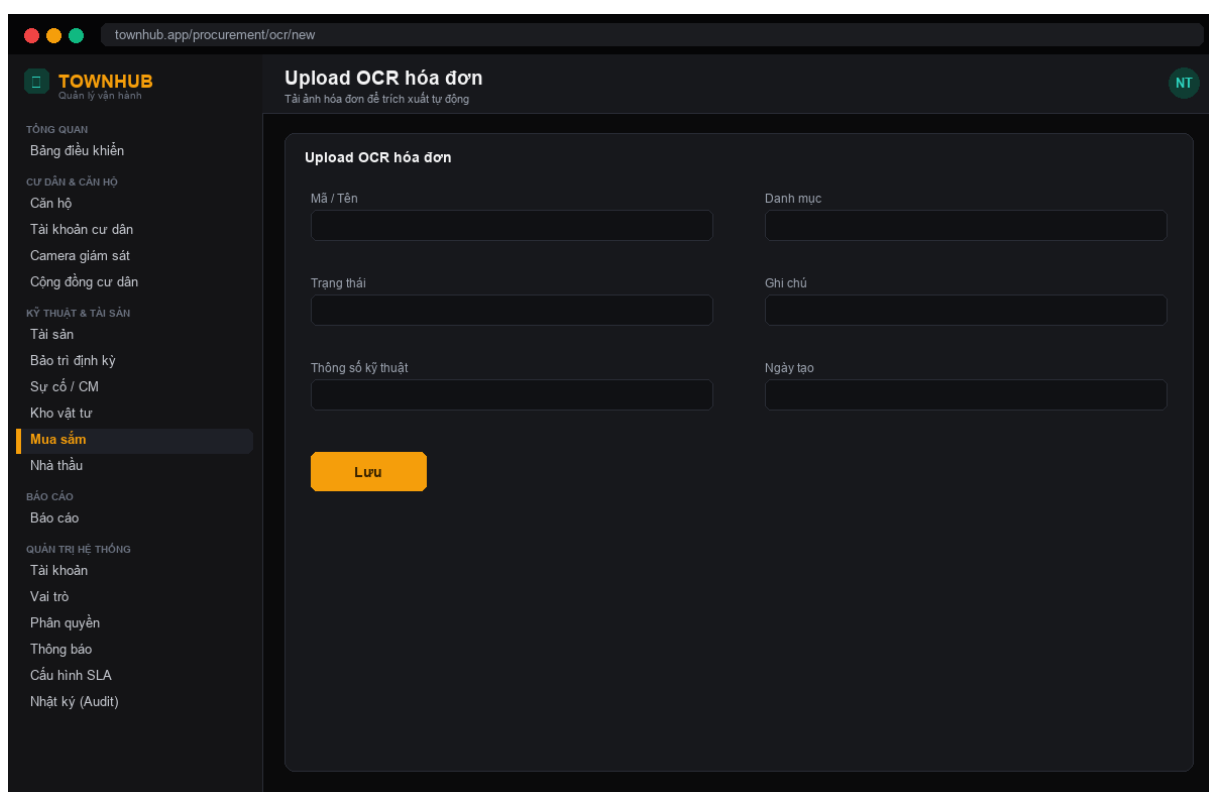
4.2.6. Thiết kế giao diện

Route	Chức năng
Tồn kho & danh mục vật tư	/inventory, /inventory/catalog
/inventory/stock-taking, /inventory/transactions/new	Kiểm kê & tạo giao dịch
PR / PO / hóa đơn	/procurement/requests, /procurement/orders, /procurement/invoices
/procurement/ocr/new, /procurement/ocr/[id]/verify	OCR hóa đơn & xác nhận
Nhà cung cấp, hợp đồng, hiệu suất	/vendors, /vendors/contracts, /vendors/performance

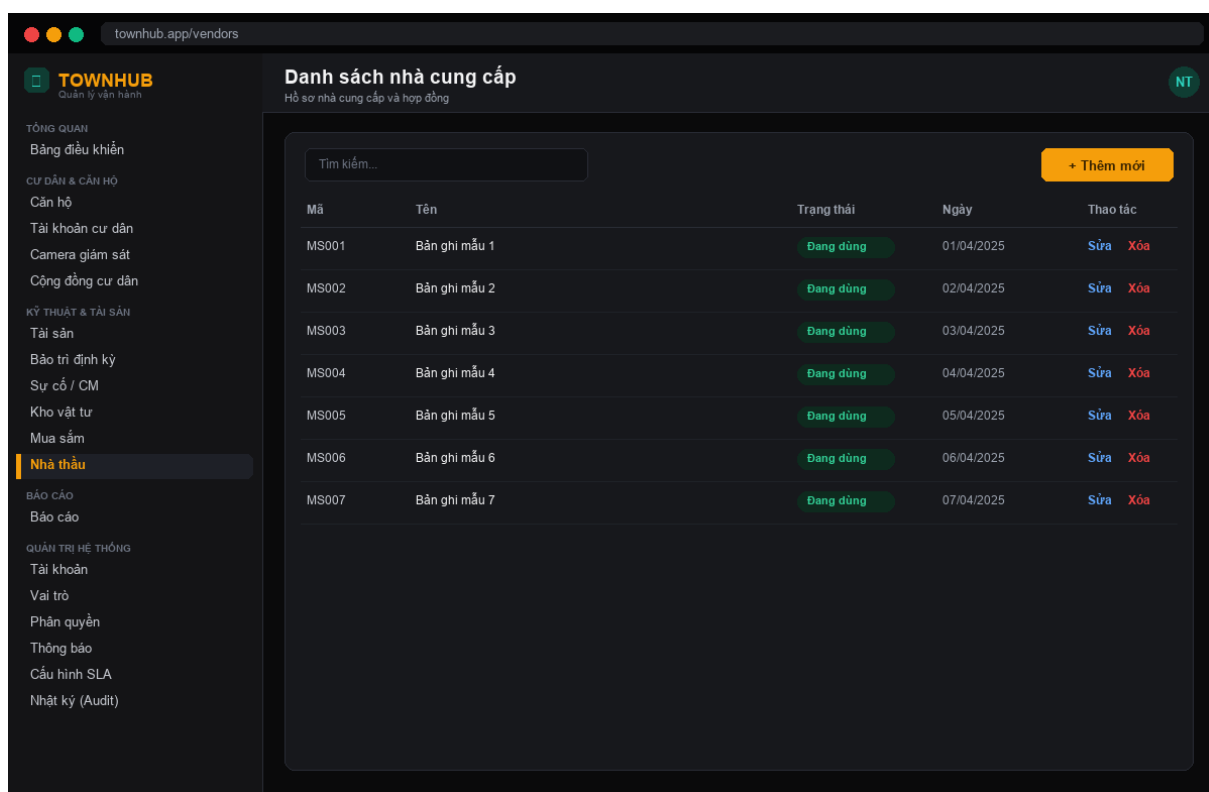
Một số màn hình tiêu biểu của phân hệ:



Hình 4.13. Giao diện Tồn kho vật tư (cảnh báo tồn thấp)



Hình 4.14. Giao diện Upload OCR hóa đơn



Hình 4.15. Giao diện Danh sách nhà cung cấp

4.2.7. Chi tiết kỹ thuật OCR hóa đơn: mô hình, huấn luyện và đánh giá

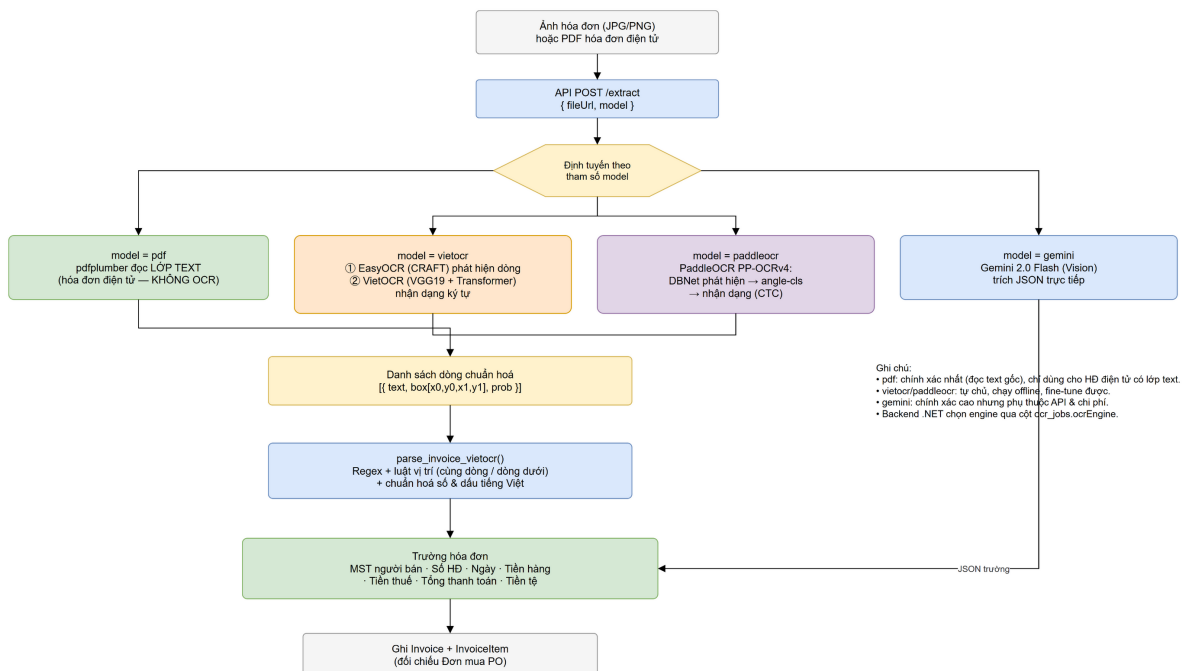
Mục này trình bày chi tiết thành phần AI/OCR dùng để số hóa hóa đơn trong phân hệ Kho – Mua sắm: kiến trúc dịch vụ, cơ sở lý thuyết các mô hình, quá trình huấn luyện trên dữ liệu tự sinh và kết quả đánh giá – so sánh giữa các engine. Đây là phần đóng góp kỹ thuật của tác giả cho phân hệ.

a) Kiến trúc dịch vụ OCR đa engine

Dịch vụ OCR viết bằng Python/FastAPI, tách khỏi backend .NET và gọi qua HTTP POST /extract; chạy bất đồng bộ theo hàng đợi – tiến trình nền (OcrJobQueue → OcrProcessing-Worker). Dịch vụ hỗ trợ bốn chế độ chọn được trên giao diện (cột ocr_jobs.ocrEngine), vừa để so sánh mô hình, vừa để chọn engine phù hợp từng loại chứng từ. Ba engine OCR đều quy kết quả về một cấu trúc dòng chuẩn hóa [{text, box, prob}] rồi dùng chung một hàm bóc tách trường (regex + luật vị trí), nên thay/đổi engine không ảnh hưởng phần lấy trường.

model	Phát hiện (detect)	Nhận dạng (recognize)	Fine-tune	Đặc điểm pdf
- (đọc lớp text)	pdfplumber	-	Hóa đơn điện tử có lớp text → đọc trực tiếp, <u>chính xác nhất vietocr</u>	EasyOCR (CRAFT)
VietOCR (VGG + Transformer)	recognition	Tự chủ, mạnh về dấu tiếng Việt <u>paddleocr</u>	PaddleOCR (DBNet)	PP-OCRv4 (SVTR_LCNet)
detect + recognize	Hợp nhất, nhẹ, fine-tune cả hai tầng <u>gemini</u>	(mô hình đa thể lo)	Gemini 2.0 Flash	- (API đóng)
Chính xác cao, phụ thuộc API và chi phí				

Hình 2.x — Kiến trúc pipeline trích xuất hóa đơn (dịch vụ OCR TownHub)



Hình 4.16. Kiến trúc pipeline trích xuất hóa đơn (dịch vụ OCR bốn engine)

b) Cơ sở lý thuyết và thuật toán các mô hình

VietOCR (vgg_transformer) - chỉ nhận dạng. Gồm hai khối: (1) trích đặc trưng thị giác bằng CNN VGG19 (batch-norm) - nén chiều cao ảnh dòng về 1, giữ chiều rộng

làm chuỗi đặc trưng; (2) Transformer encoder–decoder (seq2seq + attention) sinh lần lượt từng ký tự, huấn luyện bằng cross-entropy trên từ điển tiếng Việt (dịch vụ giải mã greedy). Ưu điểm: đọc dấu thanh rất tốt; nhược điểm: chỉ nhận dạng nên buộc ghép một bộ phát hiện bên ngoài.

PaddleOCR (PP-OCRv4) - phát hiện + nhận dạng hợp nhất. Ba bước: (1) DBNet (Differentiable Binarization) dự đoán bản đồ xác suất và bản đồ ngưỡng rồi nhị phân hóa khả vi ngay trong huấn luyện, tách biên vùng chữ gọn; (2) phân loại góc xoay dòng đúng chiều; (3) nhận dạng SVTR_LCNet huấn luyện theo cơ chế GTC kết hợp CTC và NRTR (ghi nhận trong train.log qua CTCLoss/NRTRLoss), dùng bảng ký tự tiếng Việt dict_vi.txt (137 ký tự). Ưu điểm: một bộ công cụ, det + rec đồng bộ, fine-tune được cả hai tầng, nhẹ, chạy offline.

EasyOCR và Gemini. Pipeline vietocr chỉ mượn bộ phát hiện CRAFT của EasyOCR để cắt dòng (nhận dạng giao cho VietOCR). Gemini 2.0 Flash là mô hình ngôn ngữ – thị giác, nhận thẳng ảnh và trả về JSON các trường; chính xác cao nhưng phụ thuộc API và chi phí nên giữ làm tùy chọn.

Chi tiết kiến trúc và công thức các mô hình. Phần dưới đi sâu vào cơ chế toán học của từng thành phần để làm rõ vì sao mỗi mô hình phù hợp với một khâu của bài toán đọc hóa đơn (phát hiện vùng chữ, nhận dạng chuỗi, và huấn luyện đầu–cuối).

(1) *VietOCR — trích đặc trưng VGG19 và bộ giải mã Transformer.* Ảnh dòng chữ $I \in \mathbb{R}^{H \times W \times 3}$ được đưa qua CNN VGG19 (kèm batch-norm). Các tầng gộp (pooling) nén chiều cao về 1 nhưng giữ chiều rộng, tạo chuỗi đặc trưng thị giác

$$X = (x_1, x_2, \dots, x_T), \quad x_t \in \mathbb{R}^d,$$

với T tỉ lệ theo bề rộng ảnh (mỗi x_t ứng với một ”lát” dọc của dòng chữ). Chuỗi này được bộ mã hóa Transformer tinh chỉnh bằng *tự chú ý đa đầu* (multi-head self-attention):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

trong đó Q, K, V là các phép chiếu tuyến tính của X (kèm mã vị trí — positional encoding — để giữ thứ tự trái–phải của ký tự). Bộ giải mã sinh lần lượt từng ký tự $y_1, \dots, y_L$ theo cơ chế tự hồi quy, tối thiểu hóa hàm mất mát entropy chéo:

$$\mathcal{L}_{\text{seq2seq}} = - \sum_{t=1}^L \log P(y_t \mid y_{<t}, X).$$

Khi suy luận, dịch vụ dùng giải mã tham lam (greedy): mỗi bước chọn $y_t = \arg \max_y P(y \mid y_{<t}, X)$. Nhờ mô hình hóa phụ thuộc dài giữa các ký tự và có từ điển tiếng Việt riêng, VietOCR đọc dấu thanh (huyền, sắc, ngã, hỏi, nặng) rất chính xác; nhược điểm là mô hình *chỉ nhận dạng*, buộc phải ghép một bộ phát hiện dòng bên ngoài.

(2) *PaddleOCR (PP-OCRv4)* — phát hiện DBNet, phân loại góc, nhận dạng SVTR_LCNet. Tầng phát hiện dùng DBNet với cơ chế nhị phân hóa khả vi (Differentiable Binarization). Thay cho phép nhị phân hóa cứng (không khả vi) trên bản đồ xác suất P , DBNet học đồng thời một bản đồ ngưỡng T và xấp xỉ hàm bậc thang bằng một sigmoid dốc:

$$\hat{B}_{i,j} = \frac{1}{1 + e^{-k(P_{i,j} - T_{i,j})}},$$

với k là hệ số khuếch đại (thường $k = 50$). Vì $\hat{B}$ khả vi nên biên vùng chữ được huấn luyện trực tiếp bằng lan truyền ngược; tổng mất mát gồm ba thành phần

$$\mathcal{L}_{\text{det}} = \mathcal{L}_s + \alpha \mathcal{L}_b + \beta \mathcal{L}_t,$$

lần lượt cho bản đồ xác suất, bản đồ nhị phân xấp xỉ và bản đồ ngưỡng. Sau phát hiện, một bộ phân loại góc (cls) xoay mỗi dòng về đúng chiều $0^\circ/180^\circ$. Tầng nhận dạng SVTR_LCNet kết hợp trộn đặc trưng cục bộ và toàn cục (local/global mixing) trên một xương sống nhẹ (LCNet), phù hợp chạy CPU. PP-OCRv4 huấn luyện recognizer theo cơ chế GTC (Guided Training of CTC): nhánh chính dùng mất mát CTC

$$\mathcal{L}_{\text{CTC}} = -\log \sum_{\pi \in \mathcal{B}^{-1}(y)} P(\pi \mid X),$$

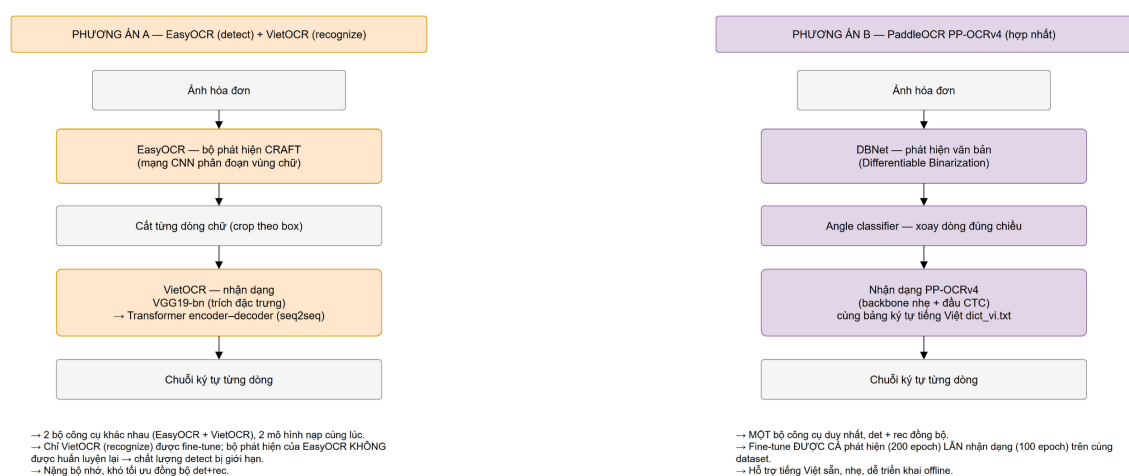
trong đó $\mathcal{B}$ là toán tử gộp khung—bỏ ký tự trống (blank) ánh xạ đường căn chỉnh π về chuỗi nhãn y ; CTC cho phép huấn luyện không cần căn chỉnh ký tự—khung thủ công. Một nhánh chú ý phụ (NRTR) ”dẫn dắt” nhánh CTC trong lúc huấn luyện (ghi nhận qua CTCloss/NRTRloss trong `train.log`) rồi được lược bỏ khi suy luận để giữ tốc độ. Ưu điểm nổi bật: một bộ công cụ hợp nhất, *fine-tune được cả hai tầng det+rec* trên cùng dữ liệu tự gán nhãn, nhẹ và chạy offline.

(3) *EasyOCR/CRAFT* và *Gemini*. Pipeline vietocr chỉ mượn bộ phát hiện CRAFT của EasyOCR để định vị và cắt dòng (CRAFT dự đoán bản đồ vùng ký tự và bản đồ liên kết giữa các ký tự để gom thành từ/dòng), phần nhận dạng giao cho VietOCR. Vì CRAFT không được *fine-tune* trên dữ liệu hóa đơn nên trở thành nút cổ chai của pipeline này. Gemini 2.0 Flash là mô hình thị giác—ngôn ngữ (VLM): nhận thẳng ảnh và một chỉ dẫn

(prompt) yêu cầu trả về JSON các trường; độ chính xác cao với bố cục lạ nhưng phụ thuộc API bên ngoài và chi phí, nên được giữ làm phương án dự phòng.

So sánh vai trò. Tổng hợp lại: DBNet/CRAFT giải bài toán *định vị* (đâu là dòng chữ), còn VietOCR/SVTR_LCNet giải bài toán *đọc* (dòng chữ đó là gì); CTC và seq2seq là hai họ hàm mất mát khác nhau cho khâu đọc — CTC căn chỉnh ngẫm và suy luận nhanh (một lượt), seq2seq+attention chính xác đầu thanh nhưng suy luận tuần tự nên chậm hơn. Chính sự khác biệt này dẫn tới các đánh đổi định lượng ở mục d) và lý do chọn engine ở mục e).

Hình 2.y — Luồng AI 2 tầng (EasyOCR + VietOCR) so với PaddleOCR hợp nhất



Hình 4.17. Luồng AI hai tầng (EasyOCR + VietOCR) so với PaddleOCR hợp nhất

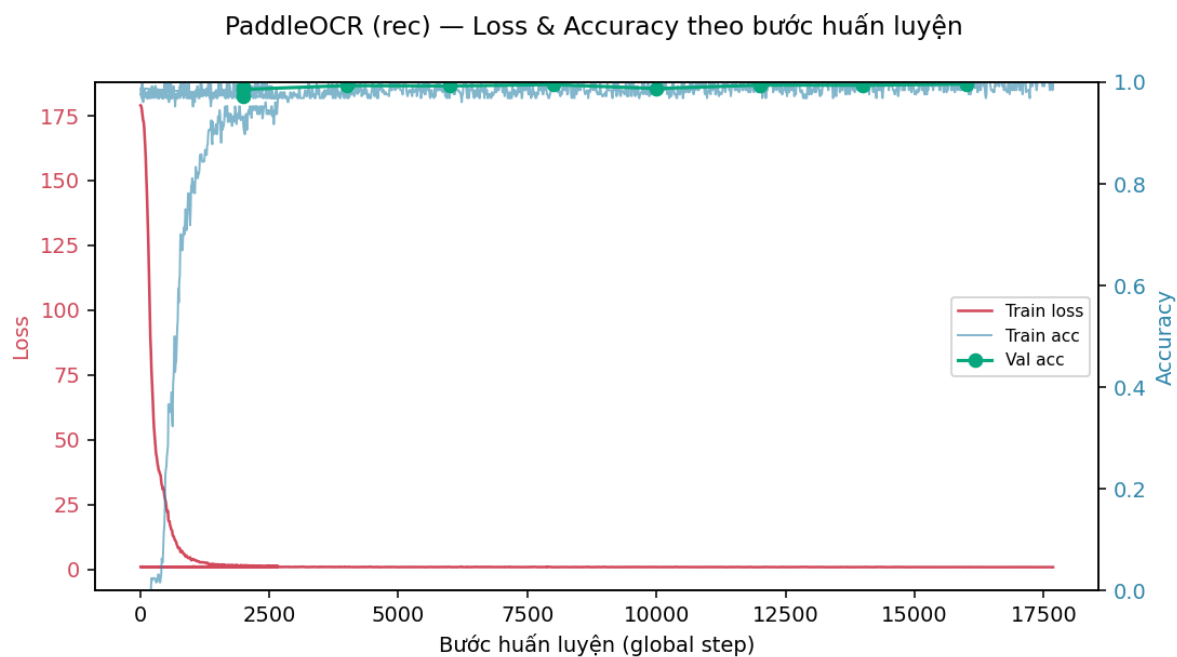
c) Dữ liệu, huấn luyện và luồng xử lý

Vì không có sẵn dữ liệu hóa đơn thật đã gán nhãn, đề tài tự sinh dữ liệu hóa đơn GTGT tiếng Việt tổng hợp (synthetic): script tự render hóa đơn và tự ghi lại tọa độ + nội dung từng ô chữ, nên xuất được cả nhãn phát hiện (det) lẫn nhận dạng (rec) mà không cần gán tay, kèm bảng ký tự dict_vi.txt. Dữ liệu được random đa dạng bố cục và tăng cường cho “giống thật” (nền giấy, nghiêng/méo, nhiễu, nén JPEG, dấu mực đỏ, chữ ký tay); mọi phép biến hình đều kéo theo tọa độ box nên nhãn luôn khớp. Quy mô mặc định 400–500 hóa đơn, chia train/val 90/10. Cả hai mô hình đều fine-tune từ trọng số pretrained (transfer learning) để hội tụ nhanh, cần ít dữ liệu. Cấu hình thật (đọc từ config.yml):

Mô hình	Thuật toán / Khởi tạo	Epoch	Batch	Optimizer / LR VietOCR (rec)
VGG19 + Transformer, từ vgg_transformer	20 000 iters	32	mặc định VietOCR PaddleOCR rec	SVTR_LCNet + GTC(CTC+NRTR), từ PP-OCRv4
80	64	Adam + Cosine, lr 0,001 PaddleOCR det	DBNet, từ PP-OCRv4 det	150
8	lr 0,001			

Luồng xử lý. Pipeline vietocr: ảnh → EasyOCR/CRAFT phát hiện dòng → cắt dòng → VietOCR (VGG19 → Transformer) → chuỗi text. Pipeline paddleocr: ảnh → DBNet phát hiện → phân loại góc → SVTR_LCNet nhận dạng → chuỗi text. Cả hai gộp về danh sách dòng, đưa vào hàm bóc tách trường và ghi Invoice/InvoiceItem để đối chiếu PO.

Kết quả huấn luyện PaddleOCR (số thật từ train.log). Với nhận dạng (rec), loss giảm nhanh về gần 0 quanh bước ~2 500 và độ chính xác huấn luyện tiến sát 1,0; trên tập val, accuracy toàn chuỗi đạt 0,9964 (99,64%) và norm_edit_dis 0,9992 tại epoch 41. Với phát hiện (det), hmean tốt nhất trên val chỉ đạt 0,2298 (precision 0,228; recall 0,232) - cho thấy phát hiện bố cục hóa đơn học trên dữ liệu synthetic còn yếu; khuyến nghị bổ sung 50–150 hóa đơn thật gán bằng PPOCRLabel, hoặc kết hợp detector pretrain với recognizer đã fine-tune.

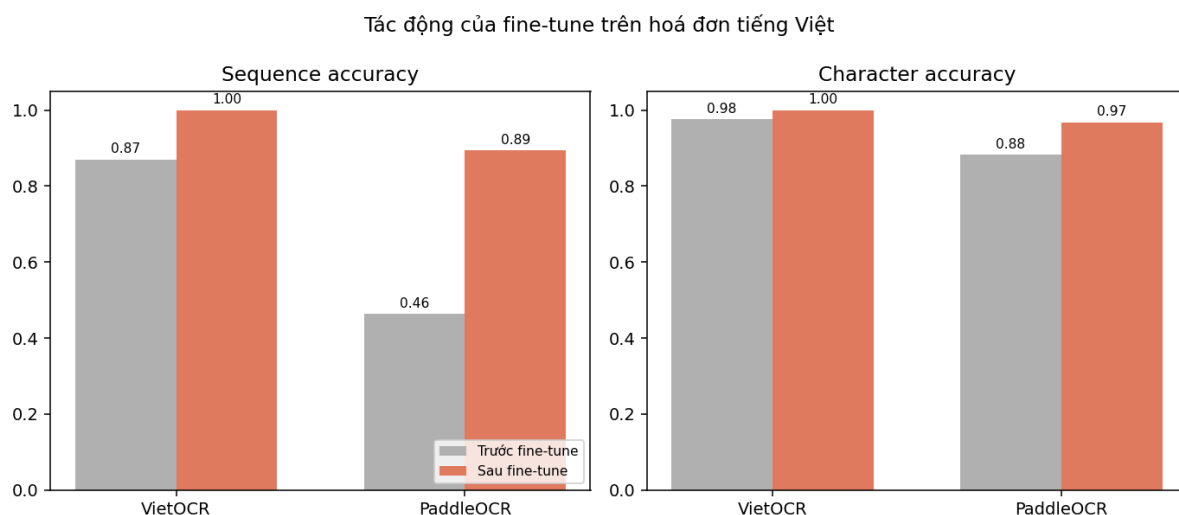


Hình 4.18. Đường cong huấn luyện PaddleOCR (rec): loss và accuracy theo bước

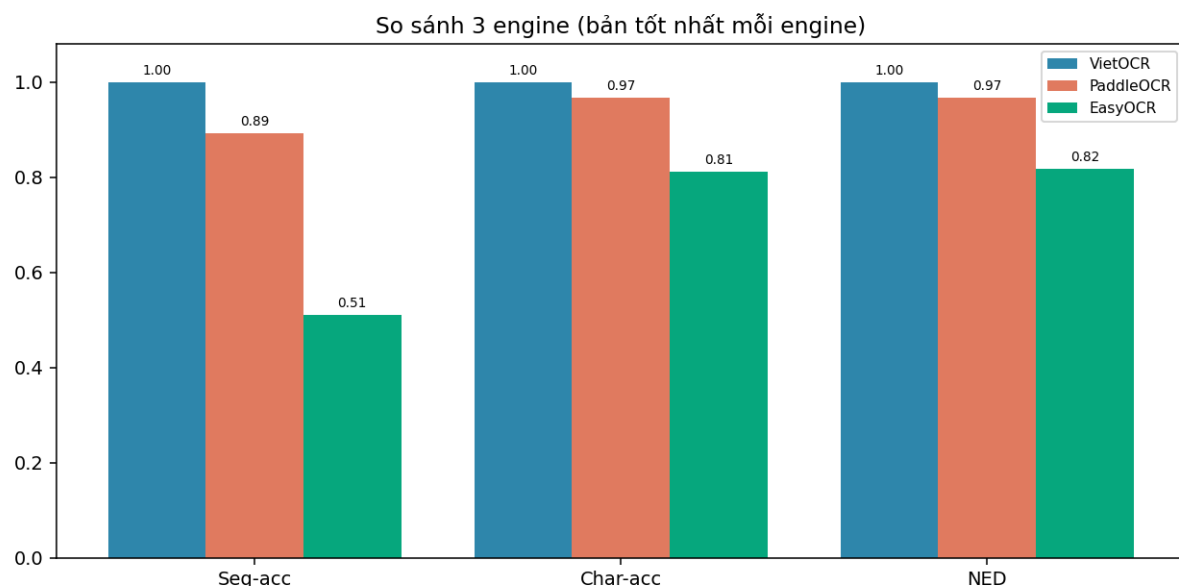
d) So sánh và đánh giá ba engine

Ba engine (EasyOCR, VietOCR, PaddleOCR) được đánh giá trên cùng tập val 300 mẫu với các chỉ số: Seq-acc (đọc đúng toàn chuỗi), Char-acc ($1 - \text{CER}$), NED, $\text{CER} < 0,1$ và độ trễ (ms/ảnh, đo trên CPU):

Cấu hình	Seq-acc	Char-acc	NED	CER<0,1	ms/ảnh
0,870	0,976	0,977	0,903	ViếtOCR gốc (trước)	1,000
1,000	1,000	1,000	0,463	ViếtOCR fine-tune (sau)	0,882
0,882	0,513	PaddleOCR gốc (trước)	0,893	0,967	0,968
0,913	65	PaddleOCR fine-tune (sau)	0,811	0,818	0,613
156	EasyOCR (baseline)	0,510	0,974	0,933	Oracle (best-of-3, chặn trên)
Ensemble (bỏ phiếu)	0,920	0,974	0,974	0,933	Oracle (best-of-3, chặn trên)
1,000	-	-	-	-	-

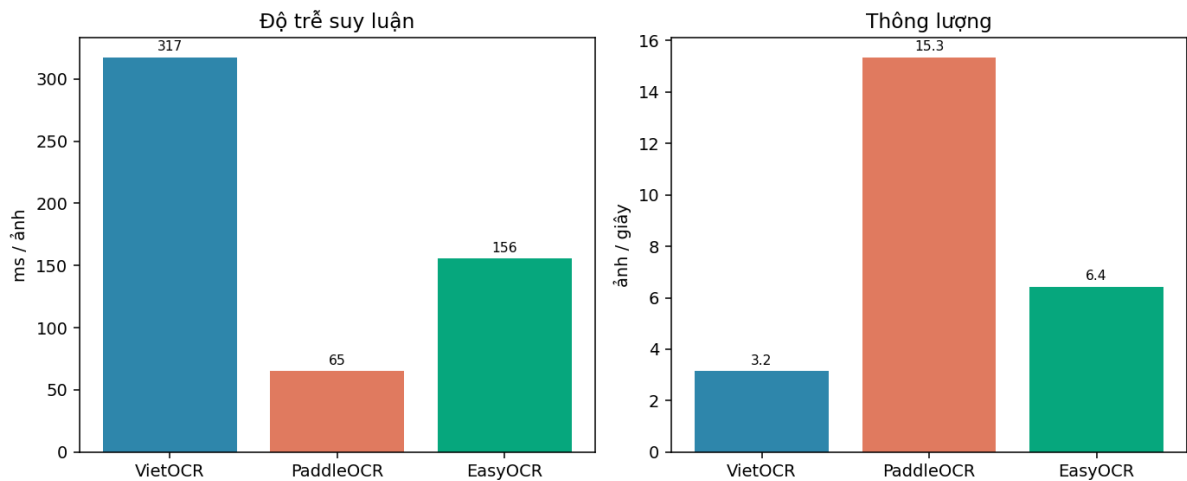


Hình 4.19. Tác động của fine-tune (Sequence và Character accuracy)

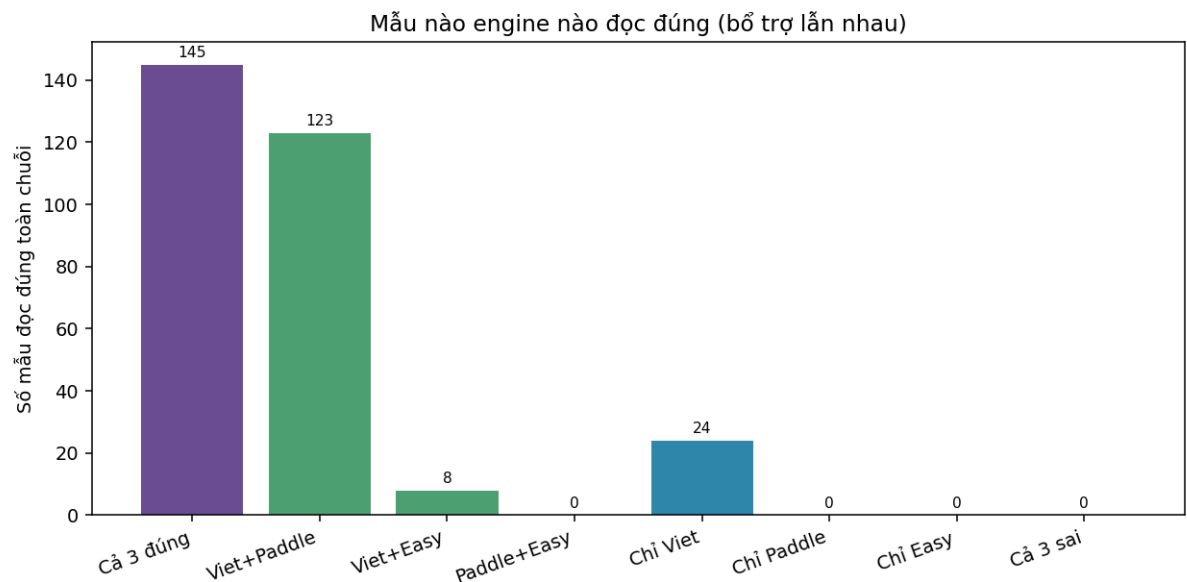


Hình 4.20. So sánh ba engine theo Seq-acc / Char-acc / NED (tập val 300 mẫu)

Nhận xét. (1) Fine-tune có tác động lớn: Seq-acc VietOCR tăng 0,870 $\rightarrow$ 1,000; PaddleOCR tăng 0,463 $\rightarrow$ 0,893 (PaddleOCR gốc pretrain tiếng Trung nên yếu trên tiếng Việt, hưởng lợi nhiều nhất). (2) Về độ chính xác nhận dạng, VietOCR fine-tune đứng đầu (đọc đúng cả 300/300 mẫu), trên PaddleOCR (0,893) và EasyOCR (0,510). (3) Về tốc độ, PaddleOCR nhanh nhất (65 ms/ảnh, nhanh hơn VietOCR $\sim$ 4,9 lần) và hợp nhất det+rec.



Hình 4.21. Độ trễ suy luận (ms/ảnh) và thông lượng (ảnh/giây) trên CPU



Hình 4.22. Tính bổ trợ giữa các engine (mẫu nào engine nào đọc đúng)

Tính bổ trợ và kết hợp. Trong 300 mẫu, số mẫu “cả ba cùng sai” = 0; VietOCR đọc đúng 300, PaddleOCR 268, EasyOCR 153; “chỉ VietOCR đúng” = 24, còn “chỉ PaddleOCR/chỉ EasyOCR đúng” = 0. Do đó bỏ phiếu ba engine chỉ đạt 0,920 - thấp hơn VietOCR đơn lẻ (1,000) vì bị kéo xuống bởi engine yếu hơn; cận trên Oracle = 1,000. Với bài toán này, dùng một engine mạnh hợp lý hơn là bỏ phiếu.

e) Lựa chọn mô hình

Vì sao chọn PaddleOCR thay vì kết hợp EasyOCR + VietOCR. Phương án EasyOCR + VietOCR có recognizer VietOCR rất chính xác nhưng vướng ba nhược điểm: tăng phát

hiện CRAFT của EasyOCR không được fine-tune (thành nút cổ chai), phải nạp hai bộ công cụ/hai mô hình (nặng), và chậm nhất (317 ms/ảnh). PaddleOCR là một bộ công cụ hợp nhất, fine-tune được cả DBNet lẫn recognizer trên cùng dataset tự gán nhãn, nhẹ và nhanh nhất (65 ms/ảnh). Vì vậy PaddleOCR được chọn làm engine tự chủ mặc định (đánh đổi một phần độ chính xác nhận dạng lấy pipeline gọn – nhanh – tinh chỉnh trọn gói); VietOCR được giữ làm lựa chọn khi cần độ chính xác nhận dạng cao nhất; Gemini dự phòng cho bố cục lạ và PDF text-layer cho hóa đơn điện tử.

f) Thuật toán bóc tách cấu trúc hóa đơn kháng nghiêng (deskew)

Đặt vấn đề. Tầng OCR chỉ trả về danh sách ô văn bản kèm hộp bao (toạ độ), chưa phải dữ liệu có cấu trúc. Để dựng lại các trường (số hóa đơn, ngày, mã số thuế, các dòng tiền) và bảng chi tiết hàng hóa, hệ thống phải gom các ô về đúng hàng và cột dựa trên toạ độ. Ảnh hóa đơn chụp bằng điện thoại thường bị nghiêng vài độ; khi đó độ lệch tung độ giữa mép trái và mép phải của bảng (xấp xỉ $\tan\theta$ nhân với bề rộng bảng) vượt quá chiều cao một dòng, khiến thao tác gom theo tung độ y gộp nhầm hai dòng sản phẩm liền kề vào cùng một hàng. Đề tài đề xuất thuật toán hai pha khắc phục bằng hình học – thống kê, chạy sau tầng OCR và không cần huấn luyện.

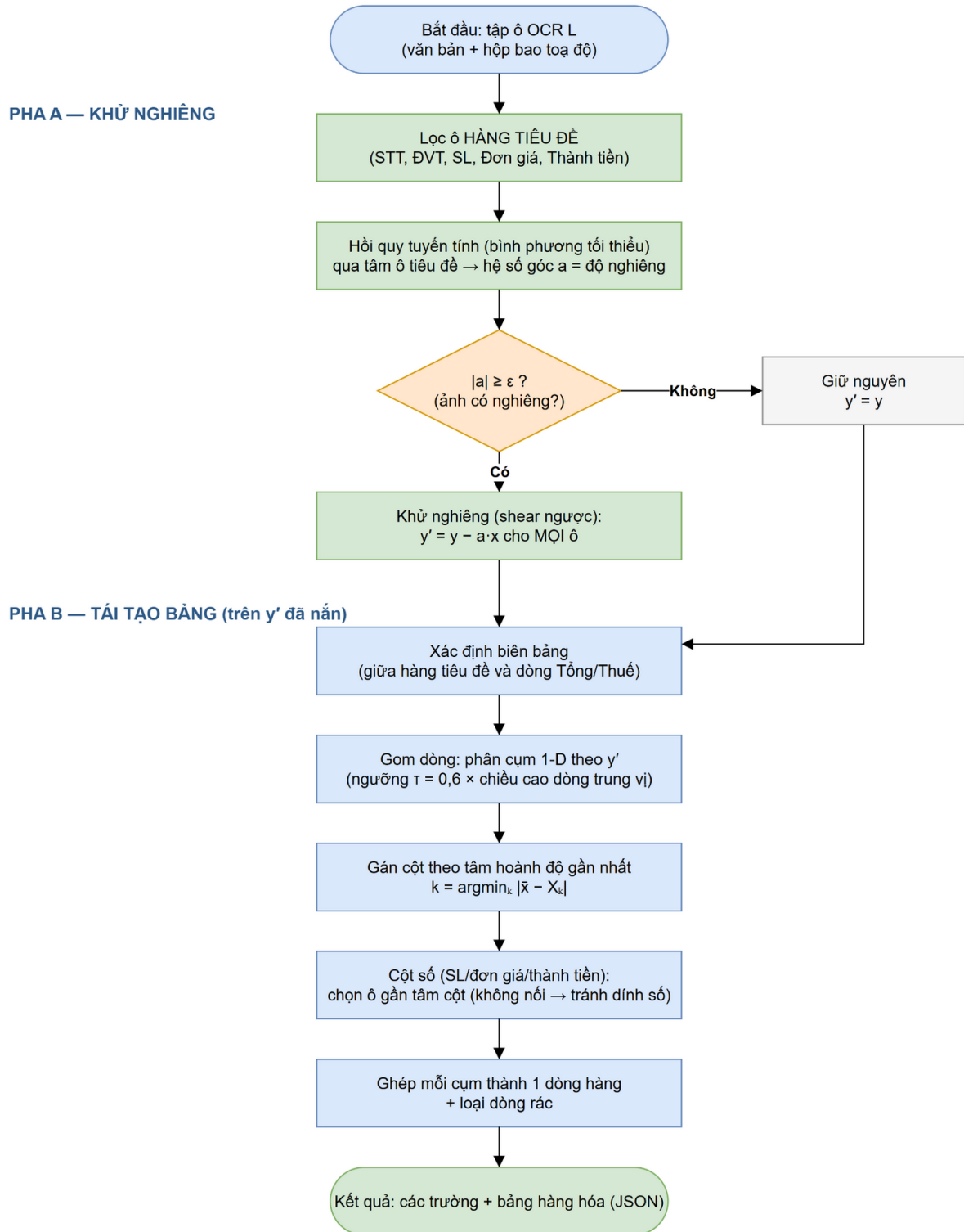
Pha A – Ước lượng và khử nghiêng. Các ô thuộc hàng tiêu đề bảng (STT, Tên hàng, ĐVT, SL, Đơn giá, Thành tiền) trong thực tế nằm trên cùng một phương ngang. Thuật toán lọc các ô này rồi khớp một đường thẳng qua tâm của chúng bằng phương pháp bình phương tối thiểu; hệ số góc a của đường thẳng chính là độ nghiêng của trang (góc $\theta = \arctan a$). Sau đó áp phép biến đổi trượt (shear) ngược trên trục tung cho mọi ô để nắn thẳng toạ độ:

$$a = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2 ; y' = y - a \cdot x_i$$

Sau bước này, mọi ô thuộc cùng một dòng vật lý có tung độ y' xấp xỉ bằng nhau (bảng được “làm thẳng” về mặt toạ độ mà không phải xử lý lại ảnh). Nếu $|a|$ nhỏ hơn một ngưỡng ε (ảnh vốn thẳng) thì bỏ qua bước trừ, bảo đảm tính bất biến với ảnh không nghiêng.

Pha B – Tái tạo bảng theo cụm toạ độ. Trên hệ toạ độ đã nắn thẳng, thuật toán lần lượt: (1) xác định biên bảng nằm giữa hàng tiêu đề và dòng “Tổng/Thuế”; (2) gom dòng bảng phân cụm một chiều theo y' với ngưỡng bằng 0,6 lần chiều cao dòng trung vị; (3) gán mỗi ô vào cột có tâm hoành độ gần nhất, riêng các cột số (SL, đơn giá, thành tiền) chọn ô gần tâm cột nhất thay vì nối chuỗi nhằm tránh dính các con số; (4) ghép mỗi cụm thành một dòng hàng và loại bỏ các dòng rác (không có mô tả chữ hoặc không có số

tiền).



Hình 4.23. Sơ đồ khối thuật toán bóc tách hóa đơn kháng nghiêng (deskew).

Mã giả. Toàn bộ thuật toán được tóm tắt như sau:

Thuật toán `ExtractInvoiceTable(L) # L`: tập ô OCR (văn bản + hộp bao)

```

H <- các ô có từ khóa cột tiêu đề
a <- HồiQuyHệSốGóc({ (xc, yc) : c thuộc H }) # bình phương tối thiểu
nếu |a| >= eps: y'(c) <- yc(c) - a * xc(c) với mọi c thuộc L # khử nghiêng
ngược lại: y'(c) <- yc(c)
Xk <- tâm hoành độ các cột (từ H)
tau <- 0.6 * trung_vị(chiều cao ô)
B <- { c : y'_tiêuđề < y'(c) < y'_dừng } # vùng bảng
R <- PhânCụm1D(B theo y', ngưỡng tau) # gom dòng
với mỗi cụm r trong R:
cột(c) <- argmin_k |xc(c) - Xk| với mọi c trong r
dòng <- { mô tả: nối ô chữ ; số: ô gần tâm mỗi cột số }
nếu hợp lệ: xuất dòng
trả về R

```

Độ phức tạp và kết quả. Chi phí gồm hồi quy $O(|H|)$, sắp xếp để gom cụm $O(n \log n)$ và gán cột $O(n \cdot K)$ với K là số cột (khoảng 6), do đó tổng độ phức tạp là $O(n \log n)$ và bộ nhớ $O(n)$, với n là số ô OCR. Trên hóa đơn thử nghiệm bị nghiêng, thuật toán tách đúng toàn bộ 7/7 dòng hàng cùng các trường tổng tiền, trong khi cách gom theo tung độ thô chỉ nhận được 3 hàng do bị dòn. Ưu điểm là thuần hình học – thống kê, không cần mạng nơ-ron, và tự bất biến khi ảnh đã thẳng.

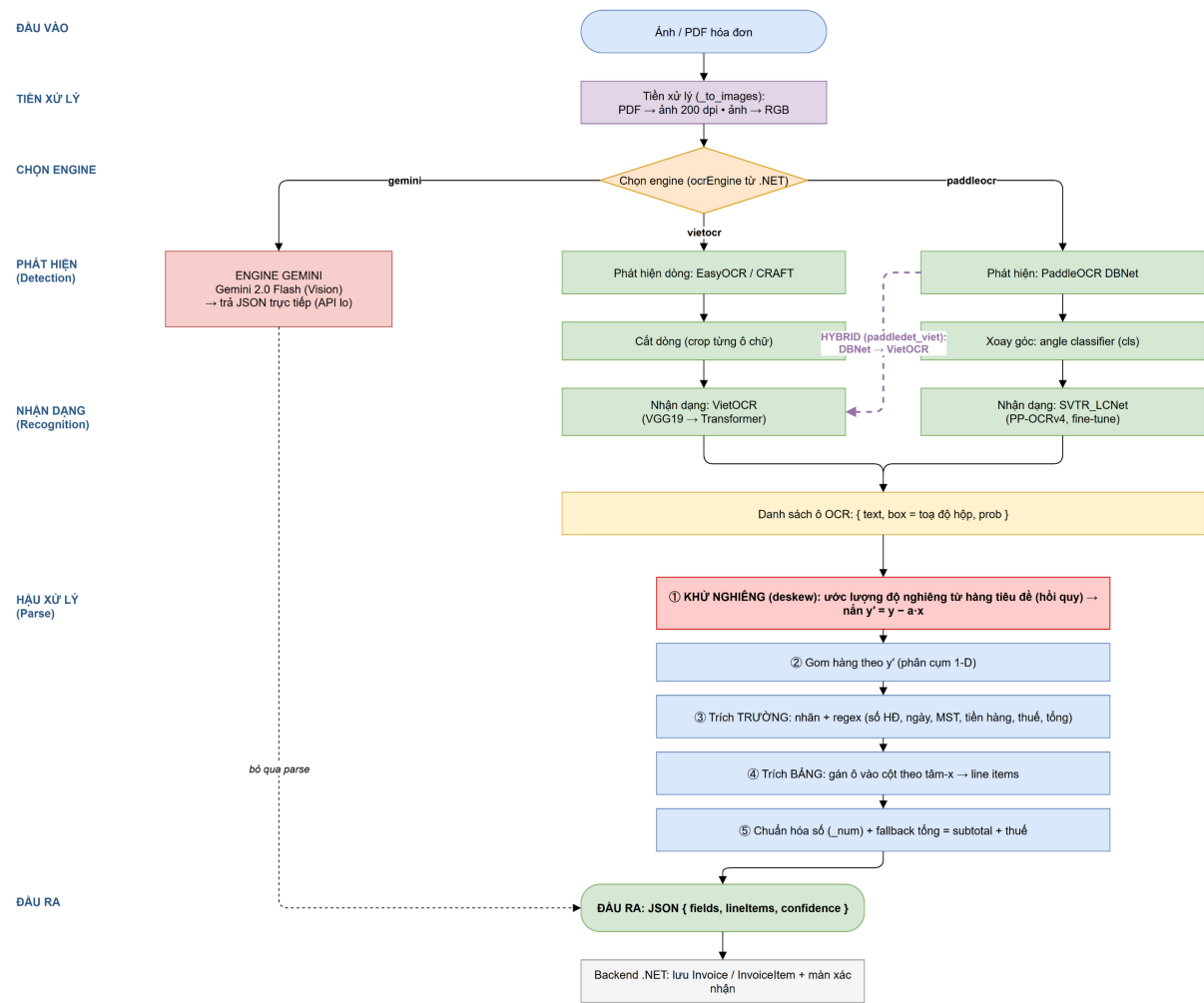
g) Tổng hợp luồng xử lý AI end-to-end

Tổng quan. Hình 4.24 tổng hợp toàn bộ luồng xử lý AI từ ảnh đầu vào đến JSON đầu ra, làm rõ dữ liệu đi qua những lớp nào, được xử lý và bóc tách ra sao, và vị trí của bước khử nghiêng.

Các lớp xử lý. (1) Đầu vào là ảnh hoặc tệp PDF hóa đơn. (2) Tiền xử lý (_to_images) chuyển PDF thành ảnh 200 dpi hoặc chuẩn hóa ảnh về hệ màu RGB. (3) Tùy engine người dùng chọn (trường ocrEngine truyền từ backend .NET), luồng rẽ nhánh: engine gemini gửi thẳng ảnh cho Gemini 2.0 Flash và nhận về JSON có cấu trúc, bỏ qua các bước phân tích phía sau; hai engine tự chủ (vietocr, paddleocr) đi tiếp qua lớp Phát hiện rồi Nhận dạng. (4) Lớp Phát hiện: pipeline vietocr dùng EasyOCR/CRAFT để tìm và cắt từng dòng, pipeline paddleocr dùng DBNet kèm bộ phân loại góc (cls). (5) Lớp Nhận dạng: VietOCR (VGG19 + Transformer) hoặc SVTR_LCNet (PP-OCrv4, đã fine-tune) đọc chữ, tạo ra danh sách ô OCR gồm văn bản, hộp bao (toạ độ) và độ tin cậy.

Bóc tách và vị trí khử nghiêng. (6) Lớp Hậu xử lý (parse) biến danh sách ô rời rạc

thành dữ liệu có cấu trúc qua năm bước: ① Khử nghiêng (deskew) — ước lượng độ nghiêng từ hàng tiêu đề và nắn tọa độ theo $y' = y - a \cdot x$; ② Gom hàng theo y' đã nắn; ③ Trích các trường bằng nhãn và biểu thức chính quy (số hóa đơn, ngày, MST, tiền hàng, thuế, tổng); ④ Trích bảng chi tiết bằng cách gán ô vào cột theo tâm hoành độ; ⑤ Chuẩn hóa con số và bù giá trị tổng. Đáng chú ý, khử nghiêng là bước ĐẦU TIÊN của lớp hậu xử lý, chạy ngay sau khi có danh sách ô OCR và trước mọi thao tác gom hàng, gán cột — vì các bước sau đều dựa trên tọa độ đã được nắn thẳng. (7) Đầu ra là JSON gồm fields, lineItems và confidence, được backend .NET lưu vào Invoice/InvoiceItem và hiển thị ở màn hình xác nhận.



Hình 4.24. Luồng xử lý AI end-to-end của dịch vụ OCR: các lớp xử lý và vị trí bước khử nghiêng.

h) Biến thể hybrid: Paddle DBNet (phát hiện) + VietOCR (nhận dạng)

Ý tưởng. Từ kết quả ở mục d), VietOCR cho độ chính xác nhận dạng cao nhất trong ba engine, trong khi bộ phát hiện DBNet của PaddleOCR (đã fine-tune) tốt hơn CRAFT của

EasyOCR — vốn không được fine-tune và là nút cổ chai của pipeline vietocr. Vì vậy đề tài thử nghiệm thêm một biến thể hybrid (engine paddledet_viet) ghép hai điểm mạnh: dùng DBNet để phát hiện dòng, rồi đưa từng vùng vào VietOCR để nhận dạng.

Cơ chế ghép. Do DBNet trả về đa giác bốn điểm (có thể xoay, nghiêng), lớp ghép thực hiện phép biến đổi phối cảnh để đưa mỗi vùng về ảnh dòng nằm ngang (tương tự hàm `get_rotate_crop_image` của PaddleOCR) trước khi VietOCR đọc. Kết quả trả về cùng định dạng danh sách ô (văn bản, hộp bao, độ tin cậy) nên tái sử dụng nguyên vẹn bộ hậu xử lý ở mục f và g (khử nghiêng, trích trường và bảng).

Phân tích ablation. So sánh paddleocr (DBNet + Paddle SVTR) với paddledet_viet (DBNet + VietOCR) là một ablation giữ nguyên detector, chỉ thay recognizer, do đó cô lập đúng ảnh hưởng của tầng nhận dạng. Cần lưu ý: phép đánh giá nhận dạng ở mục d) chạy trên các dòng đã cắt sẵn (ground-truth) nên không phụ thuộc detector; ở mức này biến thể hybrid dùng recognizer VietOCR nên có độ chính xác nhận dạng trùng với cột VietOCR (cao nhất). Khác biệt thực sự của hybrid nằm ở tầng phát hiện: DBNet fine-tune cắt dòng tốt và ổn định hơn CRAFT, nên khi chạy end-to-end trên hóa đơn cả trang, hybrid được kỳ vọng vượt pipeline vietocr (CRAFT + VietOCR) nhờ detector mạnh hơn, đồng thời giữ chất lượng đọc của VietOCR. Việc đối chiếu end-to-end trên từng hóa đơn được thực hiện bằng công cụ so sánh ba pipeline (mục g).

Đánh đổi và lựa chọn. Bù lại các ưu điểm trên, hybrid phải nạp đồng thời hai framework (PaddlePaddle và PyTorch) nên tốn bộ nhớ hơn; VietOCR nhận dạng theo từng dòng thay vì theo lô như SVTR nên chậm hơn; và triển khai một dịch vụ dùng hai framework cũng phức tạp hơn PaddleOCR hợp nhất. Vì vậy hệ thống chọn PaddleOCR làm engine mặc định cho vận hành, còn biến thể hybrid được cung cấp như một tùy chọn (paddledet_viet) phục vụ đối chiếu và các tình huống ưu tiên độ chính xác nhận dạng hơn tốc độ.

4.3. Nghiệp vụ Quản lý sự cố và SLA

4.3.1. Đặt vấn đề và phạm vi

Trong vận hành tòa nhà, sự cố (điện, nước, HVAC, thang máy, PCCC...) phát sinh liên tục và cần được tiếp nhận, phân công, xử lý đúng cam kết thời gian (SLA). Phân hệ Quản lý sự cố (CM) số hóa toàn bộ quy trình từ báo sự cố đến đóng và đánh giá, đồng thời theo dõi tuân thủ SLA và leo thang khi trễ hạn.

- Tiếp nhận sự cố từ cư dân/kỹ thuật viên (ticket) theo loại và mức ưu tiên.

- Phân công kỹ thuật viên, cập nhật trạng thái, đính kèm ảnh trước/sau.
- Cấu hình SLA theo mức ưu tiên (thời gian phản hồi, thời gian giải quyết).
- Theo dõi hạn SLA, tự động leo thang khi sắp/đã quá hạn; SLA Dashboard.
- Đánh giá mức độ hài lòng của người báo sau khi đóng sự cố.

4.3.2. Môi liên hệ với Base và các phân hệ khác

Nhiệm vụ Sự cố & SLA khép kín chuỗi nghiệp vụ của phân hệ Quản lý tài sản - kế thừa dữ liệu tài sản, kho do chính tác giả xây dựng và sử dụng các dịch vụ nền tảng chung của nhóm:

- **Với Tài sản:** ticket gắn tài sản liên quan để truy vết thiết bị hỏng.
- **Với Kho/Mua sắm:** khi xử lý cần vật tư có thể xuất kho hoặc tạo đề xuất mua (PR).
- **Với Base:** phân quyền RBAC theo vai trò (cư dân/KTV/điều phối/quản lý); gửi thông báo khi phân công, sắp/đã quá hạn SLA.

4.3.3. Phân tích yêu cầu chức năng

a) Vòng đời và trạng thái sự cố

Trạng thái ticket diễn tiến: Mới tiếp nhận → Chờ phân công → Đã phân công → Đang xử lý → Chờ vật tư (nếu thiếu) → Đã xử lý → Đã đóng. Mỗi lần đổi trạng thái được ghi vào lịch sử (TicketStatusHistory).

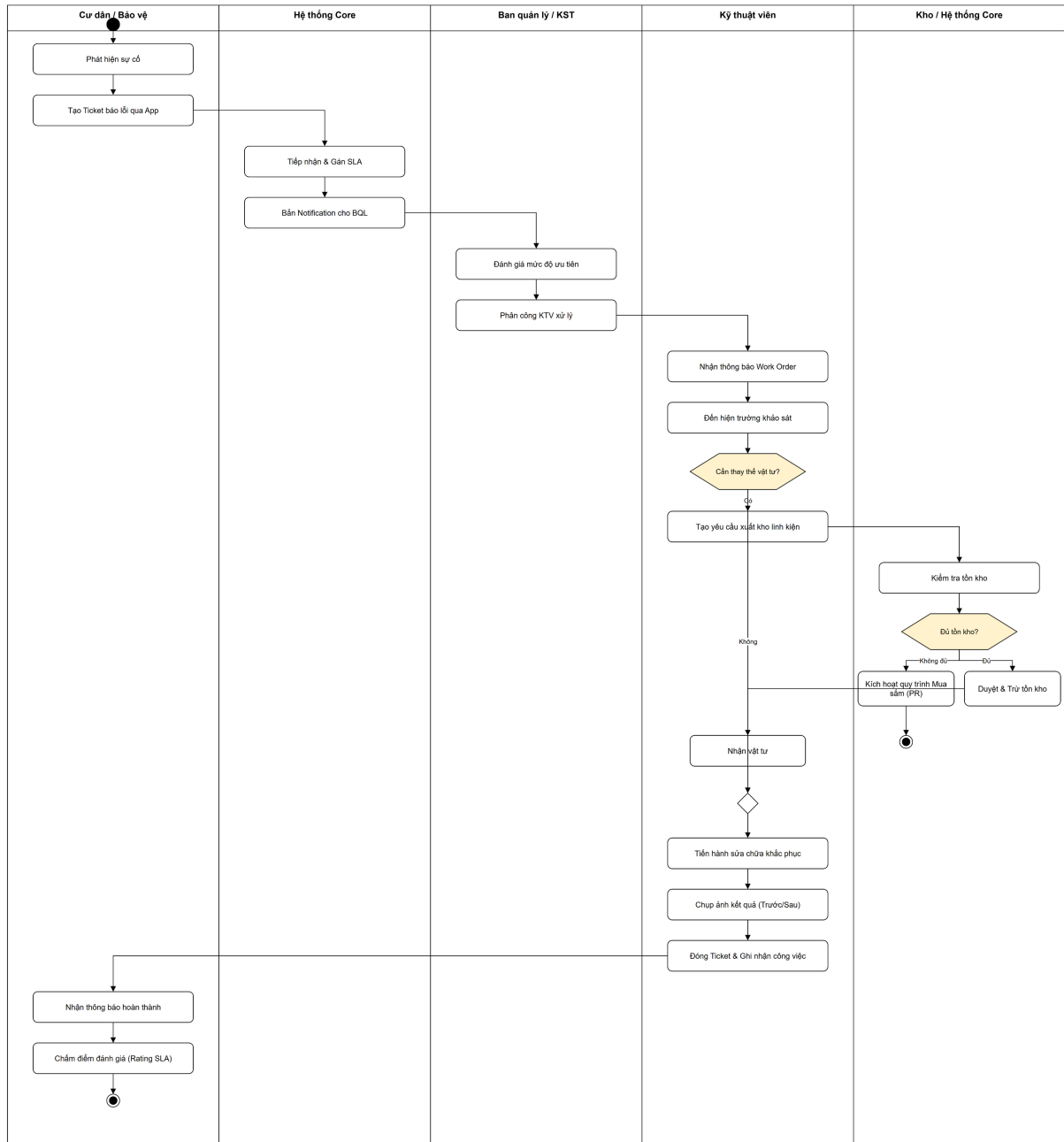
b) Loại sự cố, nguồn báo và mức ưu tiên

Loại sự cố: Điện, Nước, HVAC, Thang máy, PCCC, Khác. Mỗi mức ưu tiên gắn với một cam kết SLA (thời gian phản hồi và giải quyết). Khi tạo ticket có thể để trống SLA để hệ thống tự áp theo mức ưu tiên.

c) Tiếp nhận, phân công và xử lý

Quy trình: Báo sự cố (tạo ticket) → tiếp nhận & phân công KTV (TicketAssignment) → KTV xử lý, cập nhật trạng thái, xuất vật tư/tạo PR nếu thiếu, chụp ảnh trước/sau (TicketAttachment) → đóng sự cố khi khắc phục xong. Luồng xử lý sự cố và giám sát SLA được minh họa ở Hình 4.25.

Hình 4.15 — Lưu đồ nghiệp vụ: Xử lý sự cố và giám sát SLA



Hình 4.25. Lưu đồ nghiệp vụ: Xử lý sự cố và giám sát SLA

d) SLA và leo thang

Cấu hình SLA (SlaConfig) định nghĩa thời gian phản hồi và giải quyết theo mức ưu tiên. Hệ thống theo dõi hạn SLA suốt quá trình; khi sắp/đã quá hạn sẽ leo thang và ghi nhật ký (SlaEscalationLog). Màn hình SLA Dashboard tổng hợp tỉ lệ tuân thủ, số ticket quá hạn.

e) Đánh giá mức độ hài lòng

Sau khi đóng, người báo (cư dân) chấm sao mức độ hài lòng (TicketRating), phục vụ đánh giá chất lượng dịch vụ.

4.3.4. Yêu cầu phi chức năng

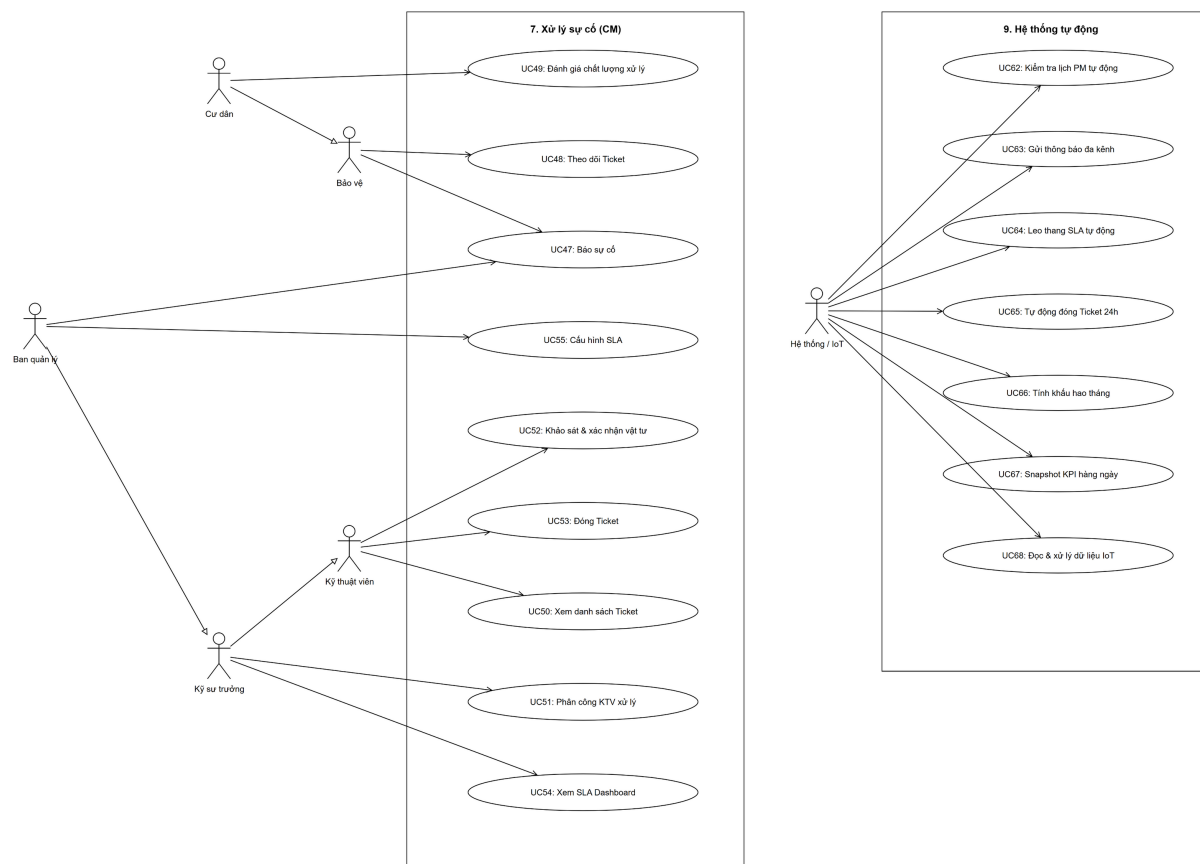
- Tính kịp thời: theo dõi và cảnh báo SLA theo thời gian thực.
- Truy vết đầy đủ: lịch sử trạng thái, phân công, đính kèm và leo thang.

4.3.5. Thiết kế phân hệ

a) Danh sách Use Case chính

Các use case chính của phân hệ được thể hiện trên biểu đồ use case ở Hình 4.26 và liệt kê trong bảng dưới đây.

Hình 4.16 — Biểu đồ Use Case: Quản lý sự cố và SLA



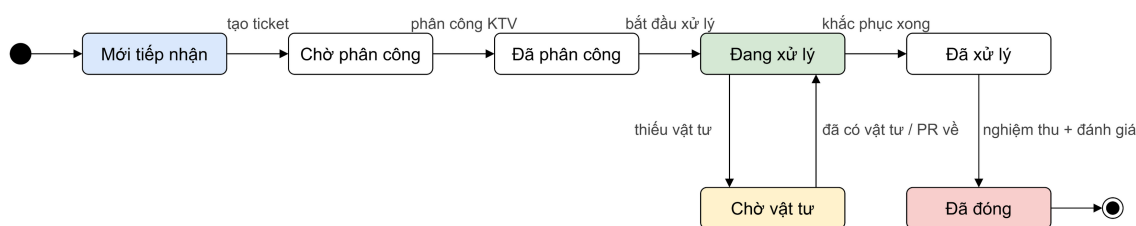
Hình 4.26. Biểu đồ Use Case – Quản lý sự cố và SLA

Mã UC	Use case	Tác nhân
Báo sự cố (tạo ticket)	Cư dân/KTV UC-CM-02	Tiếp nhận & phân công KTV UC-CM-01
Điều phối UC-CM-03	Xử lý & cập nhật trạng thái, đính kèm ảnh	KTV UC-CM-04
Cấu hình & theo dõi SLA, leo thang	Quản lý/Hệ thống UC-CM-05	Đánh giá mức độ hài lòng
Cư dân		

b) Sơ đồ trạng thái (state machine) của ticket

Vòng đời trạng thái của một ticket sự cố được thể hiện ở Hình 4.27, trong đó trạng thái ”Đang xử lý” có thể chuyển sang ”Chờ vật tư” khi thiếu vật tư và quay lại khi đã có.

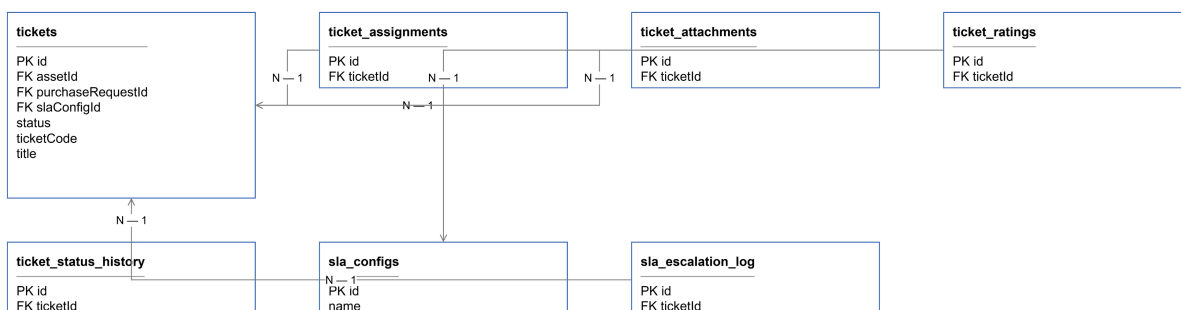
Hình 4.17 — Sơ đồ trạng thái vòng đời ticket sự cố



Mỗi lần đổi trạng thái ghi TicketStatusHistory; quá hạn SLA ghi SlaEscalationLog và leo thang

Hình 4.27. Sơ đồ trạng thái vòng đời ticket sự cố

ERD schema asset — phân hệ Quản lý sự cố & SLA



Hình 4.28. Sơ đồ thực thể – liên kết (ERD) phân hệ Quản lý sự cố & SLA

c) Thiết kế cơ sở dữ liệu (thực thể chính)

Thực thể	Ý nghĩa
Sự cố: tiêu đề, loại, ưu tiên, nguồn báo, tài sản liên quan, trạng thái TicketAssignment	Ticket Phân công kỹ thuật viên xử lý TicketStatusHistory
Lịch sử thay đổi trạng thái TicketAttachment	Ảnh/tài liệu đính kèm (trước/sau) TicketRating
Đánh giá mức độ hài lòng SlaConfig	Cấu hình SLA theo mức ưu tiên SlaEscalationLog
Nhật ký leo thang khi trễ hạn	

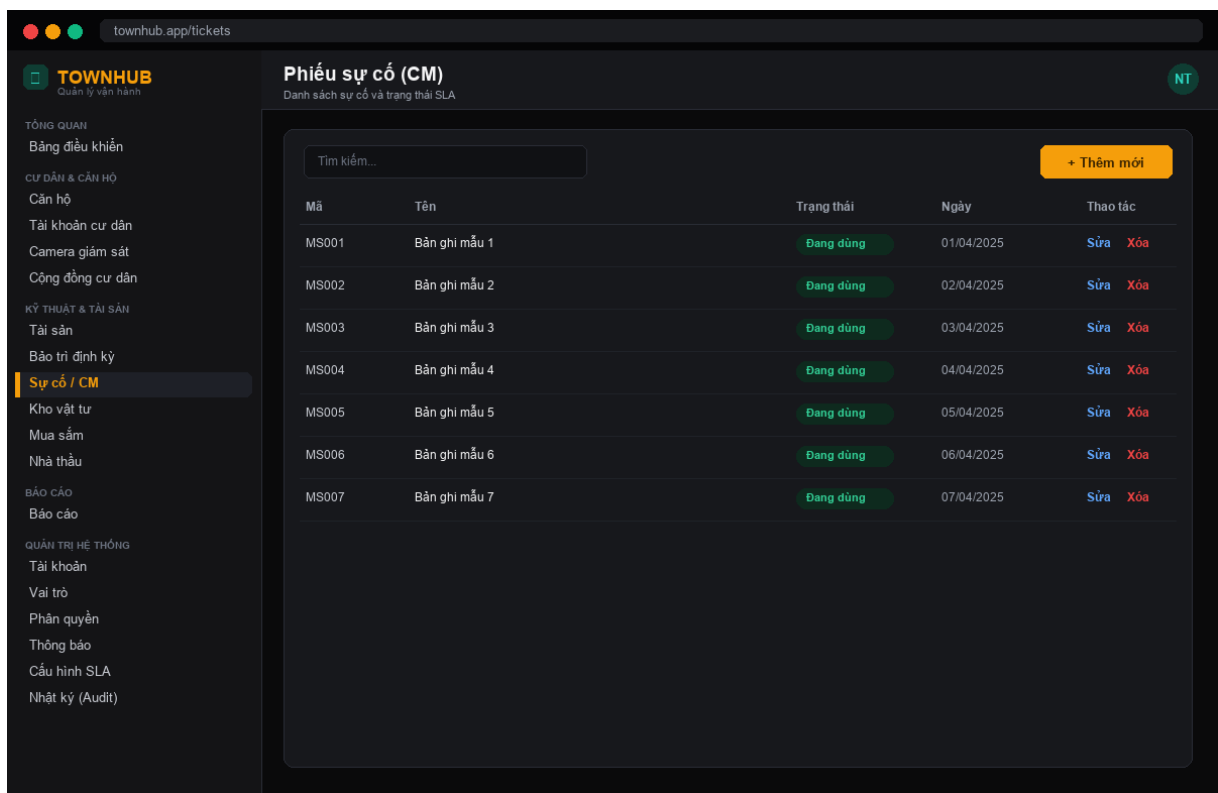
d) Thiết kế API

Phương thức & Endpoint	Chức năng
Tạo ticket sự cố GET /api/asset/ticket/get-all, get/{id}	POST /api/asset/ticket/create Danh sách & chi tiết POST /api/asset/ticket/assign
Phân công kỹ thuật viên POST /api/asset/ticket/change-status	Đổi trạng thái xử lý POST /api/asset/ticket/add-attachment
Đính kèm ảnh trước/sau POST /api/asset/ticket/rate	Đánh giá hài lòng GET .../get-status-history, get-escalation-logs/{id}
Lịch sử trạng thái & leo thang /api/asset/sla-config/*	CRUD cấu hình SLA

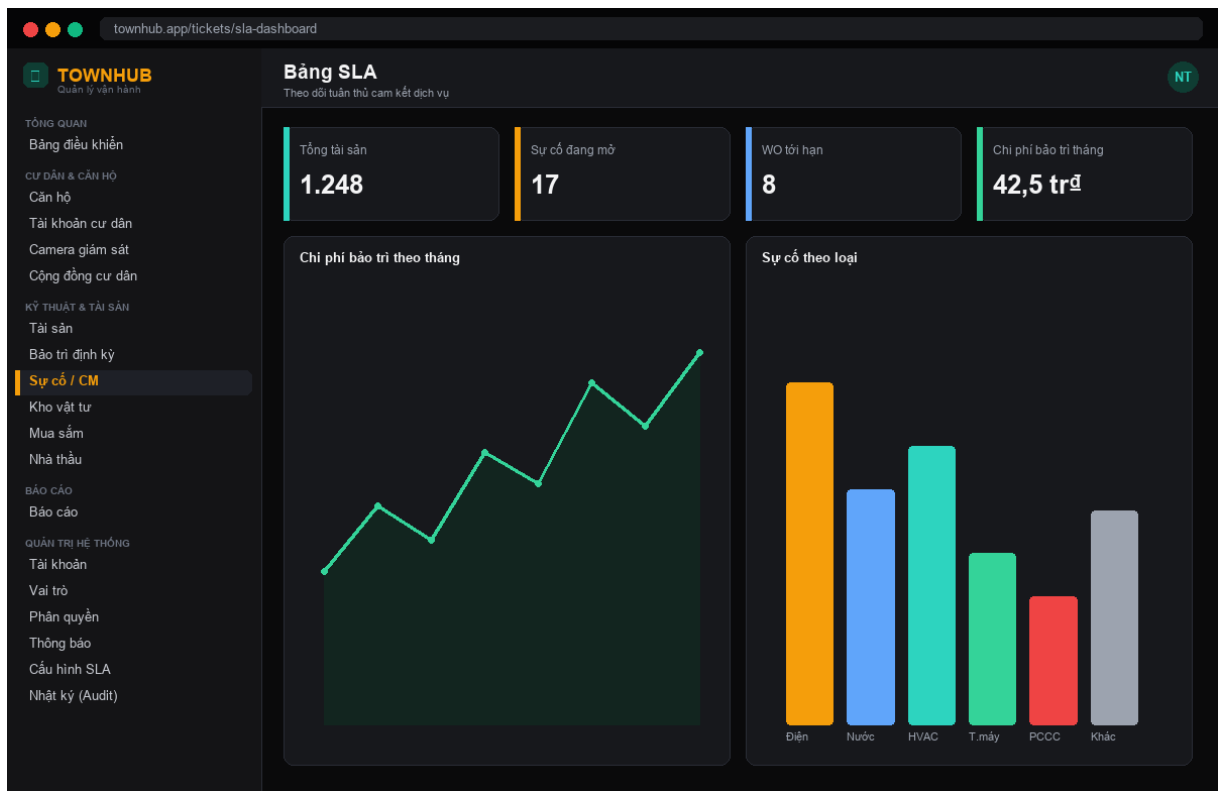
4.3.6. Thiết kế giao diện

Route	Chức năng
Danh sách & chi tiết sự cố /tickets/new	/tickets, /tickets/[id] Báo sự cố mới /tickets/sla-dashboard
Bảng theo dõi SLA /my-tickets/[id], /settings/sla	Sự cố của tôi & cấu hình SLA

Một số màn hình tiêu biểu của phân hệ:



Hình 4.29. Giao diện Danh sách phiếu sự cố (CM)



Hình 4.30. Giao diện Bảng theo dõi SLA

4.4. Kết luận chương

Chương 4 đã phân tích và thiết kế hoàn chỉnh phân hệ Quản lý tài sản do tác giả (Nguyễn Xuân Thuồng) đảm nhiệm, gồm ba nghiệp vụ: Tài sản & bảo trì (vòng đời tài sản, khấu hao, mã QR, điều chuyển, bảo trì phòng ngừa, thanh lý, tích hợp kế toán); Kho – Mua sắm – Nhà cung cấp (tồn kho, quy trình PR→PO→hóa đơn, OCR hóa đơn, đánh giá NCC); và Quản lý sự cố & SLA (quy trình xử lý khép kín, giám sát SLA và leo thang tự động). Ba nghiệp vụ liên thông chặt chẽ với nhau, kế thừa dữ liệu và dịch vụ nền tảng từ module Base của nhóm - minh chứng cho cách tổ chức phân chia module độc lập nhưng liên thông của đồ án. Kết quả triển khai và kiểm thử được trình bày ở Chương 5.

Chương 5. Triển khai, kiểm thử và đánh giá

Chương này trình bày môi trường và kết quả triển khai hệ thống, công tác kiểm thử và đánh giá tổng thể, bao gồm cả các khía cạnh đạo đức và an toàn dữ liệu.

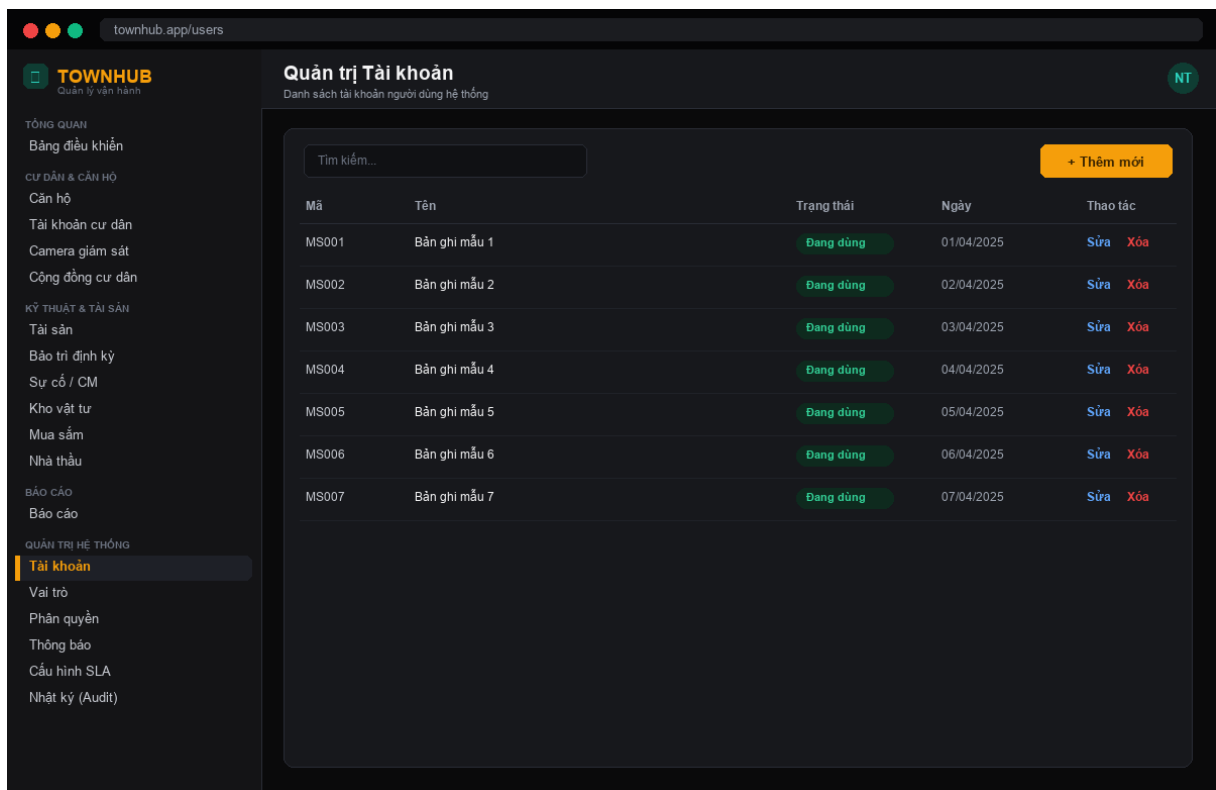
5.1. Môi trường và công nghệ triển khai

Backend được đóng gói bằng Docker theo mô hình multi-stage build (image nền dotnet/aspnet:8.0) và triển khai trên nền tảng Fly.io (buộc HTTPS, tự động khởi động/dừng). Kết nối cơ sở dữ liệu ưu tiên đọc từ cấu hình, sau đó tới biến môi trường DATABASE_URL; hệ thống hỗ trợ Forwarded Headers để hoạt động sau proxy. Trong quá trình phát triển, tài liệu API được sinh tự động bằng Swagger để phục vụ thử nghiệm và kiểm thử. Frontend chạy trên Next.js (npm run dev khi phát triển, build production khi phát hành), cấu hình địa chỉ backend qua biến môi trường.

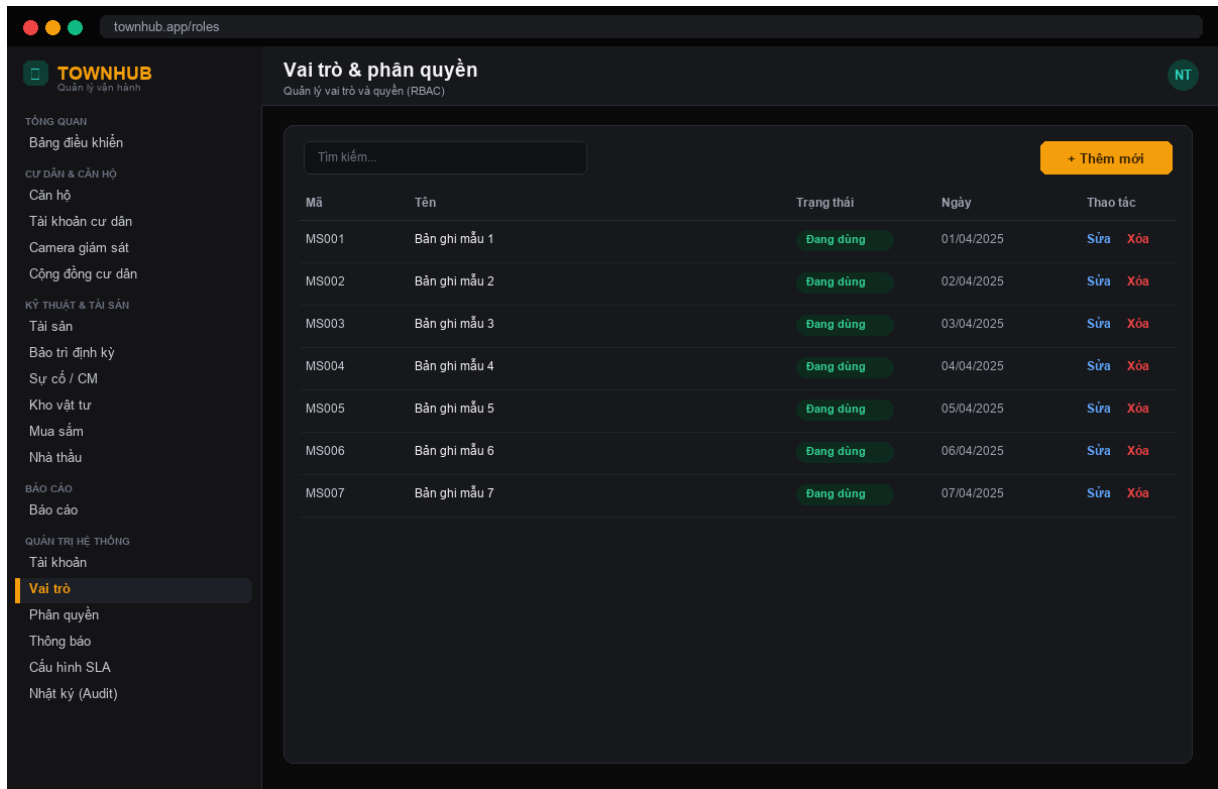
Thành phần	Cách triển khai
Docker multi-stage, Fly.io, cổng 8080, Swagger	Backend (.NET 8)
Frontend (Next.js 16)	npm build/start; biến môi trường NEXT_PUBLIC_API_URL
EF Core migrations (ba schema auth/asset/base)	CSDL PostgreSQL
Redis / Cloudinary / Email	Dịch vụ ngoài qua cấu hình kết nối

5.2. Kết quả triển khai module Base

Với phần chung, nhóm đã phối hợp triển khai module Base hoạt động ổn định với các chức năng nền tảng: đăng nhập đa phương thức (email/mật khẩu, Google, có MFA), quản lý người dùng – vai trò – quyền theo RBAC, quản lý phiên đa thiết bị, gửi thông báo, cấu hình – danh mục dùng chung và dashboard báo cáo. Các phân hệ nghiệp vụ kế thừa trực tiếp những dịch vụ này. Giao diện đăng nhập và dashboard đã trình bày ở Hình 3.5 và 3.6; giao diện quản trị tài khoản và phân quyền được thể hiện ở Hình 5.1 và 7.2.



Hình 5.1. Giao diện Quản trị Tài khoản người dùng



Hình 5.2. Giao diện Vai trò & phân quyền (RBAC)

5.3. Kết quả triển khai phân hệ Quản lý tài sản (phần riêng)

Phân hệ Quản lý tài sản do tác giả đảm nhiệm đã được hiện thực đầy đủ trên cả backend (API) và frontend (giao diện):

- **Tài sản & bảo trì:** danh mục, ghi tăng, mã QR, điều chuyển, khấu hao, thanh lý; lịch bảo trì và Work Order (các route /assets*, /pm/*).
- **Kho – Mua sắm – Nhà cung cấp:** tồn kho & cảnh báo tồn thấp, quy trình PR→PO→hóa đơn, OCR hóa đơn, quản lý NCC/hợp đồng (các route /inventory/*, /procurement/*, /vendors/*).
- **Sự cố & SLA:** tạo và xử lý ticket, phân công, theo dõi SLA, leo thang, đánh giá hài lòng (các route /tickets/*, /my-tickets/*).

Giao diện thực tế của các phân hệ này đã được minh họa chi tiết ở Chương 4 (các Hình 4.6–4.8, 4.12–4.14 và 4.18–4.19). Toàn bộ chức năng đều hoạt động trên dữ liệu thật, có phân trang, tìm kiếm và phân quyền theo vai trò.

5.4. Kiểm thử hệ thống

Hệ thống được kiểm thử theo phương pháp kiểm thử chức năng dựa trên ca kiểm thử, kết hợp thử nghiệm trực tiếp qua Swagger và giao diện; các thành viên trong nhóm kiểm thử chéo phân hệ của nhau để bảo đảm tính khách quan. Bảng dưới đây liệt kê một số ca kiểm thử tiêu biểu trải trên các phân hệ:

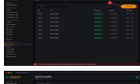
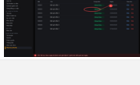
Mã	Chức năng	Dữ liệu vào	Kết quả mong đợi	KQ
Đăng nhập + MFA	email, mật khẩu, OTP	Nhận JWT, vào dashboard	Đạt TC02	Ghi tăng tài sản
mã, tên, danh mục, nguyên giá	Tạo tài sản + QR + chứng từ	Đạt TC03	Tính khấu hao kỳ	tài sản, kỳ
Ghi lịch_su_khau_hao	Đạt TC04	Cảnh báo tồn thấp	vật tư dưới ngưỡng	Xuất hiện ở get-low-stock
Đạt TC05	Duyệt PR → PO	PR hợp lệ	PR approved, tạo PO	Đạt TC06
OCR hóa đơn	ảnh hóa đơn	Trích xuất & tạo Invoice	Đạt TC07	Ticket quá hạn SLA
ticket trễ hạn	Ghi SlaEscalationLog	Đạt		

Toàn bộ ca kiểm thử và nhật ký lỗi được quản lý chi tiết trên tệp Excel dùng chung (mỗi lỗi kèm ảnh chụp, theo dõi qua 3 vòng test), minh họa ở Hình 5.3.

BienBan_KiemThu_TownHub.xlsx — Excel

FileTrang chủChènBổ cụcCông thứcDữ liệuXem

F12fx screenshot_base_05.png

A - STT	B - Ngày tạo	C - Người tạo	E - Link ảnh	F - Miêu tả lỗi	H - Người xử lý	L - Round 1	Q - Round 2	Y - Note
STT	Ngày tạo	Người tạo	Link ảnh	Miêu tả lỗi	Người xử lý	Round 1	Round 2	Note
1	17/03/2025	Nguyễn Xuân Khải		Nút Đăng nhập không vô hiệu khi gọi API, bấm nhiều lần tạo request trùng	Nguyễn Xuân Thường	Không đạt	Đạt	Fix 2 vòng
2	18/03/2025	Lương Thanh Tài		Nhập sai OTP không hiện thông báo lỗi rõ ràng	Nguyễn Xuân Thường	Không đạt	Đạt	
3	18/03/2025	Nguyễn Xuân Thường		Đăng nhập sai nhiều lần không bị giới hạn (brute-force)	Nguyễn Xuân Thường	Không đạt	Không đạt	Fix 3 vòng
4	20/03/2025	Nguyễn Xuân Khải		Tạo tài khoản trùng email không báo lỗi thân thiện	Lương Thanh Tài	Không đạt	Đạt	
5	22/03/2025	Nguyễn Xuân Khải		Bỏ chọn hết quyền vẫn lưu được vai trò rỗng	Nguyễn Xuân Thường	Không đạt	Đạt	
6	26/03/2025	Nguyễn Xuân Thường		Bộ lọc Audit log lịch mới giờ (7h)	Nguyễn Xuân Khải	Không đạt	Đạt	
					</			

Hình 5.3. Nhật ký lỗi & kiểm thử trên Excel (sheet Lỗi Base)

Kết quả kiểm thử cho thấy các chức năng chính hoạt động đúng yêu cầu. Các lỗi phát hiện trong quá trình phát triển đã được ghi nhận và khắc phục qua các vòng test.

5.5. Đánh giá hệ thống

Về mức độ đáp ứng yêu cầu, hệ thống đã hoàn thành các yêu cầu chức năng đặt ra ở Chương 1 cho cả phần Base và phân hệ Quản lý tài sản (phần riêng); kiến trúc module hóa giúp mã nguồn rõ ràng, dễ mở rộng và bảo trì. Về hiệu năng, các danh sách lớn được phân trang và lọc phía máy chủ; việc xử lý OCR bất đồng bộ qua hàng đợi giúp giao diện không bị chặn. Về bảo mật, hệ thống áp dụng JWT, MFA và phân quyền RBAC chi tiết. So với mục tiêu ban đầu, đề tài đạt được một sản phẩm chạy được, có giao diện và API hoàn chỉnh, tích hợp thành phần thông minh (OCR, khâu hao tự động).

5.6. Đánh giá đạo đức, an toàn dữ liệu và bối cảnh xã hội

Hệ thống xử lý dữ liệu cá nhân của cư dân và dữ liệu tài chính – kế toán của tòa nhà, do đó vấn đề đạo đức và an toàn dữ liệu được cân nhắc ngay trong thiết kế: áp dụng nguyên tắc phân quyền tối thiểu (chỉ vai trò phù hợp mới xem được dữ liệu nhạy cảm), lưu vết đầy đủ qua Audit Log, bảo vệ xác thực bằng MFA và mã hóa/không lưu mật khẩu dạng thô. Về bối cảnh kỹ thuật – xã hội, việc số hóa quản lý vận hành giúp minh bạch chi phí,

giảm sai sót và nâng cao chất lượng dịch vụ cho cư dân; đồng thời hệ thống tôn trọng quyền riêng tư và tuân thủ quy định về sử dụng dữ liệu.

5.7. Kết luận chương

Chương 5 đã trình bày môi trường và kết quả triển khai, công tác kiểm thử cùng đánh giá tổng thể hệ thống trên các mặt chức năng, hiệu năng, bảo mật và đạo đức – an toàn dữ liệu. Kết quả cho thấy hệ thống TownHub đáp ứng tốt các mục tiêu đề ra.

Kết luận

1. Kết quả đạt được

Đề tài đã xây dựng thành công hệ thống quản lý vận hành tòa nhà TownHub theo kiến trúc module hóa (.NET 8, Next.js 16, PostgreSQL). Phần Base do cả nhóm phối hợp xây dựng cung cấp đầy đủ dịch vụ nền tảng (xác thực – phân quyền, người dùng, thông báo, cấu hình, báo cáo). Về phần cá nhân, tác giả đã phân tích, thiết kế và hiện thực hoàn chỉnh phân hệ Quản lý tài sản, gồm các nghiệp vụ: Tài sản & bảo trì (vòng đời tài sản, khấu hao đa phương pháp, mã QR, điều chuyển, bảo trì phòng ngừa, thanh lý, tích hợp kế toán); Kho – Mua sắm – Nhà cung cấp (tồn kho, quy trình PR→PO→hóa đơn, OCR hóa đơn bằng VietOCR, quản lý nhà cung cấp); và Quản lý sự cố & SLA (quy trình xử lý khép kín, giám sát SLA và leo thang tự động). Hệ thống chạy được, có giao diện và API hoàn chỉnh, được kiểm thử và triển khai bằng Docker/Fly.io.

2. Hạn chế

Bên cạnh kết quả đạt được, đề tài còn một số hạn chế: mô hình OCR hóa đơn cần thêm dữ liệu thực tế để nâng độ chính xác; một số báo cáo/thống kê nâng cao và tối ưu hiệu năng ở quy mô dữ liệu lớn chưa được kiểm chứng đầy đủ; ứng dụng di động cho cư dân/kỹ thuật viên mới ở mức cơ bản.

3. Hướng phát triển

Trong tương lai, hệ thống có thể mở rộng theo các hướng: cải thiện độ chính xác OCR và bổ sung trích xuất thông minh hơn; tích hợp thiết bị IoT để giám sát tình trạng thiết bị theo thời gian thực và dự báo bảo trì; bổ sung phân tích dữ liệu và cảnh báo thông minh (dự báo hỏng hóc, tối ưu tồn kho); hoàn thiện ứng dụng di động và mở rộng các phân hệ khác của toàn hệ thống.

4. Đóng góp của cá nhân

Trong khuôn khổ đề án nhóm, tác giả (Nguyễn Xuân Thường) chịu trách nhiệm chính phân hệ Quản lý tài sản - gồm các nghiệp vụ Tài sản & bảo trì, Kho – Mua sắm – Nhà cung cấp và Sự cố & SLA (thuộc module Asset), đồng thời tham gia xây dựng phần Base dùng chung cùng cả nhóm. Toàn bộ phân tích, thiết kế, hiện thực và kiểm thử của phân hệ nêu trên do tác giả trực tiếp thực hiện; đồng thời tác giả phối hợp với các thành viên khác trong việc thống nhất quy ước API, liên thông dữ liệu với hai phân hệ Quản lý dân cư và Quản lý dịch vụ, và review chéo khi hợp nhất mã nguồn. Sự phân định phân chung – phần riêng này thể hiện rõ đóng góp cá nhân theo Điều 7 Quy định DATN.

Tài liệu tham khảo

→ Trích dẫn chuẩn IEEE/APA, đánh số [1], [2]... khớp với chỗ trích trong bài.

- Microsoft, "ASP.NET Core documentation", learn.microsoft.com.
- Vercel, "Next.js Documentation", nextjs.org/docs.
- ...

Phụ lục

→ Nhật ký thực hiện ĐATN (Mẫu ĐATN-02); bảng phân công chi tiết; đặc tả API đầy đủ (Swagger); sơ đồ CSDL tổng thể (schema auth/asset/base); hướng dẫn cài đặt & sử dụng.

Phụ lục. Nhật ký thực hiện ĐATN (Mẫu ĐATN-02)

Mẫu ĐATN-02

**TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ
NỘI**
KHOA CÔNG NGHỆ THÔNG TIN

**CỘNG HOÀ XÃ HỘI CHỦ NGHĨA
VIỆT NAM**
Độc lập - Tự do - Hạnh phúc

Nhật ký thực hiện ĐATN

(Ban hành kèm theo Quyết định số /QĐ-ĐHXDHN ngày tháng năm 2026 của Hiệu trưởng Trường Đại học Xây dựng Hà Nội)

Họ và tên SV	Nguyễn Xuân Thường
Mã SV	
0220766	Xây dựng hệ thống quản lý vận hành tòa
Tên đề tài	nhà – TownHub
	Đợt
2026	ThS. Hoàng Nam Thắng
GVHD	Đơn vị
Khoa Công nghệ thông tin	(không có)
GV đồng hướng dẫn (nếu có)	

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/kỹ năng áp dụng	Vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Tuần 1:/..... -/.....							
Làm việc độc lập	~12 giờ	Đọc chương 1 học phần ĐATN; tìm hiểu bài toán quản lý vận hành tòa nhà; khảo sát yêu cầu các phân hệ Tài sản, Kho–Mua sắm, Sự cố.	Hoàn thành khảo sát và đặc tả yêu cầu; phác thảo kiến trúc phân hệ phụ trách.	Phân tích yêu cầu; kiến trúc phần mềm; .NET 8, Next.js.	Chuẩn hóa lại phạm vi từng phân hệ.	Cần bám sát chuẩn đầu ra học phần.	SV / GVHD
Làm việc theo nhóm	~8 giờ	Thống nhất kiến trúc tổng thể; phân chia module Base và phần riêng; thiết lập Git, quy ước code.	Khung dự án chạy được; phân công công việc rõ ràng.	Làm việc nhóm; Git; quản lý dự án.	Thống nhất chuẩn code và CSDL chung.	Cần phối hợp chặt chẽ hơn.	SV / GVHD
Tuần 2:/..... -/.....							
Làm việc độc lập	~14 giờ	Thiết kế CSDL schema asset (ERD); thiết kế API và phân quyền cho phân hệ Tài sản.	Hoàn thành ERD và bộ API tài sản; sinh sơ đồ bằng draw.io.	Thiết kế CSDL quan hệ; REST API; RBAC.	Rà soát ràng buộc khóa ngoại.	Cần đặt tên nhất quán.	SV / GVHD
Tuần 3:/..... -/.....							

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/kỹ năng áp dụng	Vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Làm việc độc lập	~14 giờ	Cài đặt vòng đời tài sản: ghi tăng, điều chuyển, thanh lý; khấu hao tự động; sinh & in mã QR.	Các chức năng tài sản chạy được; khấu hao định kỳ tự động.	EF Core; job nền; sinh mã QR.	Kiểm thử các phương pháp khấu hao.	Cần thêm ràng buộc nghiệp vụ.	SV / GVHD
Tuần 4:/..... -/.....							
Làm việc độc lập	~14 giờ	Cài đặt bảo trì phòng ngừa (PM): lịch bảo trì, lệnh công việc (WO), checklist, nghiệm thu.	Quy trình PM và Work Order hoạt động; đính kèm ảnh minh chứng.	Máy trạng thái; hàng đợi; lịch cron.	Chuẩn hóa trạng thái WO.	Cần tối ưu truy vấn danh sách.	SV / GVHD
Tuần 5:/..... -/.....							
Làm việc độc lập	~14 giờ	Cài đặt Kho – Mua sắm – NCC: tồn kho, cảnh báo tồn thấp, quy trình PR → PO, quản lý NCC/hợp đồng.	Quy trình mua sắm nhiều cấp chạy được; cảnh báo tồn kho.	Thiết kế quy trình duyệt; giao dịch CSDL.	Rà soát luồng duyệt nhiều cấp.	Cần bổ sung nhật ký thao tác.	SV / GVHD
Tuần 6:/..... -/.....							

Loại công việc	Thời gian làm việc	Nhiệm vụ được giao	Kết quả thực hiện	Kiến thức/kỹ năng áp dụng	Vấn đề cần bổ sung, chỉnh sửa	Tự đánh giá & rút kinh nghiệm	Chữ ký
Làm việc độc lập	~14 giờ	Tích hợp OCR hóa đơn: sinh dataset hóa đơn tổng hợp; huấn luyện & đánh giá VietOCR, PaddleOCR; nối dịch vụ OCR với backend.	Dịch vụ OCR chạy; PaddleOCR val-acc 99,64%; có bảng so sánh 3 engine.	AI/OCR; PyTorch/Paddle; FastAPI; hàng đợi nền.	Cần thêm hóa đơn thật để cải thiện phát hiện.	Đánh giá khách quan giữa các engine.	SV / GVHD
Tuần 7:/..... -/.....							
Làm việc độc lập	~14 giờ	Cài đặt Quản lý sự cố & SLA: tạo ticket, phân công KTV, giám sát và leo thang SLA, dashboard.	Phân hệ sự cố hoạt động; theo dõi SLA thời gian thực.	Sự kiện/thông báo; đo lường SLA; realtime.	Chuẩn hóa mức ưu tiên và SLA.	Cần bổ sung báo cáo thống kê.	SV / GVHD
Tuần 8:/..... -/.....							
Làm việc độc lập	~14 giờ	Kiểm thử và tích hợp toàn hệ thống; sửa lỗi; viết thuyết minh DATN; dựng sơ đồ (draw.io); chuẩn bị bảo vệ.	Hệ thống chạy ổn định; hoàn thành quyền thuyết minh và biên bản kiểm thử.	Kiểm thử; viết tài liệu; trình bày.	Hoàn thiện định dạng theo mẫu DATN.	Cần luyện trình bày bảo vệ.	SV / GVHD

Ghi chú: sinh viên điền mốc thời gian từng tuần và ký xác nhận cùng GVHD.